

Timing analysis of critical real-time systems



Safety- and time-critical systems



Safety

"freedom from those conditions that can cause death, injury, illness, damage or loss of equipment or property, or environmental harm »

Software Quality Assurance Handbook, 4th ed., 2008

- Safety is not reliability (probability that a system will perform its intended function satisfactorily)
- Safety is not security (protection or defense against attack, interference, or espionage)

Safety- and time-critical systems



Safety-critical software

"software whose use in a system can result in unacceptable risk »

IEEE Standard Glossary of Software Engineering Terminology, 1990

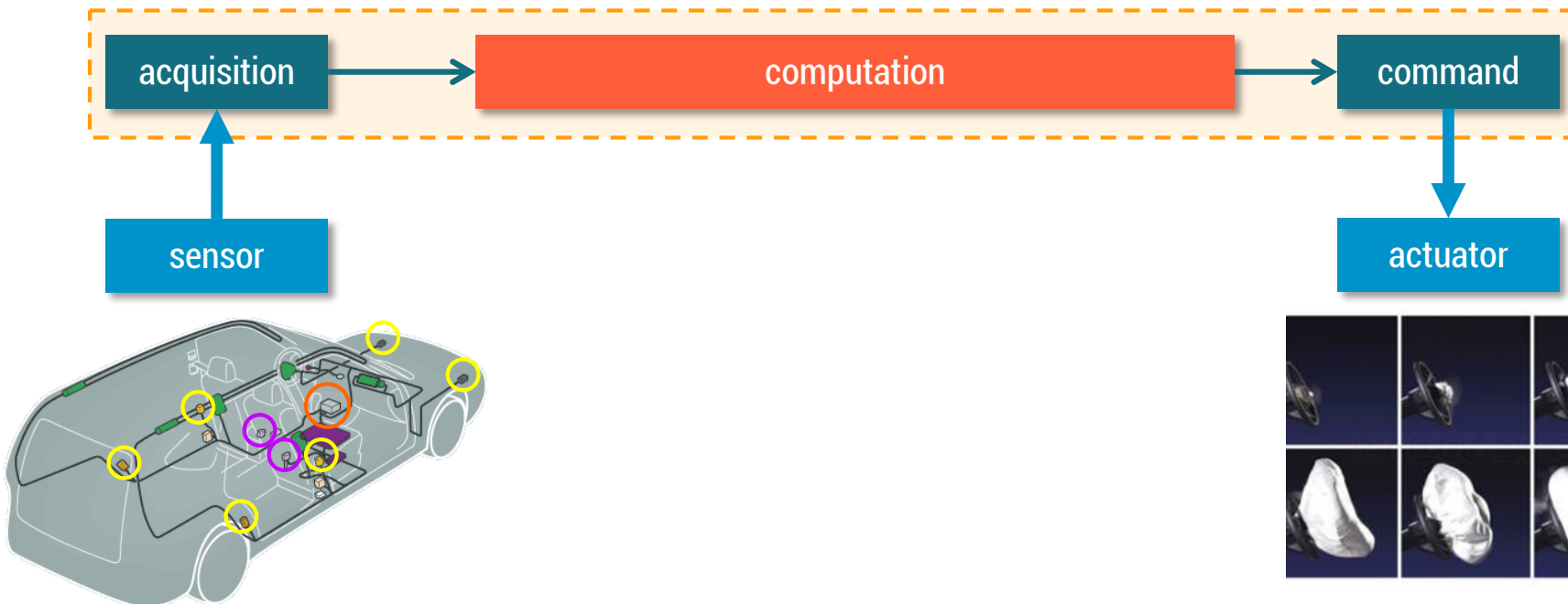
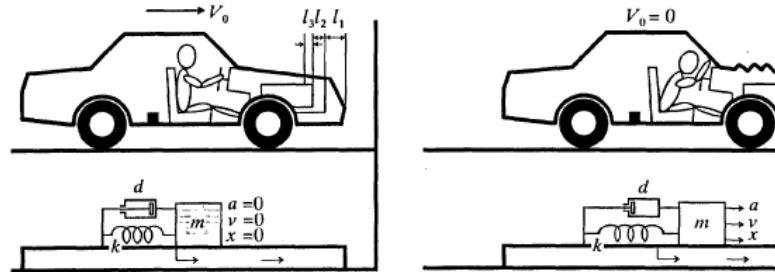
- requires failure mitigation
- possible failures:
 - hardware
 - software
 - functional
 - timing

Real-time systems:

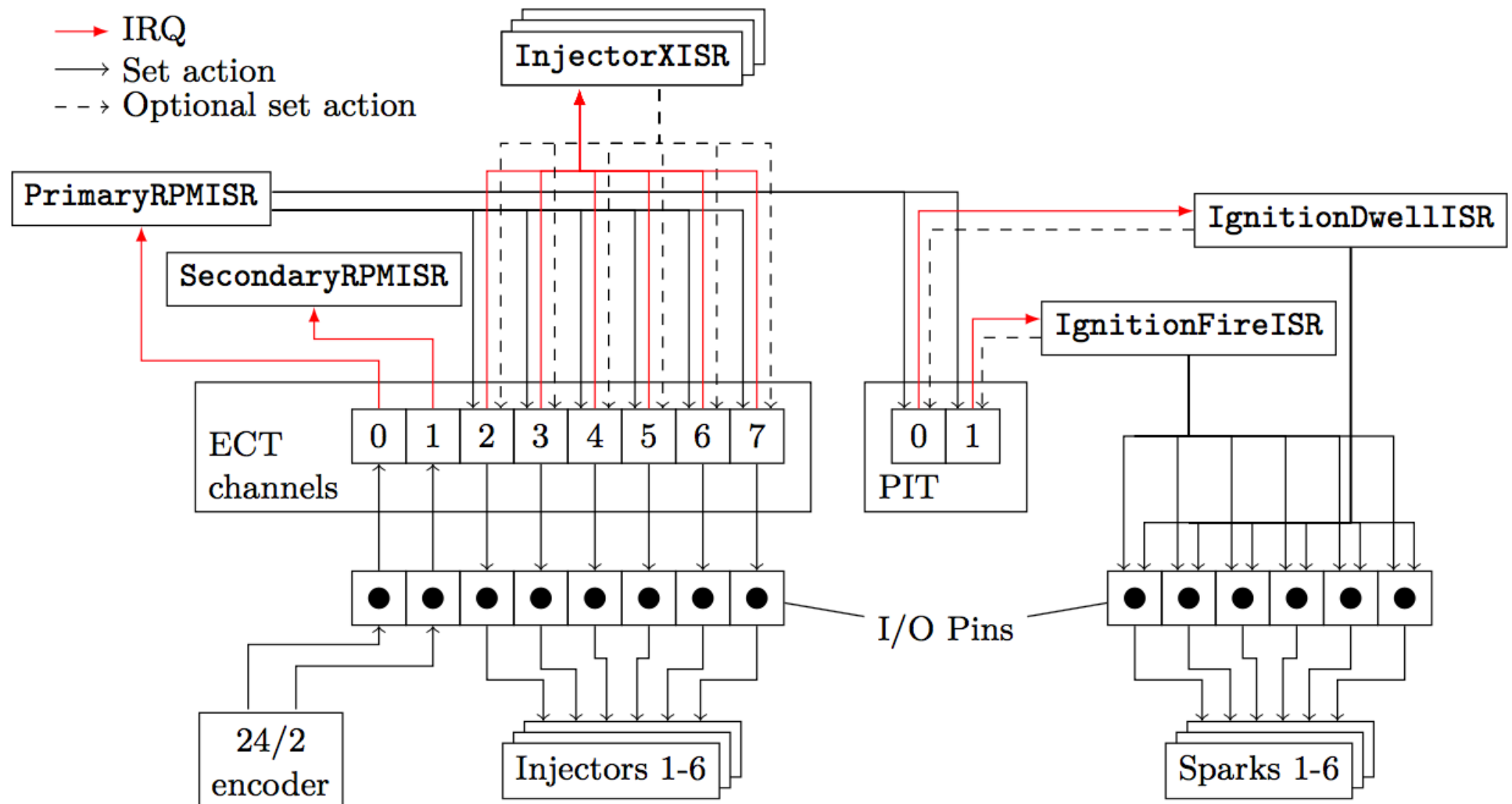
systems that are subject to timing deadlines

- missing a deadline is a failure

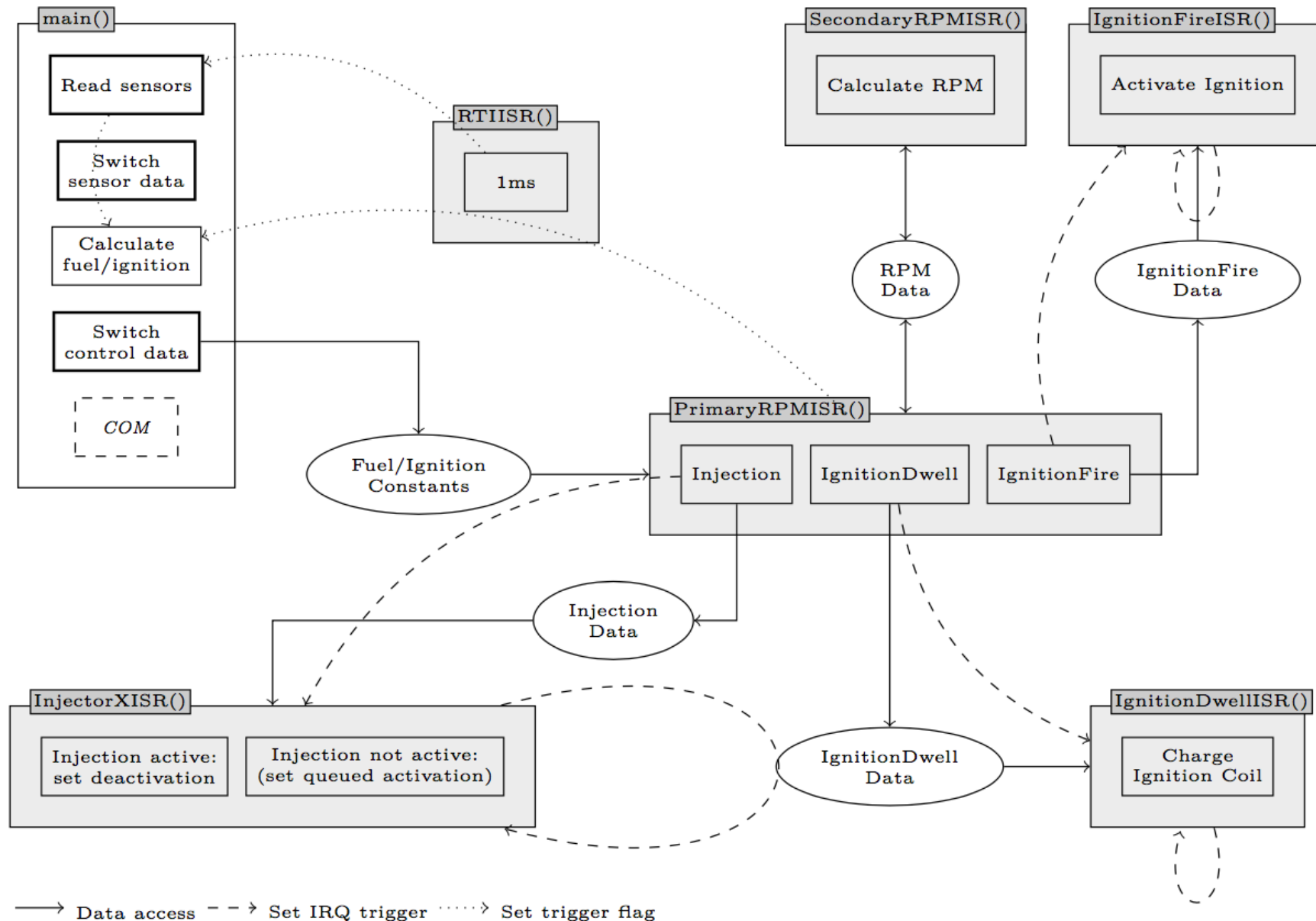
Example 1: airbag controller



Example 2: car engine controller



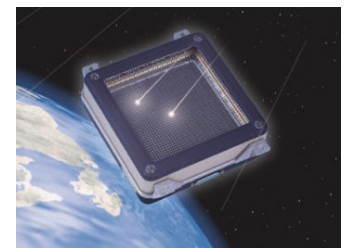
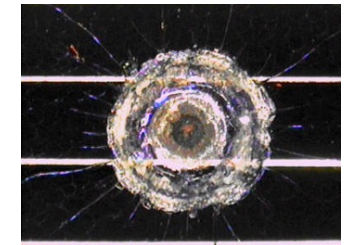
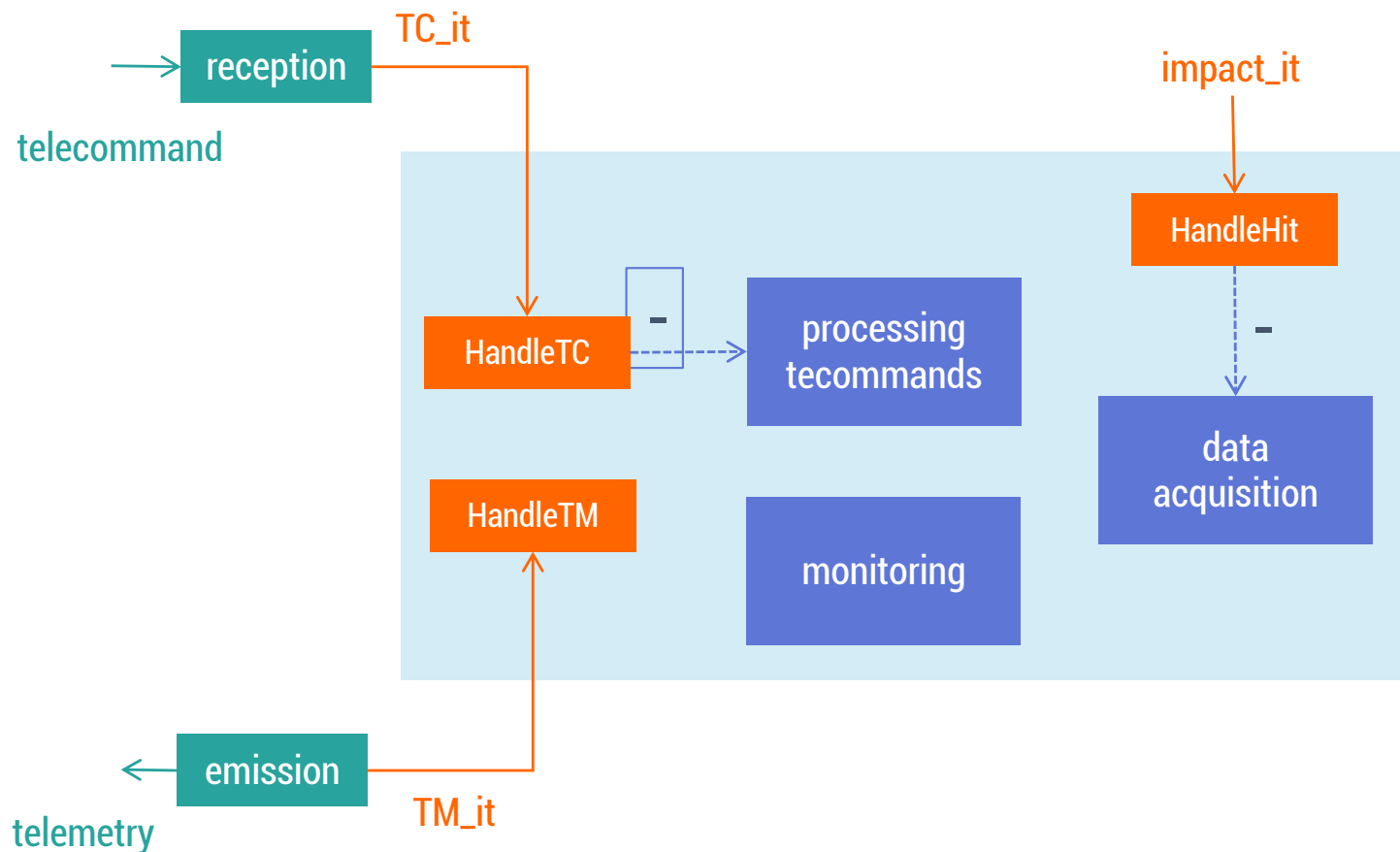
Example 2: car engine controller



Example 3: a device in aerospace



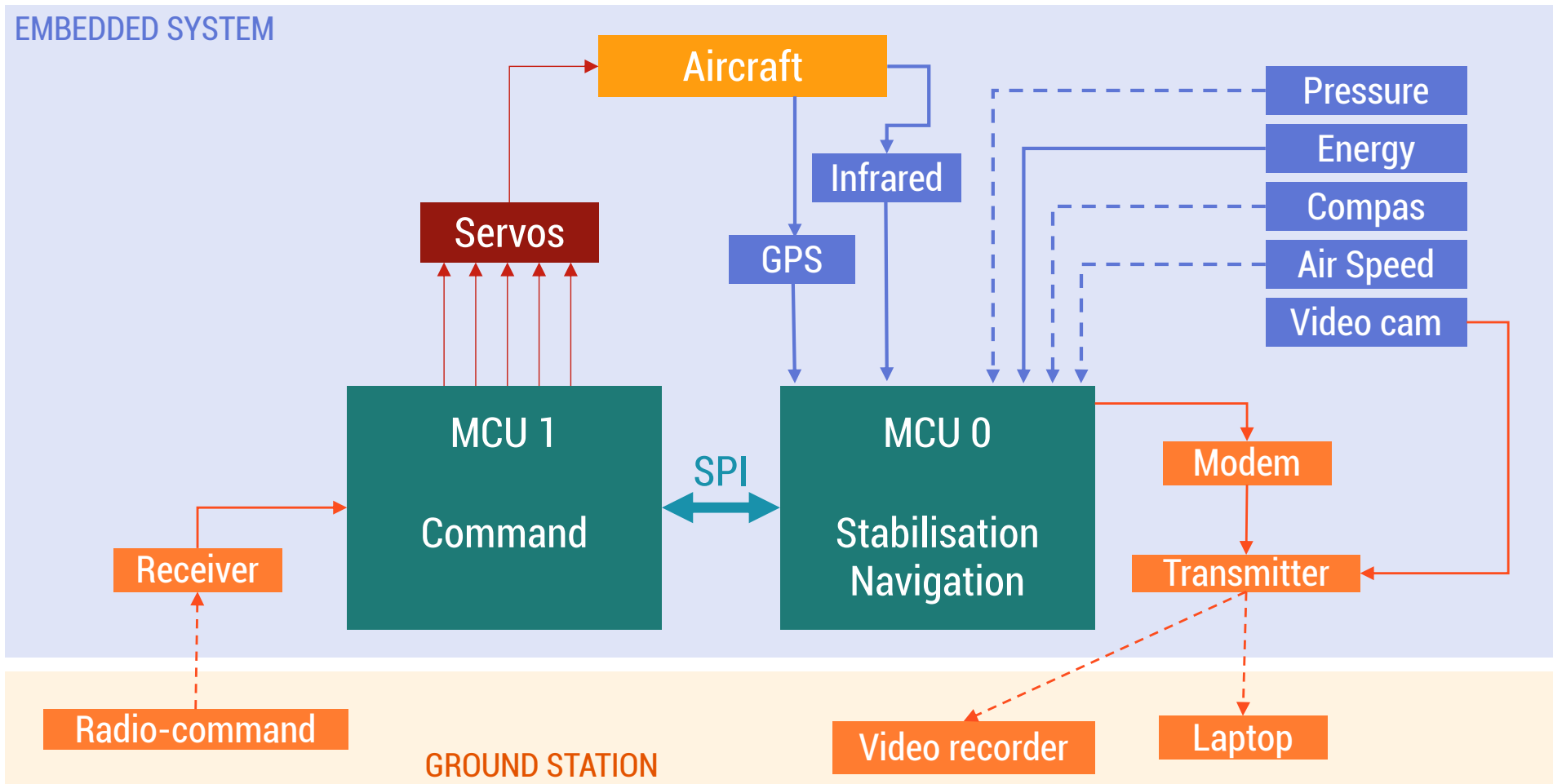
DEBIE: DEBris In orbit Evaluator



Example 4: UAV software



PapaBench (inspired from the Paparazzi project)



Example 4: UAV software



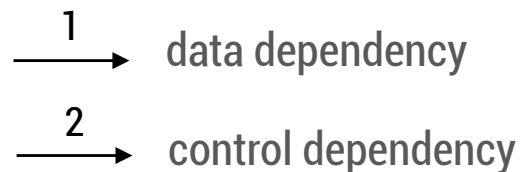
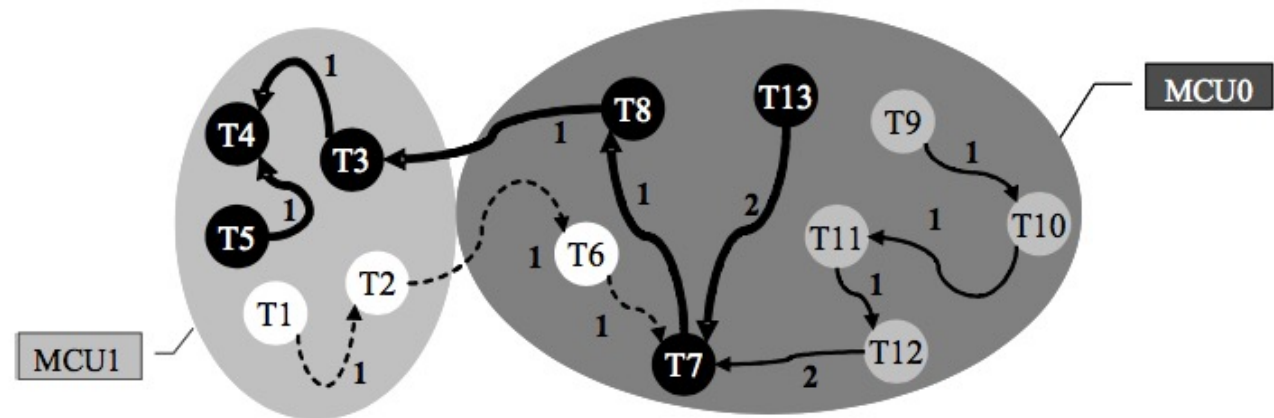
PapaBench (inspired from the Paparazzi project)

<i>ID</i>	<i>Description</i>	<i>Frequency</i>
T1	Receive Radio-Command orders	40Hz
T2	Send Data to MCU0	40Hz
T3	Receive MCU0 values	20Hz
T4	Transmit Servos	20Hz
T5	Check Failsafe	20Hz
I1	Transmission Servos interrupt	-
I2	SPI interrupt of MCU1	-
I3	Radio interrupt	-

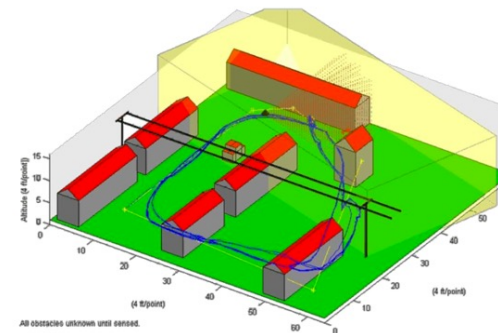
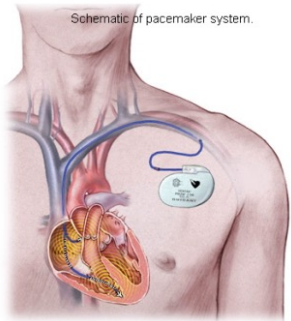
Table 1: MCU1 tasks and interrupts

<i>ID</i>	<i>Description</i>	<i>Frequency</i>
T6	Managing Radio orders	40Hz
T7	Stabilization	20Hz
T8	Send Data to MCU1	20Hz
T9	Receive GPS Data	4Hz
T10	Navigation	4Hz
T11	Altitude Control	4Hz
T12	Climb Control	4Hz
T13	Reporting Task	10Hz
I4	SPI interrupt of MCU0	-
I5	Modem interrupt	-
I6	GPS interrupt	-

Table 2: MCU0 tasks and interrupts



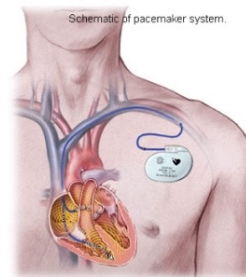
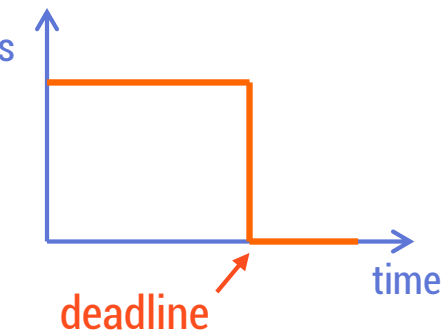
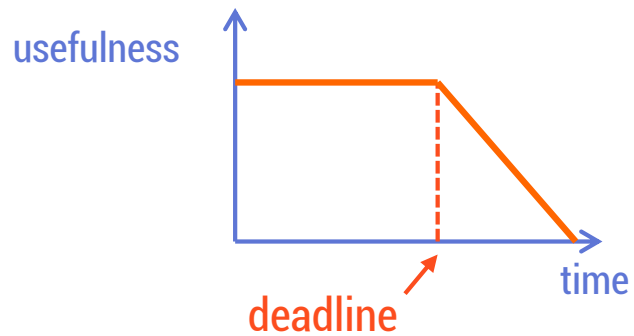
Time-critical applications



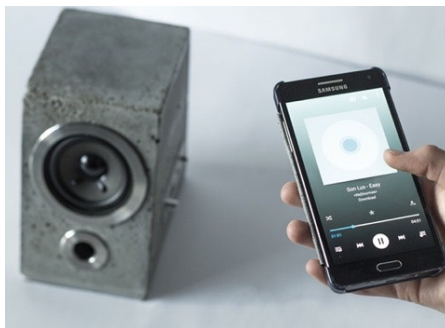
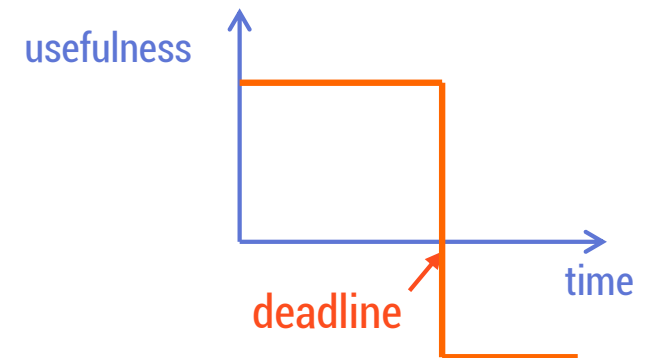
Time criticality



soft real-time



hard real-time

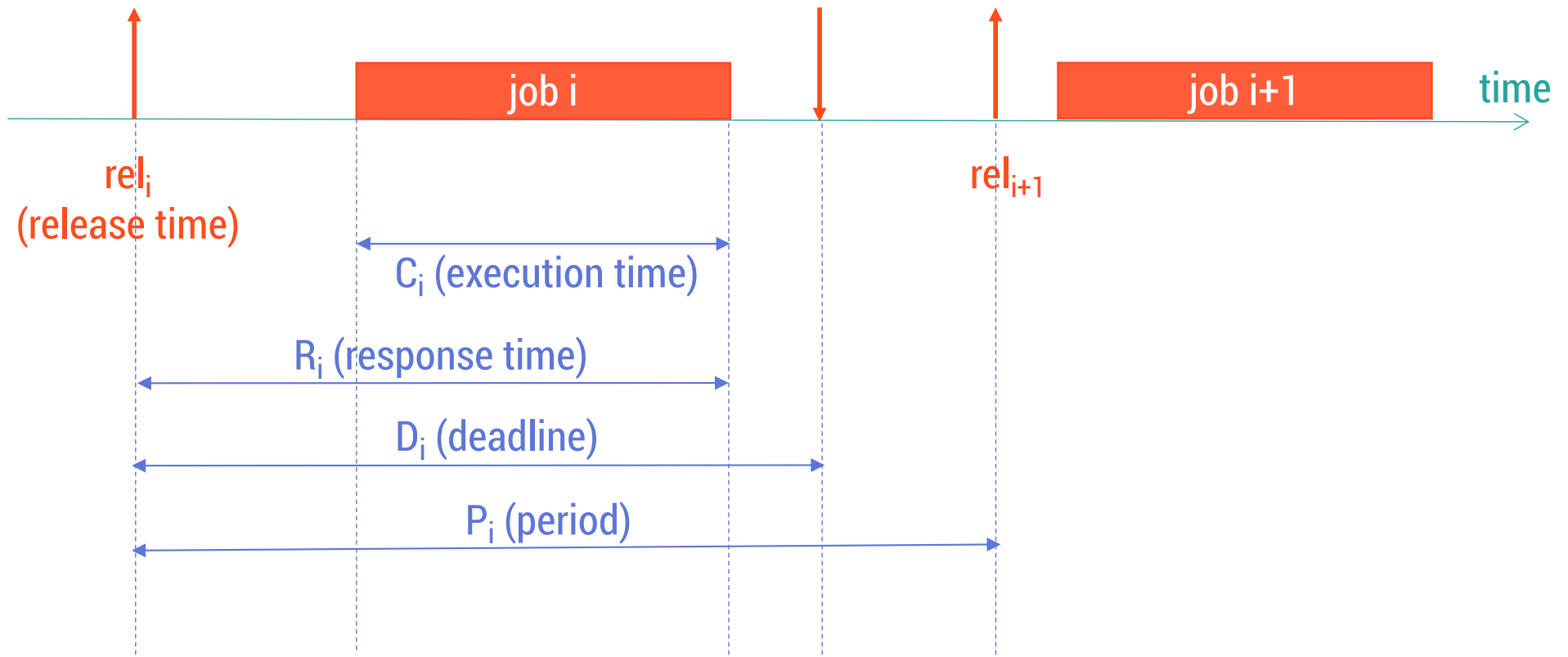


Real-time scheduling



Task model

- periodic, sporadic, aperiodic



Real-time scheduling



Objective

- allocating shared resources to tasks so that they all meet their deadlines

Approaches

- off-line vs. online
- fixed vs. dynamic priorities
 - Rate Monotonic, Deadline Monotonic, Earliest Deadline First
- preemptive vs. non-preemptive
- on a multicore platform, global vs. partitioned

Real-time scheduling



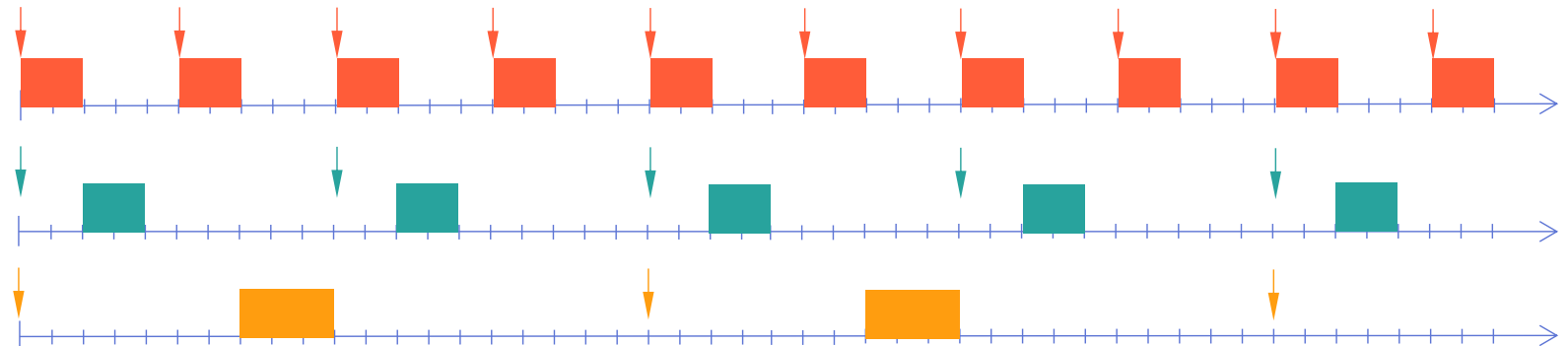
Schedulability test

- example: non-preemptive EDF (Earliest Deadline First)

$$\sum_{i=1}^n \frac{C_i}{P_i} \leq 1$$

$$C_i + \sum_{j=1}^{i-1} \left\lfloor \frac{L-1}{P_j} \right\rfloor \times C_j \leq L \quad \text{with} \quad 1 < i \leq n \quad \text{and} \quad P_1 < L \leq P_i$$

	C_i	$P_i=D_i$
τ_1	2	5
τ_2	2	10
τ_3	3	20



Variability of execution times



Reasons for variability

- input data

```
index = 0;  
while (data_in[index] != 0){  
    /* processing */  
}
```

how many
iterations?

```
if (temperature > T_MAX){  
    /* error processing */  
}  
else{  
    /* normal processing */  
}
```

which branch?

- initial hardware state

```
sum = 0;  
for (i=0 ; i<100 ; i++)  
    sum += a[i];
```

data cache
contents?

- interferences with other tasks

on a miss, latency to
the main memory?

Execution time: example 1



What is the execution time of this program?

source code

```
sum = 0;
num_even = 0;
for (i=0 ; i<100 ; i++)
{
    sum += i;
    if (i % 2 == 0)
        num_even++;
}
```

compiled code

```
        mov    r0,#0        @ r0:sum
        mov    r1,#0        @ r1:num_even
        mov    r2,#0        @ r2:i
for:     cmp    r2,#100
        bcs    end
        add    r0,r0,r2
if:      and    r3,r2,#1
        cmp    r3,#0
        bne    next
        add    r1,r1,#1
next:    add    r2,r2,#1
        b      for
end:
```

Assumptions:

- each instruction is executed in 1 cycle
 - 3 cycles if it follows a taken branch
- no pipeline

Execution time: example 1



What is the execution time of this program?

	mov	r0,#0	}	3 cycles	× 1	→	3
	mov	r1,#0					
	mov	r2,#0					
for:	cmp	r2,#100	}	2 cycles	× 1	→	2
	bcs	end		4 cycles	× 100	→	400
	add	r0,r0,r2	}	4 cycles	× 100	→	400
if:	and	r3,r2,#1					
	cmp	r3,#0					
	bne	next					
	add	r1,r1,#1	}	1 cycle	× 50	→	50
next:	add	r2,r2,#1					
	b	for	}	4 cycles	× 50	→	200
				2 cycles	× 50	→	100
end:							
							1155 cycles

Execution time: example 2



What is the execution time of this program?

source code

```
sum = 0;
num_even = 0;
for (i=0 ; i<N ; i++)
{
    sum += a[i];
    if (a[i] % 2 == 0)
        num_even ++;
}
```

compiled code

```
mov    r0,#0        @ r0:sum
mov    r1,#0        @ r1:num_even
mov    r2,#0        @ r2:i
adr    r10,a
adr    r11,N
ldr    r11,[r11] @ r11:N
for:   cmp    r2,r11
      bcs    end
      ldr    r4,[r10,r2,LSL #2]
      add    r0,r0,r4
if:    and    r3,r4,#1
      cmp    r3,#0
      bne    next
      add    r1,r1,#1
next:  add    r2,r2,#1
      b      for
end:
```

Assumptions:

- each instruction is executed in 1 cycle
 - 3 cycles if it follows a taken branch
- no pipeline

Execution time: example 2



What is the execution time of this program?

	mov	r0,#0	}	6 cycles	× 1	
	mov	r1,#0				
	mov	r2,#0				
	adr	r10,a				
	adr	r11,N				
	ldr	r11,[r11]				
for:	cmp	r2,r11	}	2 cycles	× 1	N
	bcs	end		4 cycles	× ???	
	ldr	r4,[r10,r2,LSL #2]	}	5 cycles	× ???	N
if:	add	r0,r0,r4				
	and	r3,r4,#1				
	cmp	r3,#0				
	bne	next				
	add	r1,r1,#1	}	1 cycle	× ???	depend on a[] (at most N)
next:	add	r2,r2,#1		}	2 cycles	
	b	for	4 cycles			
end:						

Execution time: example 2



Can we estimate the maximum execution time of the program?

- we need to know the maximum value of N
- the iteration counts of loops must be upper-bounded

Assumptions:

- $N \leq 10$
 - in the worst case, the loop has 10 iterations
- values of $a[]$?
 - we will consider the most unfavourable situation (see code)

Execution time: example 2



What is the (maximum) execution time of this program?

				maximum execution time	
	mov r0,#0	{	6 cycles	× 1	6
	mov r1,#0				
	mov r2,#0				
	adr r10,tab				
	adr r11,N				
	ldr r11,[r11]	{	2 cycles	× 1	2
for:	cmp r2,r11				
	bcs end		4 cycles	× 10 (max)	40
	ldr r4,[r10,r2,LSL #2]	{	5 cycles	× 10 (max)	50
if:	add r0,r0,r4				
	and r3,r4,#1				
	cmp r3,#0				
	bne next	{	1 cycle	× 10 (max)	40
next:	add r1,r1,#1				
	add r2,r2,#1	{	2 cycles	× 10 (max)	40
	b for		4 cycles		
end:					138

Execution time: example 2



Possible values of the execution time?

- for a given value of N , and n even array elements ($0 \leq n \leq N$):

$$T = 6 + 2 + 4 \times N + 5 \times N + 3 \times n + 4 \times (N - n)$$

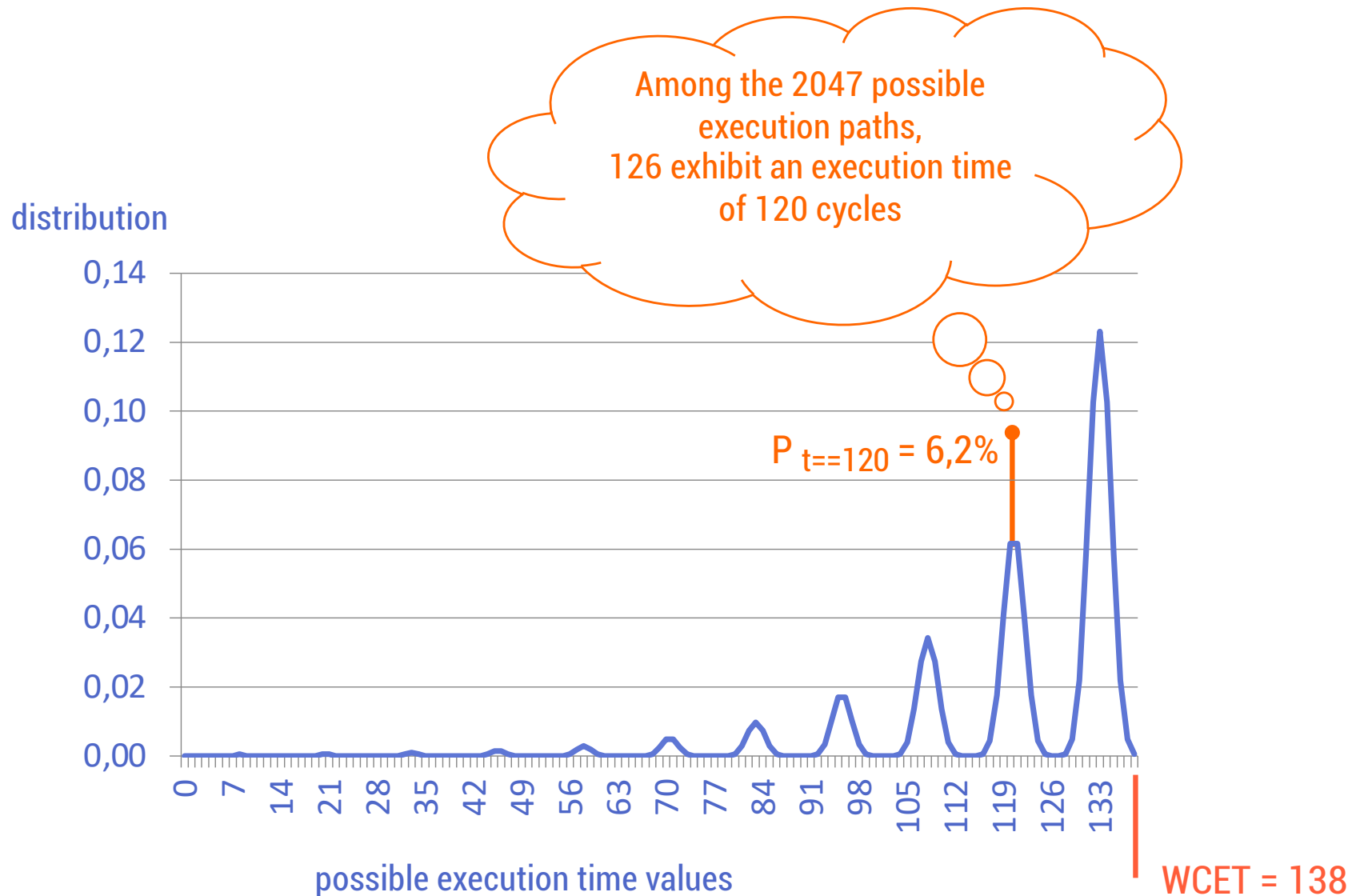
$$T = 8 + 13 \times N - n$$

Examples :

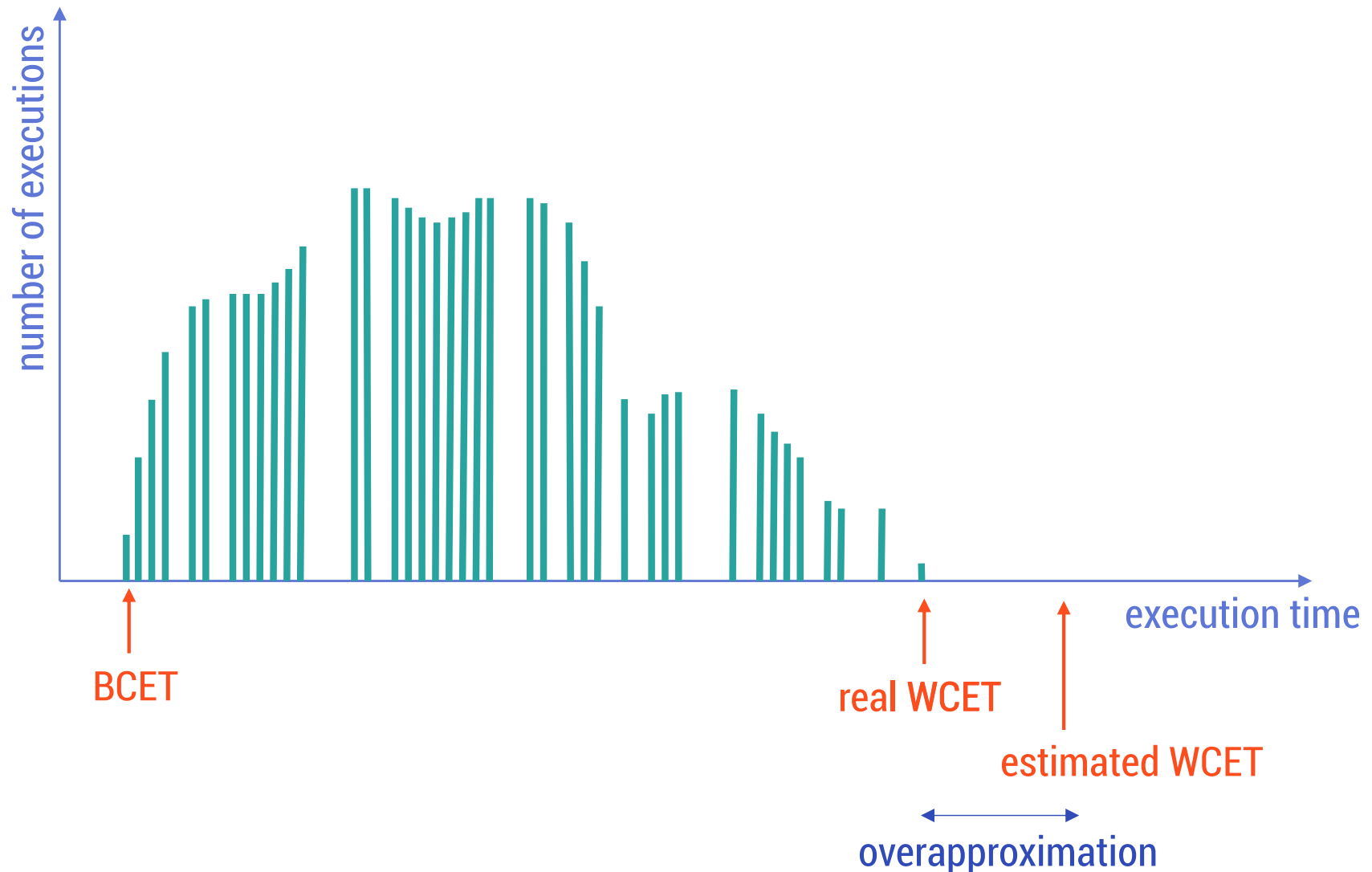
- $N = 5$ et $n = 2$ $T = 71$ 10 possible execution paths
- $N = 8$ et $n = 4$ $T = 108$ 70 possible paths
- $N = 10$ et $n = 10$ $T = 128$ a single path
- $N = 10$ et $n = 0$ $T = 138$ a single path

11000
10100
10010
10001
01100
01010
01001
00110
00101
00011

Execution time: example 2



Worst-Case Execution Time (WCET)



Outline of the course



An overview of timing analysis

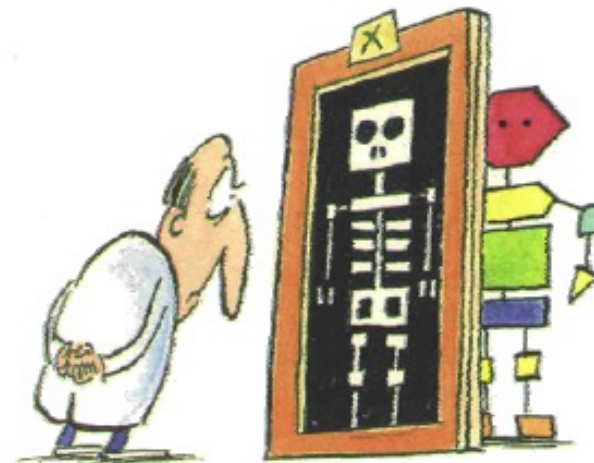
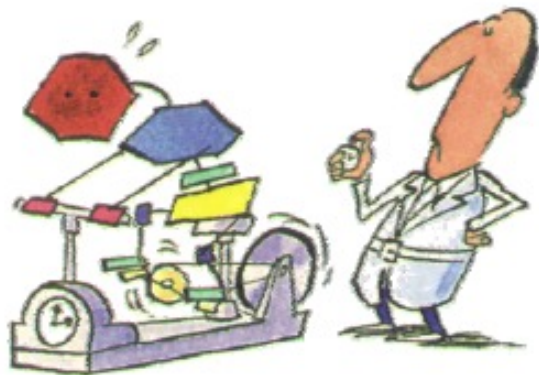
- techniques, tools

Static WCET analysis

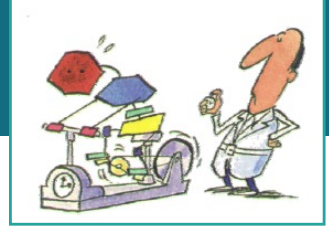
- for a task running in isolation
- for a task that is subject to external interferences

Approaches towards better timing predictability

Approaches to WCET analysis



Measurements



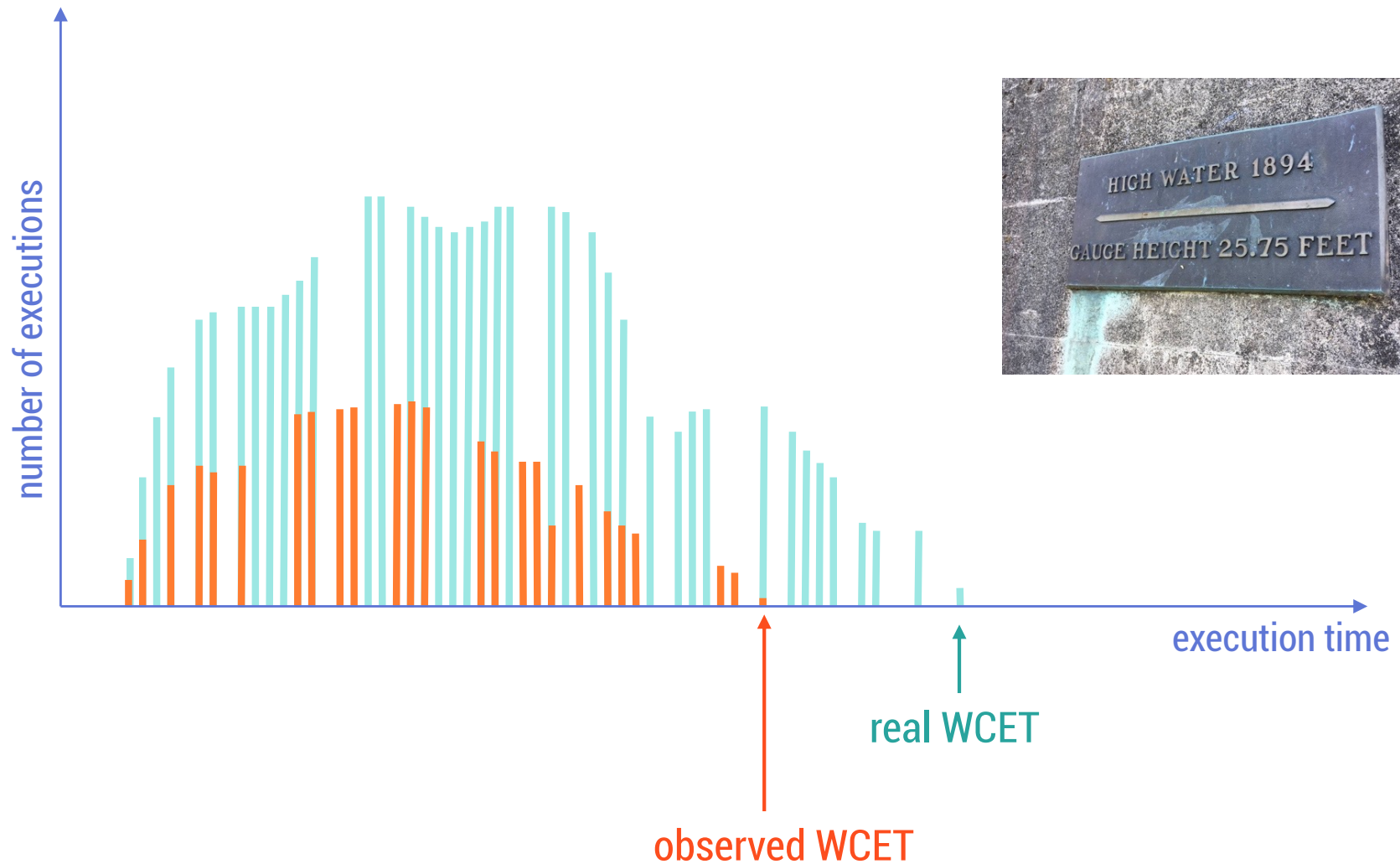
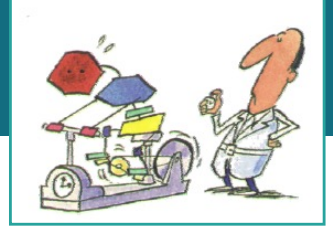
On the target hardware

- requires precise hardware timing facilities

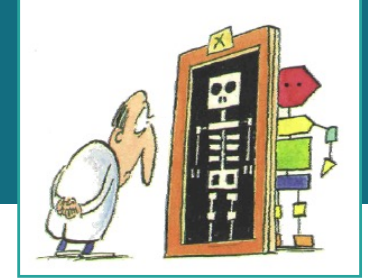
Inputs?

- worst-case input vector?
 - generally unknown
- all possible input vectors?
 - too many combinations
- a set of input vectors that cover all possible execution paths?
 - not always computable, and may be too large
 - safe?
- a set of input vectors?
 - with adding a safety margin?
 - safe?

Measurements

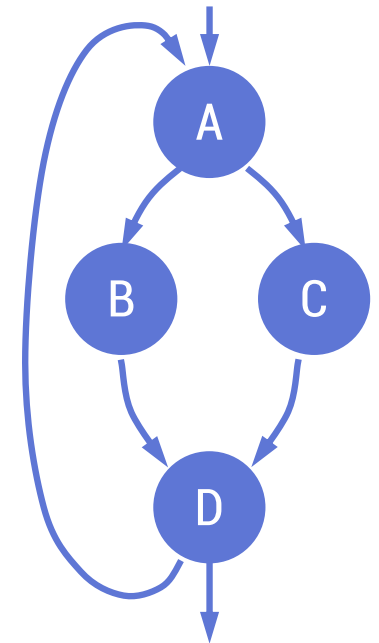


Static analysis

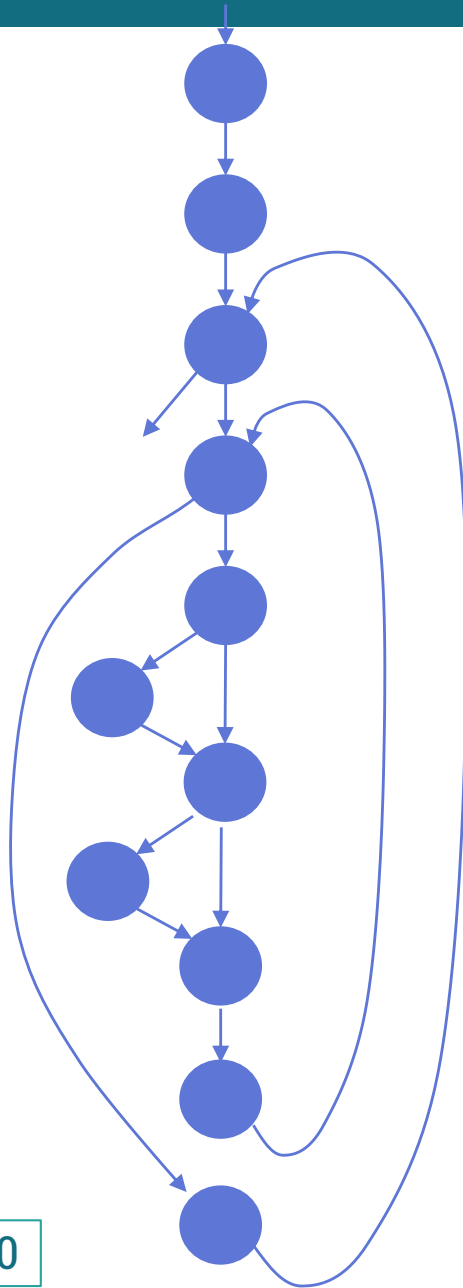
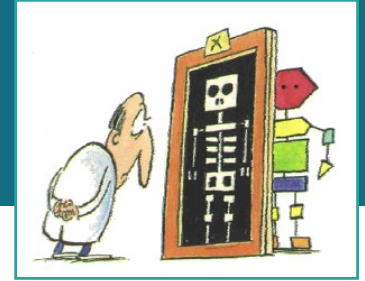


Representation of the code as a Control Flow Graph (CFG)

- The binary code is split into basic blocks (nodes)
 - a sequence of instructions
 - a single entry point
 - first instruction in the basic block
 - a single exit point
 - last instruction in the basic block
- Directed edges
 - express that two basic blocks can be executed in sequence



Three steps



```
for (int i=0 ; i<64 ; i++)  
  for (int j=0 ; j<i ; j++){  
    if (a[i][j] > MAX)  
      a[i][j] = MAX;  
    if (a[i][j] < MIN)  
      a[i][j] = MIN;  
  }
```

1. Flow analysis

- loop bounds, infeasible paths

2. Low-level/timing/microarchitectural analysis

- WCET of basic blocks

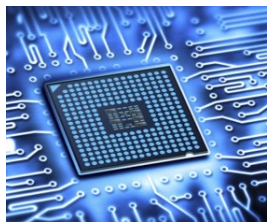
3. WCET computation

Timing/low-level analysis



The main focus of this part of the course

- the objective is to determine the (worst-case) execution times/costs of basic blocks
- based on a model of the target hardware architecture



```
mov r0,#1  
mov r1,#0  
add r2,r15,#60  
add r3,r15,#64  
ldr r3,[r2]
```

basic block

low-level
analysis

WCET of the
basic block

Determining the longest path (IPET)



Integer linear program

- global execution time

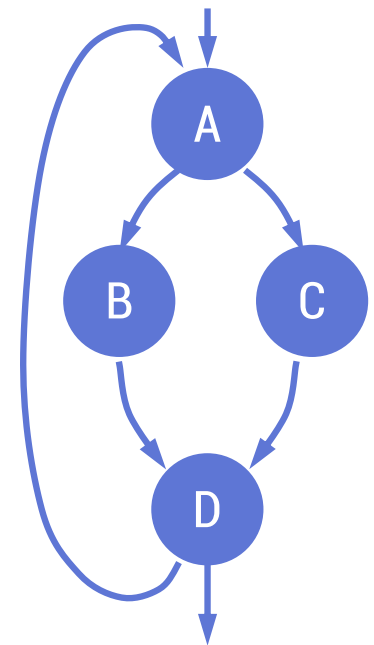
- $T = \sum x_i \cdot t_i$

execution time on
a given path

execution time of block i (known)

execution count of block i
on that path (unknown)

- maximize T to determine the WCET
 - compute the x_i values that maximize T
 - requires specifying constraints on x_i



Determining the longest path (IPET)



Constraints on the x_i

- structural constraints:

execution count of a basic block

= sum of execution counts on input edges

= sum of execution counts on output edges

$$x_A = 1 + x_{DA}$$

$$x_A = x_{AB} + x_{AC}$$

$$x_C = x_{AC}$$

$$x_C = x_{CD}$$

$$x_B = x_{AB}$$

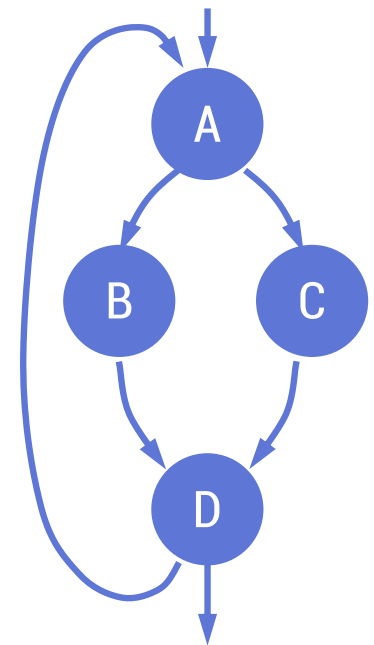
$$x_B = x_{BD}$$

$$x_D = x_{BD} + x_{CD}$$

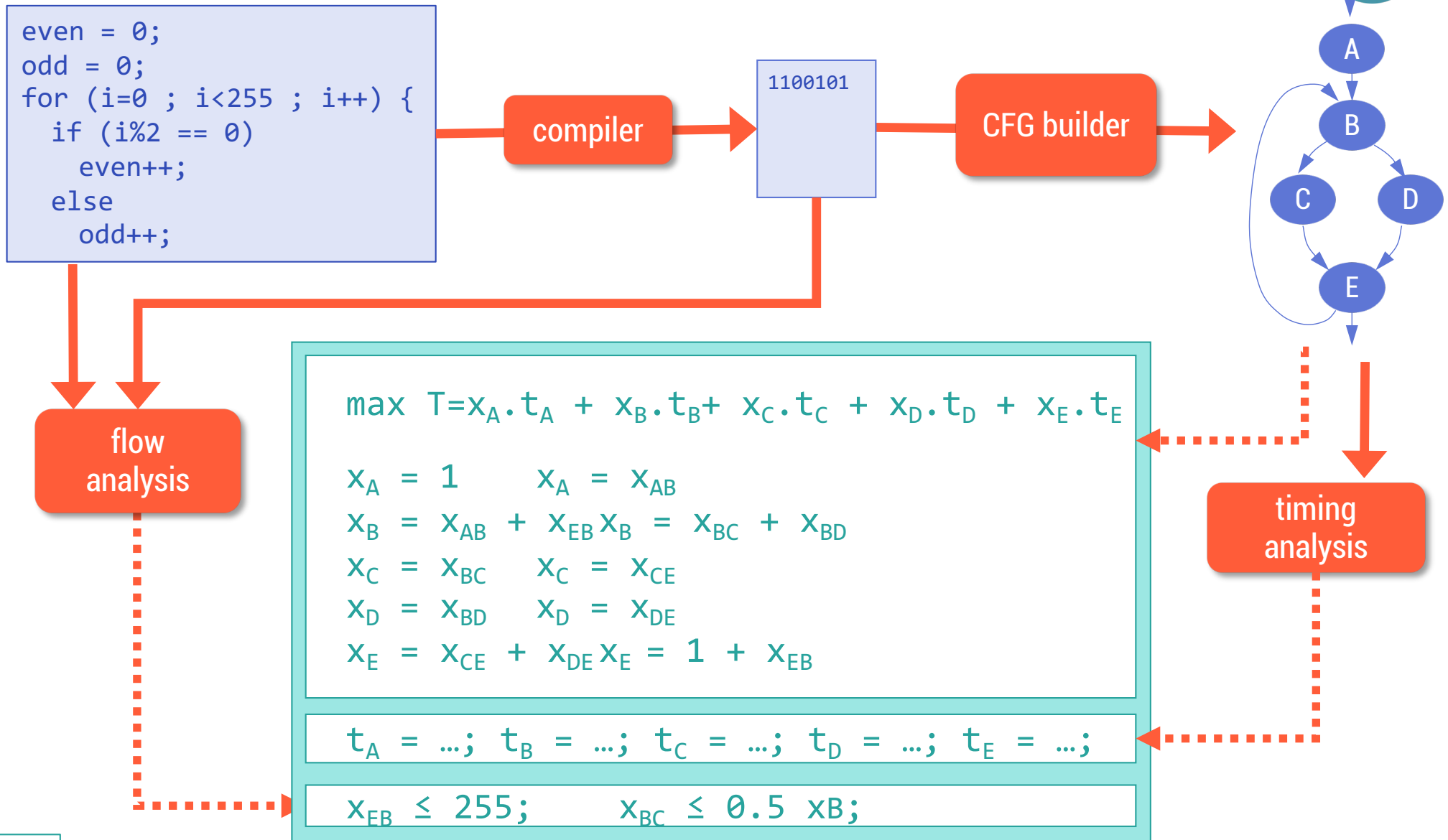
$$x_D = 1 + x_{DA}$$

- flow constraints:

$$x_{DA} \leq N$$



The Implicit Path Enumeration Technique (IPET)



The Implicit Path Enumeration Technique (IPET)



$$\max T = x_A \cdot t_A + x_B \cdot t_B + x_C \cdot t_C + x_D \cdot t_D + x_E \cdot t_E$$

$$x_A = 1 \quad x_A = x_{AB}$$

$$x_B = x_{AB} + x_{EB} \quad x_B = x_{BC} + x_{BD}$$

$$x_C = x_{BC} \quad x_C = x_{CE}$$

$$x_D = x_{BD} \quad x_D = x_{DE}$$

$$x_E = x_{CE} + x_{DE} \quad x_E = 1 + x_{EB}$$

$$t_A = \dots; \quad t_B = \dots; \quad t_C = \dots; \quad t_D = \dots; \quad t_E = \dots;$$

$$x_{EB} \leq 255; \quad x_{BC} \leq 0.5 \cdot x_B;$$

ILP
solver

$T_{\max} = \dots$ (WCET)

with:

$$x_A = \dots$$

$$x_B = \dots$$

$$x_C = \dots$$

$$x_D = \dots$$

$$x_E = \dots$$

Hybrid WCET analysis



Combines measurements and static analysis

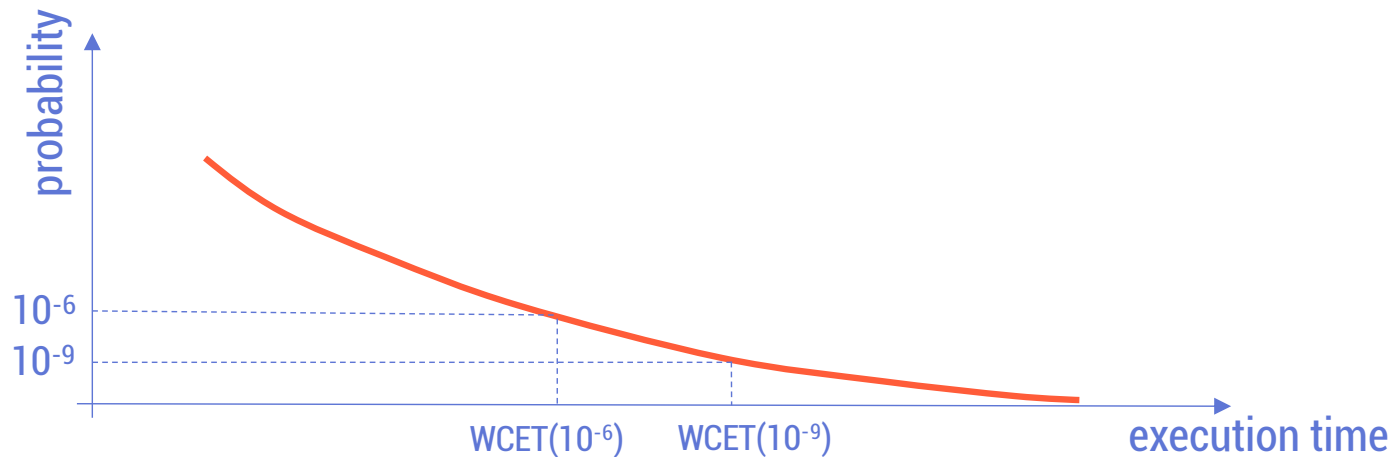
- instrumentation code (ipoints) → timestamped traces
 - (address1, timestamp1)
 - (address2, timestamp2)
 - ...
 - used to estimate the local WCETs of basic blocks
- combines the advantages and drawbacks of measurements and static analysis
- instrumentation compatible with certification?

Probabilistic approaches



Basic idea

- produce several WCET estimates with associated probabilities
- compute the probability of a timing failure (the WCET exceeds a given value)



Techniques

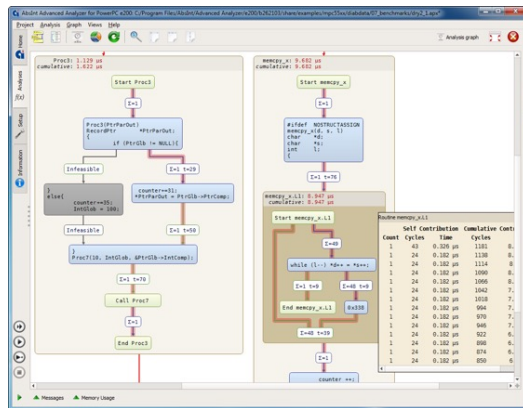
- randomized hardware
- static or measurement-based

WCET Analysis Tools

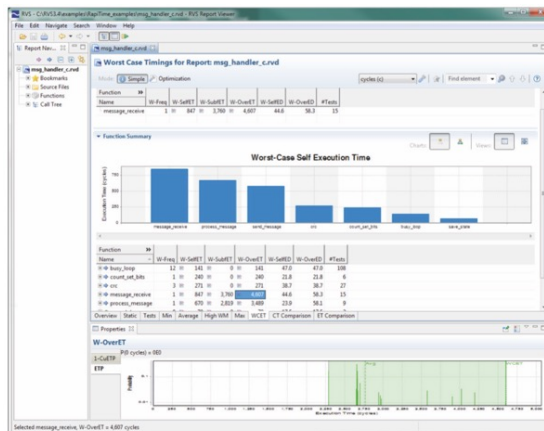


Commercial tools:

- aiT 

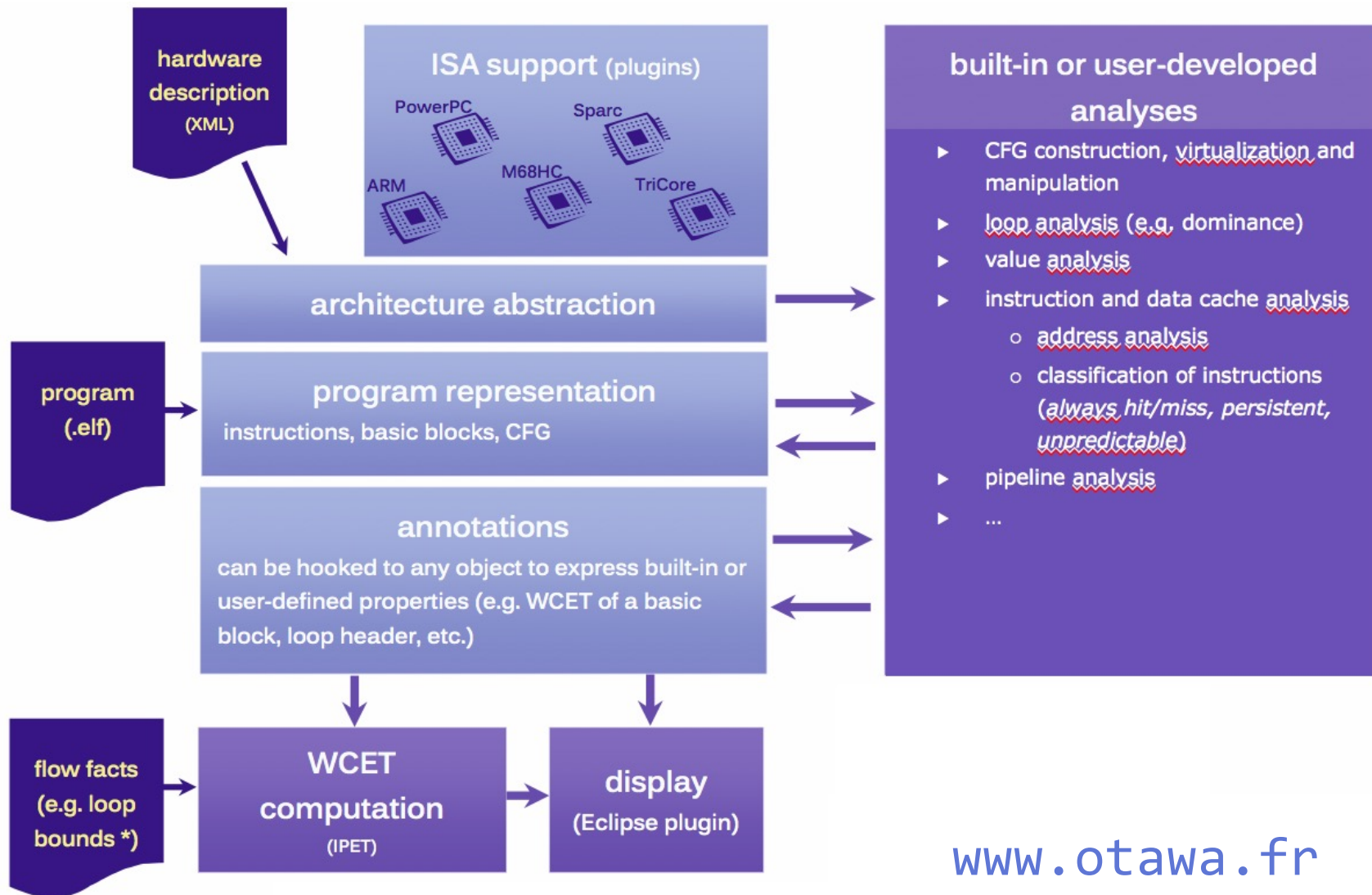


-  RapiTime 



Academic tools:

- Chronos (Singapour)
- Heptane (Rennes)
- KTA (Stockholm)
- OTAWA (Toulouse)
- SWEET (Mälardalen)





OTAWA - test-nonmanaged/crc.c - Eclipse Platform

File Edit Source Refactor Navigate Search Run Project Window Help

Workspace Platform

main/00008470/icrc/00008124/icrc1 CFG [main]

crc.c Makefile

main Call Graph [main]

```
#define HIBYTE(x) ((uchar)((x) >> 8))
unsigned char lin[256] = "asdfgeagewahAFEFaeDsF
unsigned short icrc1(unsigned short crc, unsigned short jinit, int jrev)
{
    int i;
    unsigned short ans=(crc^onech << 8);
    for (i=0;i<8;i++) {
        if (ans & 0x8000)
            ans = (ans <= 1) ^ 4129;
        else
            ans <= 1;
    }
    return ans;
}
unsigned short icrc(unsigned short crc, unsigned short jinit, int jrev)
{
    unsigned short icrc1(unsigned short crc, unsigned short jinit, int jrev);
    static unsigned short icrcb[256],init=0;
    static uchar rchr[256];
```

Task

Name	Value
WCET	115254 cycles (230,508 μ s)
Configuration	LPC213x
Flow Facts	
Unresolved controls	

Properties

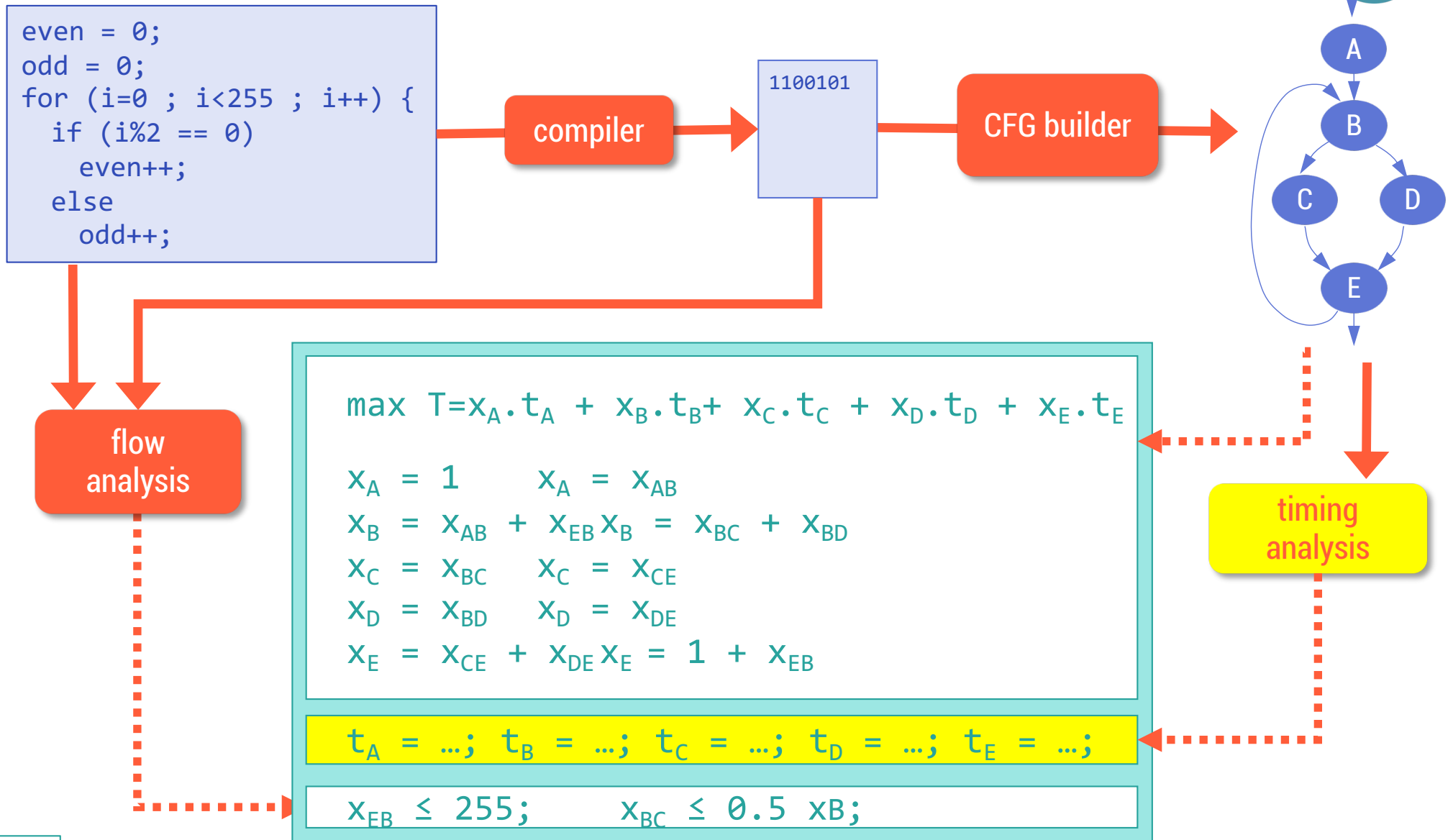
Name	Type	Value
Selection		
BasicBlock - 00008124		
Total Time		18432 cycles
Percent		15,99%
Overall Total Time		18432 cycles
Overall Percent		15,99%
time	int32	9
execution count	int32	2048
otawa::INDEX	int32	26

Writable Smart Insert 68 : 1

Timing analysis of a task running in isolation



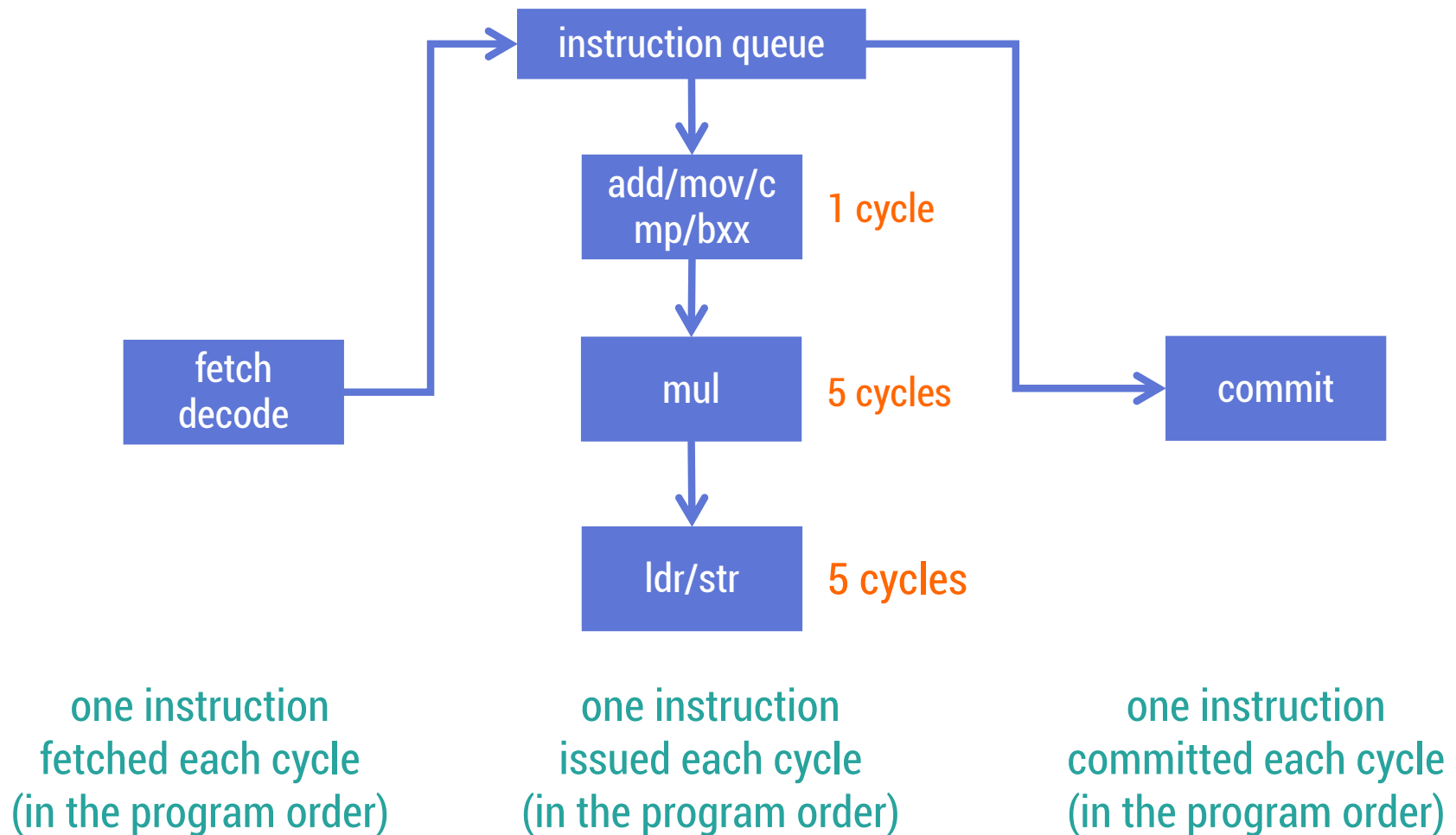
What do we want to compute?



Pipelined execution



Running example: execution pipeline



Running example: task code



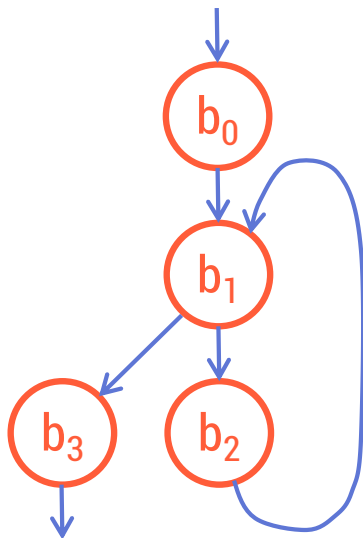
```
int product = 1;
int init_val;
for (int i=0; i<N ; i++){
    product *= i;
    a[i] = init_val;
}
```

```
__start:    mov r0,#1    @ product
            mov r1,#0    @ i
            adr r2,a
            adr r3,init_val
            ldr r3,[r3]
while:      mul r0,r1,r0
            cmp r1,#N
            bhs end
            str r3,[r2],#4
            add r1,r1,#1
            b while
end:        adr r2,product
            str r0,[r2]
```

Running example: task code



```
int product = 1;
int init_val;
for (int i=0; i<N ; i++){
    product *= i;
    a[i] = init_val;
}
```



```
__start:  mov r0,#1    @ product
          mov r1,#0    @ i
          adr r2,a
          adr r3,init_val
          ldr r3,[r3]
```

b₀

```
while:    mul r0,r1,r0
          cmp r1,#N
          bhs end
```

b₁

```
          str r3,[r2],#4
          add r1,r1,#1
          b while
```

b₂

```
end:      adr r2,product
          str r0,[r2]
```

b₃

Early approaches



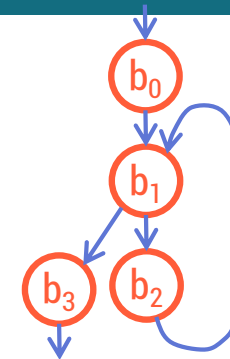
0a	__start:	mov r0,#1
0b		mov r1,#0
0c		adr r2,a
0d		adr r3,init_val
0e		ldr r3,[r3]
1a	while:	mul r0,r1,r0
1b		cmp r1,#N
1c		bhs end
2a		str r3,[r2],#4
2b		add r1,r1,#1
2c		b while
3a	end:	adr r2,product
3b		str r0,[r2]

b_1 after b_0

F	0a																		
A																			
Mu																			
Me																			
C																			

b_1 after b_2

F																			
A																			
Mu																			
Me																			
C																			



Early approaches



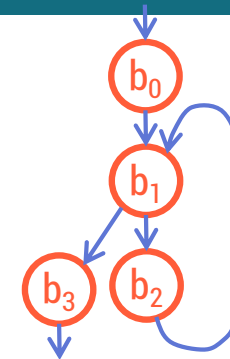
0a	__start:	mov r0,#1
0b		mov r1,#0
0c		adr r2,a
0d		adr r3,init_val
0e		ldr r3,[r3]
1a	while:	mul r0,r1,r0
1b		cmp r1,#N
1c		bhs end
2a		str r3,[r2],#4
2b		add r1,r1,#1
2c		b while
3a	end:	adr r2,product
3b		str r0,[r2]

b_1 after b_0

F	0a	0b																	
A		0a																	
Mu																			
Me																			
C																			

b_1 after b_2

F																			
A																			
Mu																			
Me																			
C																			



Early approaches



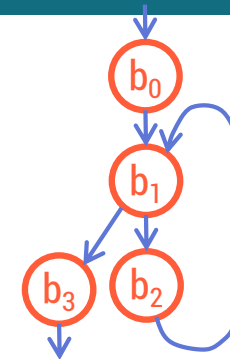
0a	__start:	mov r0,#1
0b		mov r1,#0
0c		adr r2,a
0d		adr r3,init_val
0e		ldr r3,[r3]
1a	while:	mul r0,r1,r0
1b		cmp r1,#N
1c		bhs end
2a		str r3,[r2],#4
2b		add r1,r1,#1
2c		b while
3a	end:	adr r2,product
3b		str r0,[r2]

b_1 after b_0

F	0a	0b	0c															
A		0a	0b															
Mu																		
Me																		
C			0a															

b_1 after b_2

F																		
A																		
Mu																		
Me																		
C																		



Early approaches



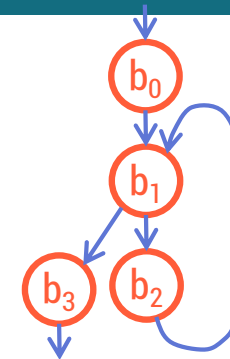
0a	__start:	mov r0,#1
0b		mov r1,#0
0c		adr r2,a
0d		adr r3,init_val
0e		ldr r3,[r3]
1a	while:	mul r0,r1,r0
1b		cmp r1,#N
1c		bhs end
2a		str r3,[r2],#4
2b		add r1,r1,#1
2c		b while
3a	end:	adr r2,product
3b		str r0,[r2]

b_1 after b_0

F	0a	0b	0c	0d													
A		0a	0b	0c													
Mu																	
Me																	
C			0a	0b													

b_1 after b_2

F																	
A																	
Mu																	
Me																	
C																	



Early approaches



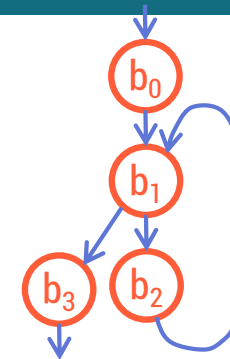
0a	__start:	mov r0,#1
0b		mov r1,#0
0c		adr r2,a
0d		adr r3,init_val
0e		ldr r3,[r3]
1a	while:	mul r0,r1,r0
1b		cmp r1,#N
1c		bhs end
2a		str r3,[r2],#4
2b		add r1,r1,#1
2c		b while
3a	end:	adr r2,product
3b		str r0,[r2]

b_1 after b_0

F	0a	0b	0c	0d	0e												
A		0a	0b	0c	0d												
Mu																	
Me																	
C			0a	0b	0c												

b_1 after b_2

F																	
A																	
Mu																	
Me																	
C																	



Early approaches



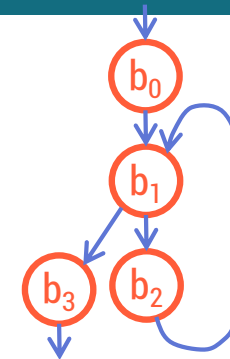
0a	__start:	mov r0,#1
0b		mov r1,#0
0c		adr r2,a
0d		adr r3,init_val
0e		ldr r3,[r3]
1a	while:	mul r0,r1,r0
1b		cmp r1,#N
1c		bhs end
2a		str r3,[r2],#4
2b		add r1,r1,#1
2c		b while
3a	end:	adr r2,product
3b		str r0,[r2]

b_1 after b_0

F	0a	0b	0c	0d	0e	1a												
A		0a	0b	0c	0d													
Mu																		
Me						0e												
C			0a	0b	0c	0d												

b_1 after b_2

F																		
A																		
Mu																		
Me																		
C																		



Early approaches



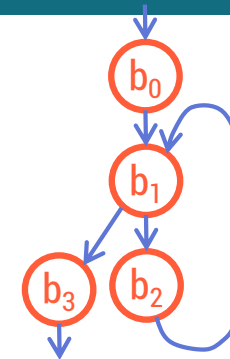
0a	__start:	mov r0,#1
0b		mov r1,#0
0c		adr r2,a
0d		adr r3,init_val
0e		ldr r3,[r3]
1a	while:	mul r0,r1,r0
1b		cmp r1,#N
1c		bhs end
2a		str r3,[r2],#4
2b		add r1,r1,#1
2c		b while
3a	end:	adr r2,product
3b		str r0,[r2]

b_1 after b_0

F	0a	0b	0c	0d	0e	1a	1b										
A		0a	0b	0c	0d												
Mu							1a										
Me						0e	0e										
C			0a	0b	0c	0d											

b_1 after b_2

F																	
A																	
Mu																	
Me																	
C																	



Early approaches



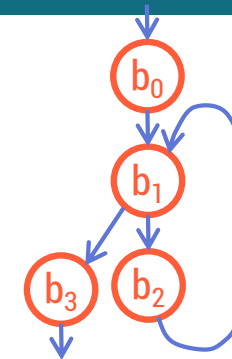
0a	__start:	mov r0,#1
0b		mov r1,#0
0c		adr r2,a
0d		adr r3,init_val
0e		ldr r3,[r3]
1a	while:	mul r0,r1,r0
1b		cmp r1,#N
1c		bhs end
2a		str r3,[r2],#4
2b		add r1,r1,#1
2c		b while
3a	end:	adr r2,product
3b		str r0,[r2]

b_1 after b_0

F	0a	0b	0c	0d	0e	1a	1b	1c									
A		0a	0b	0c	0d			1b									
Mu							1a	1a									
Me						0e	0e	0e									
C			0a	0b	0c	0d											

b_1 after b_2

F																	
A																	
Mu																	
Me																	
C																	



Early approaches



```

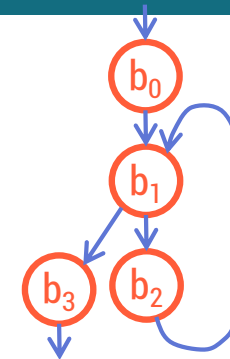
0a  __start:  mov r0,#1
0b             mov r1,#0
0c             adr r2,a
0d             adr r3,init_val
0e             ldr r3,[r3]
1a  while:   mul r0,r1,r0
1b             cmp r1,#N
1c             bhs end
2a             str r3,[r2],#4
2b             add r1,r1,#1
2c             b while
3a  end:      adr r2,product
3b             str r0,[r2]
    
```

b_1 after b_0

F	0a	0b	0c	0d	0e	1a	1b	1c									
A		0a	0b	0c	0d			1b	1c								
Mu							1a	1a	1a								
Me						0e	0e	0e	0e								
C			0a	0b	0c	0d											

b_1 after b_2

F																	
A																	
Mu																	
Me																	
C																	



Early approaches



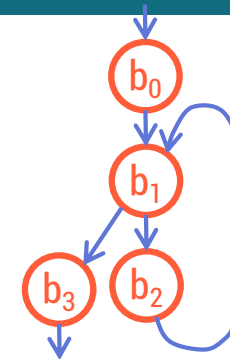
0a	__start:	mov r0,#1
0b		mov r1,#0
0c		adr r2,a
0d		adr r3,init_val
0e		ldr r3,[r3]
1a	while:	mul r0,r1,r0
1b		cmp r1,#N
1c		bhs end
2a		str r3,[r2],#4
2b		add r1,r1,#1
2c		b while
3a	end:	adr r2,product
3b		str r0,[r2]

b_1 after b_0

F	0a	0b	0c	0d	0e	1a	1b	1c									
A		0a	0b	0c	0d			1b	1c								
Mu							1a	1a	1a	1a							
Me						0e	0e	0e	0e	0e							
C			0a	0b	0c	0d											

b_1 after b_2

F																	
A																	
Mu																	
Me																	
C																	



Early approaches



```

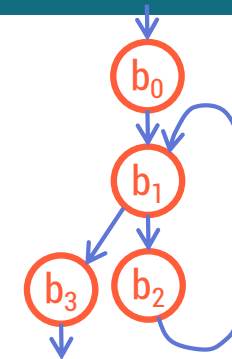
0a  __start:  mov r0,#1
0b              mov r1,#0
0c              adr r2,a
0d              adr r3,init_val
0e              ldr r3,[r3]
1a  while:   mul r0,r1,r0
1b              cmp r1,#N
1c              bhs end
2a              str r3,[r2],#4
2b              add r1,r1,#1
2c              b while
3a  end:      adr r2,product
3b              str r0,[r2]
    
```

b_1 after b_0

F	0a	0b	0c	0d	0e	1a	1b	1c									
A		0a	0b	0c	0d			1b	1c								
Mu							1a	1a	1a	1a	1a						
Me						0e	0e	0e	0e	0e							
C			0a	0b	0c	0d					0e						

b_1 after b_2

F																	
A																	
Mu																	
Me																	
C																	



Early approaches



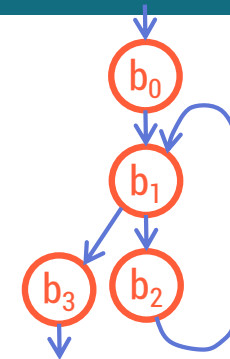
0a	__start:	mov r0,#1
0b		mov r1,#0
0c		adr r2,a
0d		adr r3,init_val
0e		ldr r3,[r3]
1a	while:	mul r0,r1,r0
1b		cmp r1,#N
1c		bhs end
2a		str r3,[r2],#4
2b		add r1,r1,#1
2c		b while
3a	end:	adr r2,product
3b		str r0,[r2]

b_1 after b_0

F	0a	0b	0c	0d	0e	1a	1b	1c									
A		0a	0b	0c	0d			1b	1c								
Mu							1a	1a	1a	1a	1a						
Me						0e	0e	0e	0e	0e							
C			0a	0b	0c	0d					0e						

b_1 after b_2

F																	
A																	
Mu																	
Me																	
C																	



Early approaches



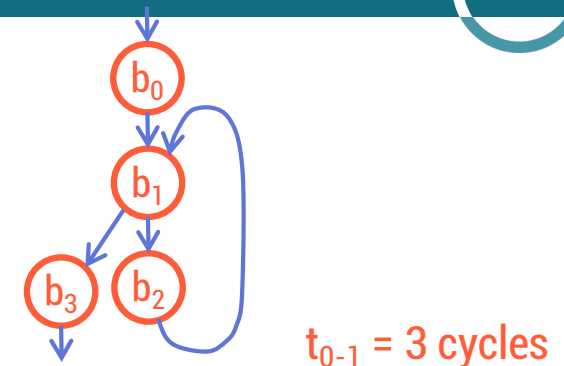
0a	__start:	mov r0,#1
0b		mov r1,#0
0c		adr r2,a
0d		adr r3,init_val
0e		ldr r3,[r3]
1a	while:	mul r0,r1,r0
1b		cmp r1,#N
1c		bhs end
2a		str r3,[r2],#4
2b		add r1,r1,#1
2c		b while
3a	end:	adr r2,product
3b		str r0,[r2]

b_1 after b_0

F	0a	0b	0c	0d	0e	1a	1b	1c										
A		0a	0b	0c	0d			1b	1c									
Mu							1a	1a	1a	1a	1a							
Me						0e	0e	0e	0e	0e								
C			0a	0b	0c	0d					0e	1a	1b	1c				

b_1 after b_2

F																		
A																		
Mu																		
Me																		
C																		



Early approaches



```

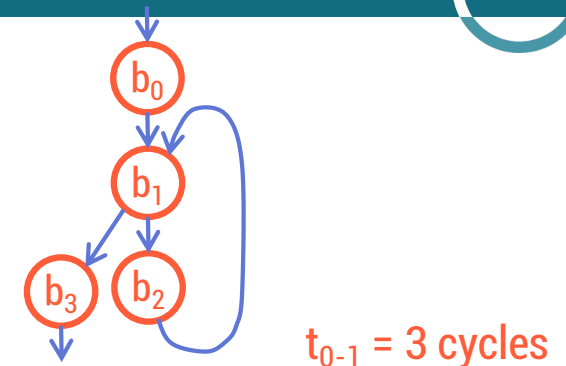
0a  __start:  mov r0,#1
0b              mov r1,#0
0c              adr r2,a
0d              adr r3,init_val
0e              ldr r3,[r3]
1a  while:   mul r0,r1,r0
1b              cmp r1,#N
1c              bhs end
2a              str r3,[r2],#4
2b              add r1,r1,#1
2c              b while
3a  end:      adr r2,product
3b              str r0,[r2]
    
```

b_1 after b_0

F	0a	0b	0c	0d	0e	1a	1b	1c									
A		0a	0b	0c	0d			1b	1c								
Mu							1a	1a	1a	1a	1a						
Me						0e	0e	0e	0e	0e							
C			0a	0b	0c	0d					0e	1a	1b	1c			

b_1 after b_2

F	2a	2b	2c	x	1a	1b	1c										
A			2b	2c			1b	1c									
Mu						1a	1a	1a	1a	1a							
Me		2a	2a	2a	2a	2a											
C							2a	2b	2c		1a	1b	1c				



Improved ILP formulation (IPET)



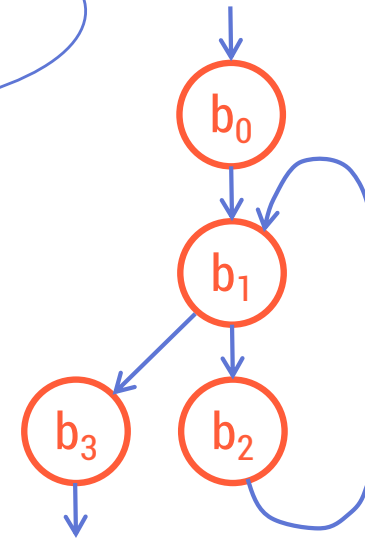
- max $T = \sum x_{i-j} \cdot t_{i-j}$

computed using
reservation tables

- with:

- $x_0 = x_{0-1} = 1$
- $x_1 = x_{0-1} + x_{2-1}$
- $x_1 = x_{1-3} + x_{1-2}$
- $x_2 = x_{1-2}$
- $x_2 = x_{2-1}$

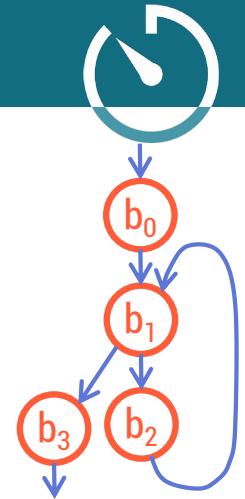
- $x_{2-1} \leq N \cdot x_{0-1}$



Early approaches

Limitations on reservation tables

- complex pipelines: superscalar, ooo, multiple FUs, etc.
- initial state/history: long timing effects



```

0a  __start:  mov r0,#1
0b              mov r1,#0
0c              adr r2,a
0d              adr r3,init_val
0e              ldr r3,[r3]
1a  while:   mul r0,r1,r0
1b              cmp r1,#N
1c              bhs end
2a              str r3,[r2],#4
2b              add r1,r1,#1
2c              b while
3a  end:      adr r2,product
3b              str r0,[r2]
    
```

b_1 after b_0

$t_0 = 11$ cycles $t_{0-1} = 3$ cycles

F	0a	0b	0c	0d	0e	1a	1b	1c										
A		0a	0b	0c	0d			1b	1c									
Mu							1a	1a	1a	1a	1a							
Me						0e	0e	0e	0e	0e								
C			0a	0b	0c	0d					0e	1a	1b	1c				

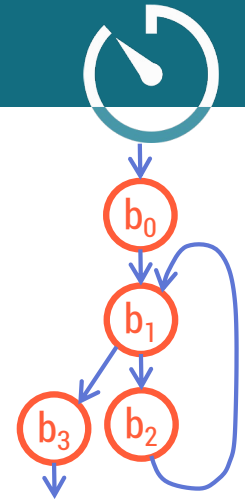
b_2 after b_1

F																		
A																		
Mu																		
Me																		
C																		

Early approaches

Limitations on reservation tables

- complex pipelines: superscalar, ooo, multiple FUs, etc.
- initial state/history: long timing effects



```

0a  __start:  mov r0,#1
0b              mov r1,#0
0c              adr r2,a
0d              adr r3,init_val
0e              ldr r3,[r3]
1a  while:   mul r0,r1,r0
1b              cmp r1,#N
1c              bhs end
2a              str r3,[r2],#4
2b              add r1,r1,#1
2c              b while
3a  end:      adr r2,product
3b              str r0,[r2]
  
```

b_1 after b_0

$t_0 = 11$ cycles $t_{0-1} = 3$ cycles

F	0a	0b	0c	0d	0e	1a	1b	1c										
A		0a	0b	0c	0d			1b	1c									
Mu							1a	1a	1a	1a	1a							
Me						0e	0e	0e	0e	0e								
C			0a	0b	0c	0d					0e	1a	1b	1c				

b_2 after b_1

$t_{1-2} = 3$ cycles

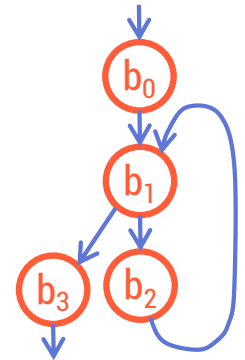
F	1a	1b	1c	2a	2b	2c												
A			1b	1c		2b	2c											
Mu		1a	1a	1a	1a	1a												
Me					2a	2a	2a	2a	2a									
C							1a	1b	1c	2a	2b	2c						

Early approaches



Limitations on reservation tables

- complex pipelines: superscalar, ooo, multiple FUs, etc.
- initial state/history: long timing effects



0a	__start:	mov r0,#1
0b		mov r1,#0
0c		adr r2,a
0d		adr r3,init_val
0e		ldr r3,[r3]
1a	while:	mul r0,r1,r0
1b		cmp r1,#N
1c		bhs end
2a		str r3,[r2],#4
2b		add r1,r1,#1
2c		b while
3a	end:	adr r2,product
3b		str r0,[r2]

$t_0 = 11$ cycles

$t_{0-1} = 3$ cycles

$t_{1-2} = 3$ cycles

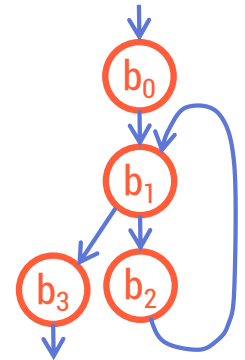
$$t_{0-1-2} = t_0 + t_{0-1} + t_{1-2} = 17 \text{ cycles}$$

Early approaches



Limitations on reservation tables

- complex pipelines: superscalar, ooo, multiple FUs, etc.
- initial state/history: long timing effects



```

0a  __start:  mov r0,#1
0b                mov r1,#0
0c                adr r2,a
0d                adr r3,init_val
0e                ldr r3,[r3]
1a  while:    mul r0,r1,r0
1b                cmp r1,#N
1c                bhs end
2a                str r3,[r2],#4
2b                add r1,r1,#1
2c                b while
3a  end:      adr r2,product
3b                str r0,[r2]
  
```

$t_0 = 11$ cycles

$t_{0-1} = 3$ cycles

$t_{1-2} = 3$ cycles

$$t_{0-1-2} = t_0 + t_{0-1} + t_{1-2} = 17 \text{ cycles}$$

b_2 after b_1 after b_0

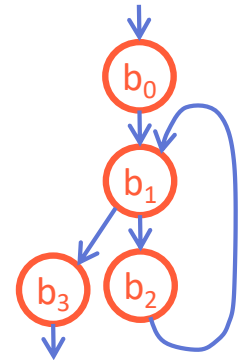
F																		
A																		
Mu																		
Me																		
C																		

Early approaches



Limitations on reservation tables

- complex pipelines: superscalar, ooo, multiple FUs, etc.
- initial state/history: long timing effects



```

0a  __start:  mov r0,#1
0b                mov r1,#0
0c                adr r2,a
0d                adr r3,init_val
0e                ldr r3,[r3]
1a  while:   mul r0,r1,r0
1b                cmp r1,#N
1c                bhs end
2a                str r3,[r2],#4
2b                add r1,r1,#1
2c                b while
3a  end:      adr r2,product
3b                str r0,[r2]
  
```

$t_0 = 11$ cycles

$t_{0-1} = 3$ cycles

$t_{1-2} = 3$ cycles

$t_{0-1-2} = t_0 + t_{0-1} + t_{1-2} = 17$ cycles

b_2 after b_1 after b_0

F	0a	0b	0c	0d	0e	1a	1b	1c	2a	2b	2c							
A		0a	0b	0c	0d			1b	1c		2b	2c						
Mu							1a	1a	1a	1a	1a							
Me						0e	0e	0e	0e	0e	2a	2a	2a	2a	2a			
C			0a	0b	0c	0d					0e	1a	1b	1c		2a	2b	2c

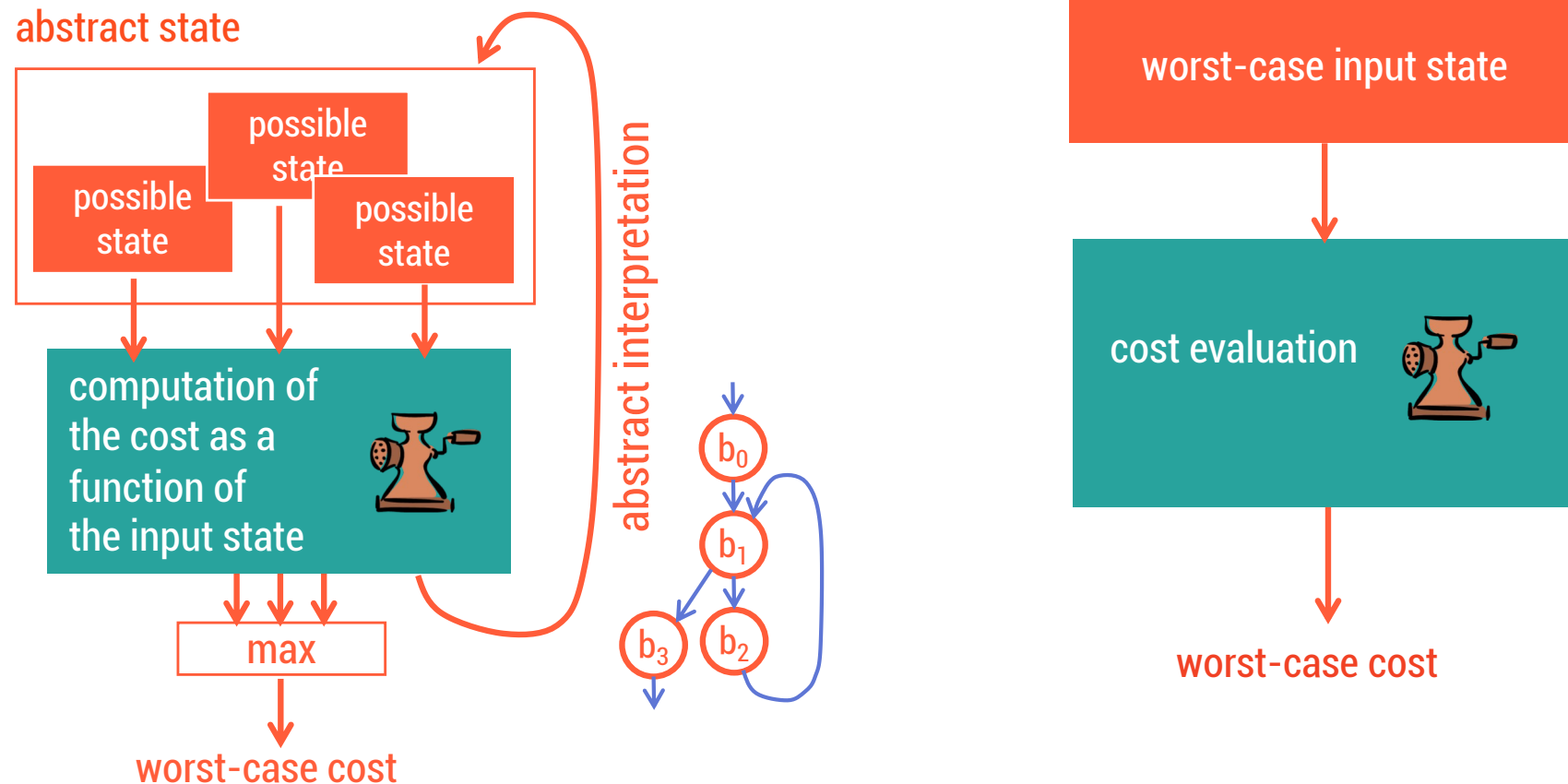
$t_{0-1-2} = 18$ cycles



Modern approaches



Abstract interpretation vs. local analysis

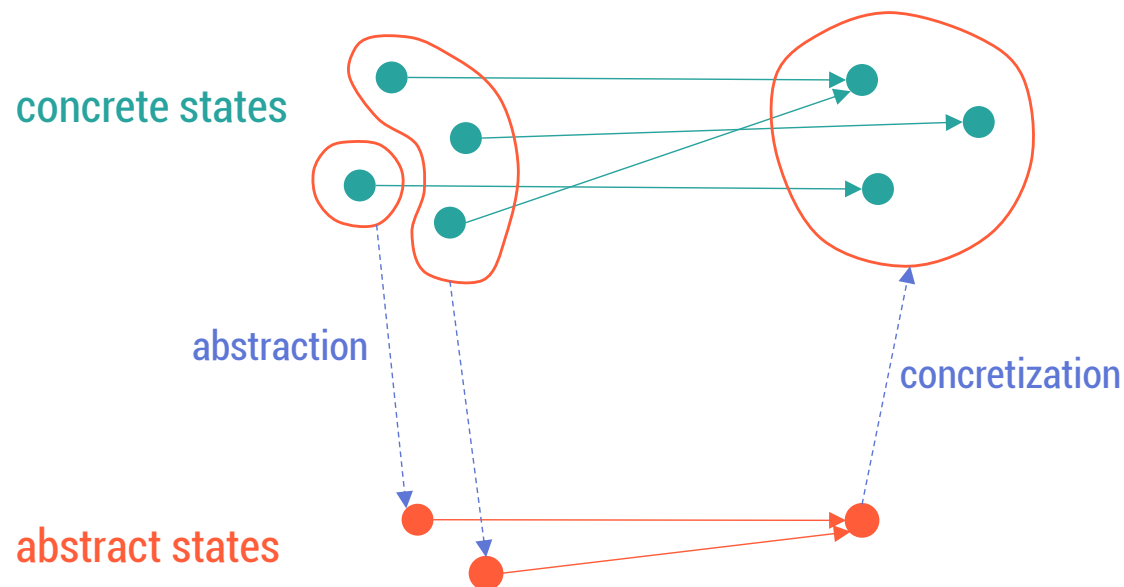


What is Abstract Interpretation?



A theory of approximation of the semantics of programs

- partial execution of a program to derive information that can be used to answer a given question



P. Cousot, R. Cousot, *Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints*, POPL, 1977

What is Abstract Interpretation?



An intuitive example (ignoring theory)

```
x = 1;  
while (x <= 100)  
  x += rand_1_10();
```

`rand_1_10()` returns an integer in $[1,10]$

Prove that $0 \leq x < 200$ at the end of the program.

What is Abstract Interpretation?

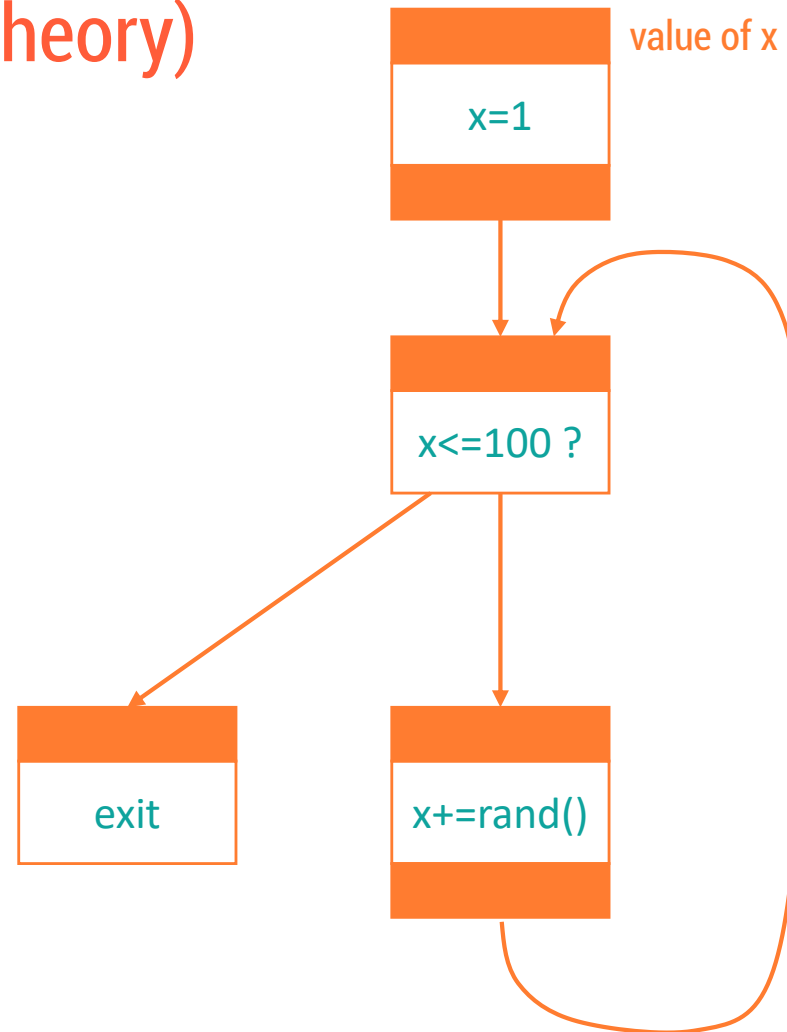


An intuitive example (ignoring theory)

```
x = 1;  
while (x <= 100)  
  x += rand_1_10();
```

rand_1_10() returns an integer in [1,10]

Prove that $0 \leq x < 200$ at the end of the program.



What is Abstract Interpretation?

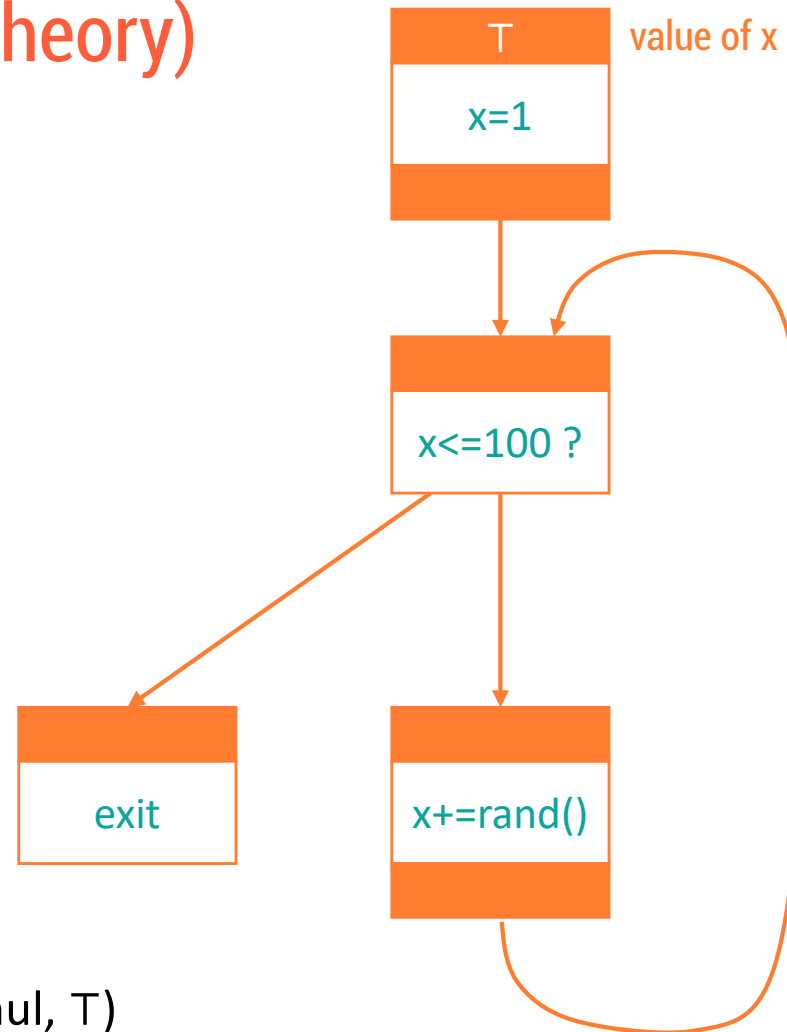


An intuitive example (ignoring theory)

```
x = 1;  
while (x <= 100)  
  x += rand_1_10();
```

rand_1_10() returns an integer in [1,10]

Prove that $0 \leq x < 200$ at the end of the program.



First abstraction: sign (pos, neg, nul, T)

What is Abstract Interpretation?

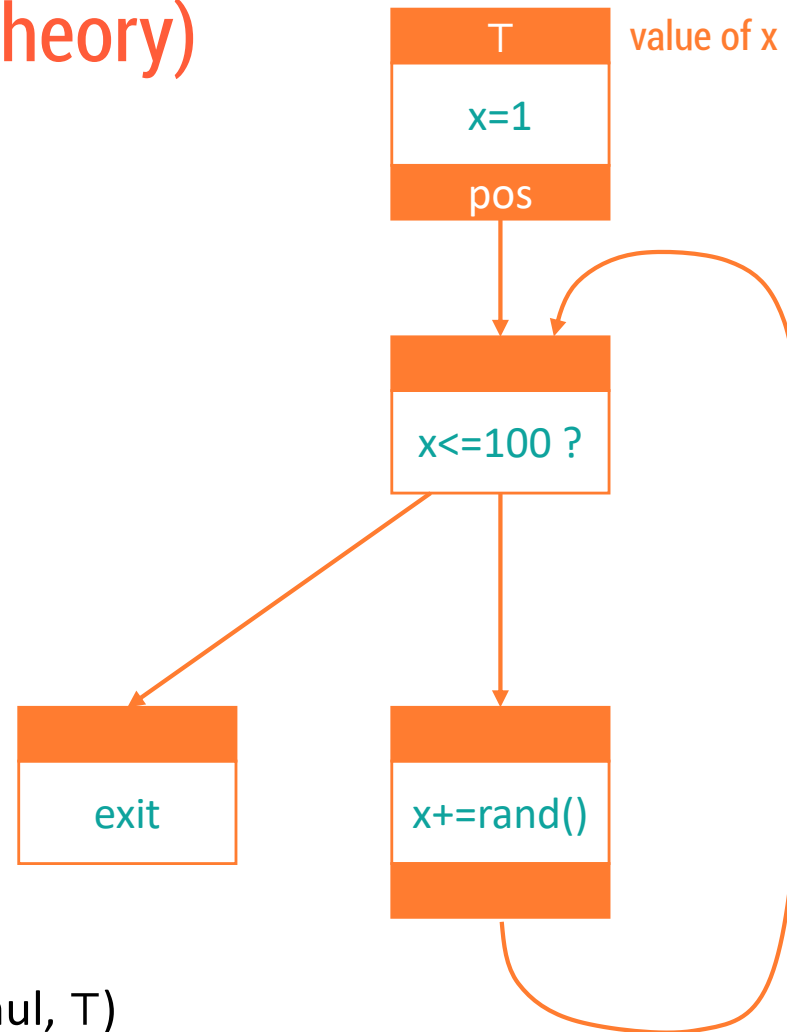


An intuitive example (ignoring theory)

```
x = 1;  
while (x <= 100)  
  x += rand_1_10();
```

rand_1_10() returns an integer in [1,10]

Prove that $0 \leq x < 200$ at the end of the program.



First abstraction: sign (pos, neg, nul, T)

What is Abstract Interpretation?

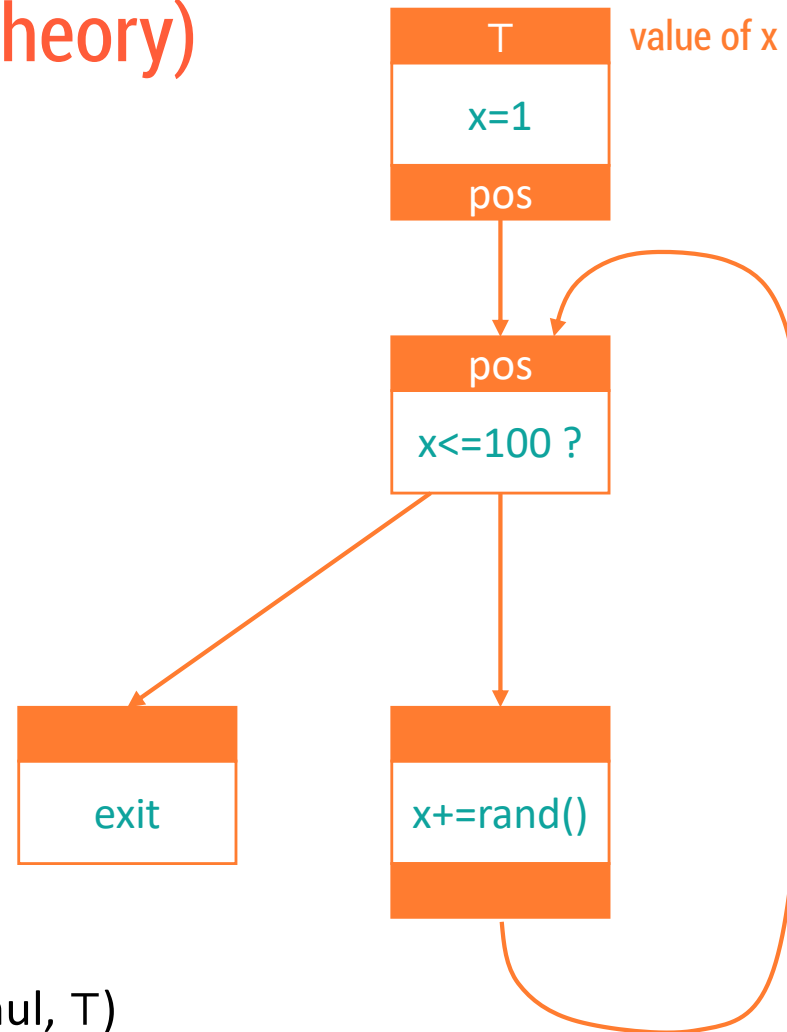


An intuitive example (ignoring theory)

```
x = 1;  
while (x <= 100)  
  x += rand_1_10();
```

rand_1_10() returns an integer in [1,10]

Prove that $0 \leq x < 200$ at the end of the program.



First abstraction: sign (pos, neg, nul, T)

What is Abstract Interpretation?

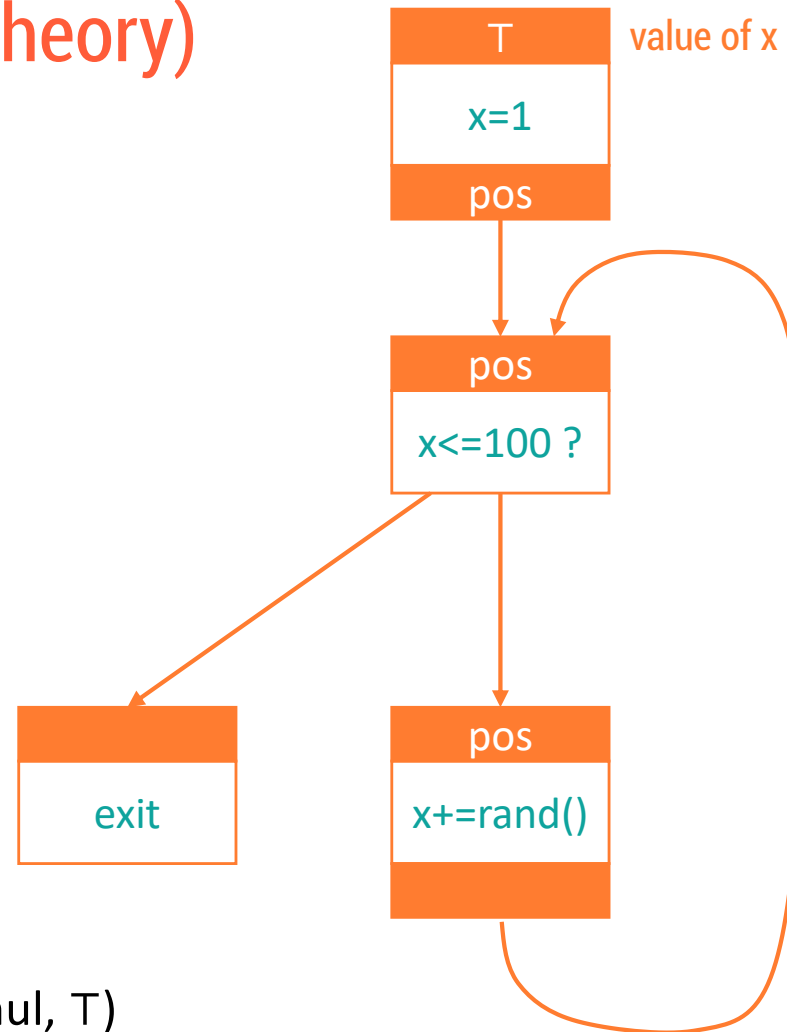


An intuitive example (ignoring theory)

```
x = 1;  
while (x <= 100)  
  x += rand_1_10();
```

rand_1_10() returns an integer in [1,10]

Prove that $0 \leq x < 200$ at the end of the program.



First abstraction: sign (pos, neg, nul, T)

What is Abstract Interpretation?

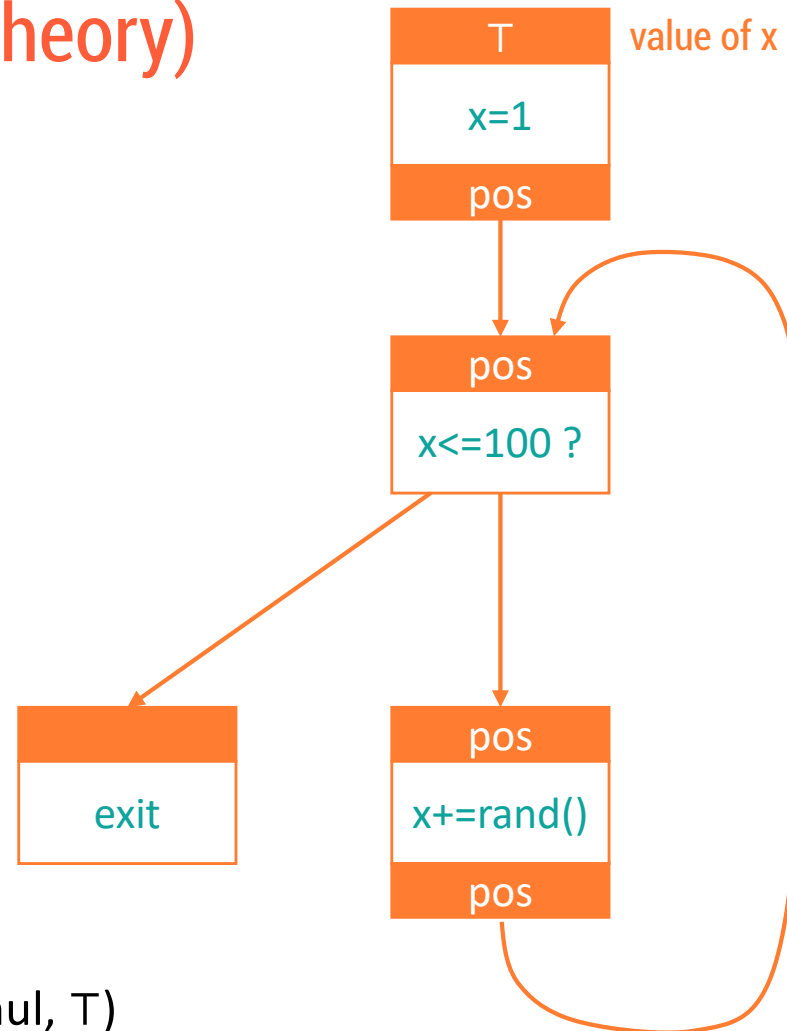


An intuitive example (ignoring theory)

```
x = 1;  
while (x <= 100)  
  x += rand_1_10();
```

rand_1_10() returns an integer in [1,10]

Prove that $0 \leq x < 200$ at the end of the program.



First abstraction: sign (pos, neg, nul, T)

What is Abstract Interpretation?

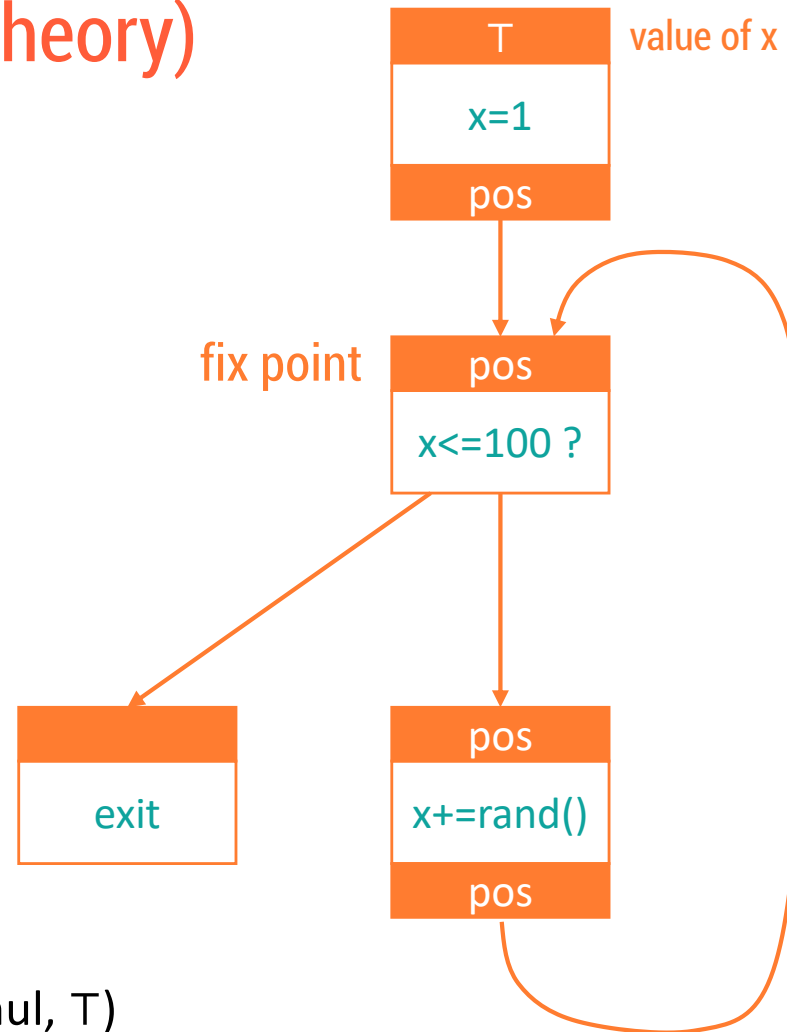


An intuitive example (ignoring theory)

```
x = 1;
while (x <= 100)
  x += rand_1_10();
```

rand_1_10() returns an integer in [1,10]

Prove that $0 \leq x < 200$ at the end of the program.



First abstraction: sign (pos, neg, nul, T)

What is Abstract Interpretation?

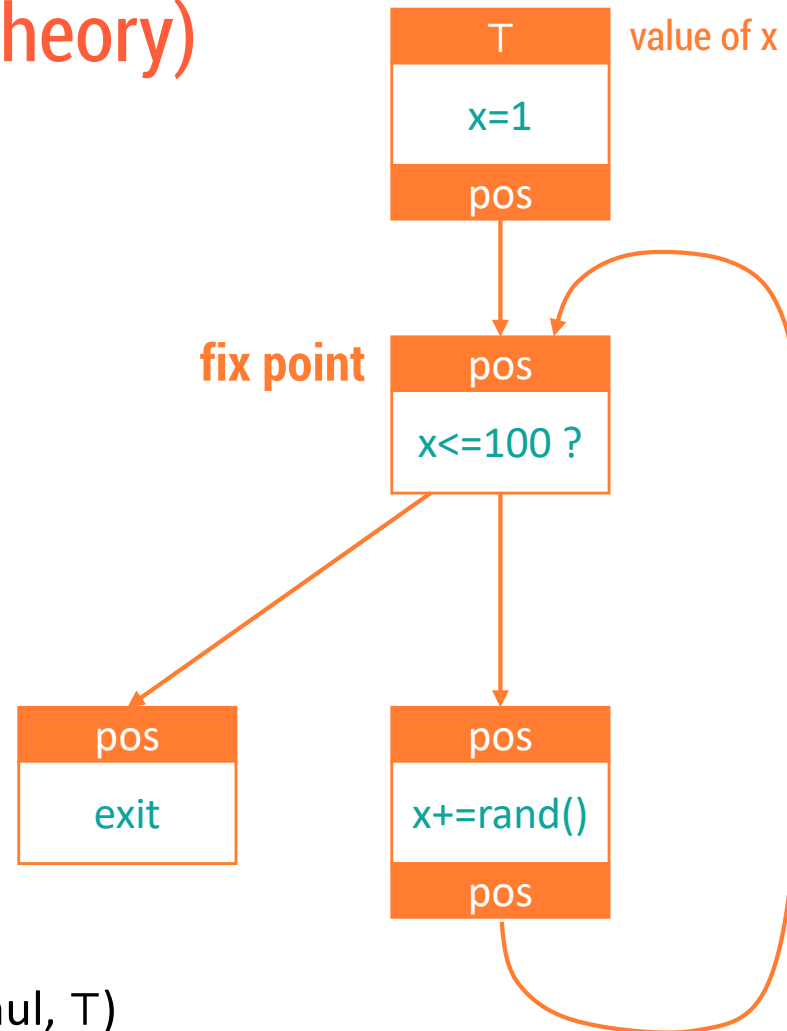


An intuitive example (ignoring theory)

```
x = 1;  
while (x <= 100)  
  x += rand_1_10();
```

rand_1_10() returns an integer in [1,10]

Prove that $0 \leq x < 200$ at the end of the program.



First abstraction: sign (pos, neg, nul, T)

Conclusion : $0 \leq x$

What is Abstract Interpretation?

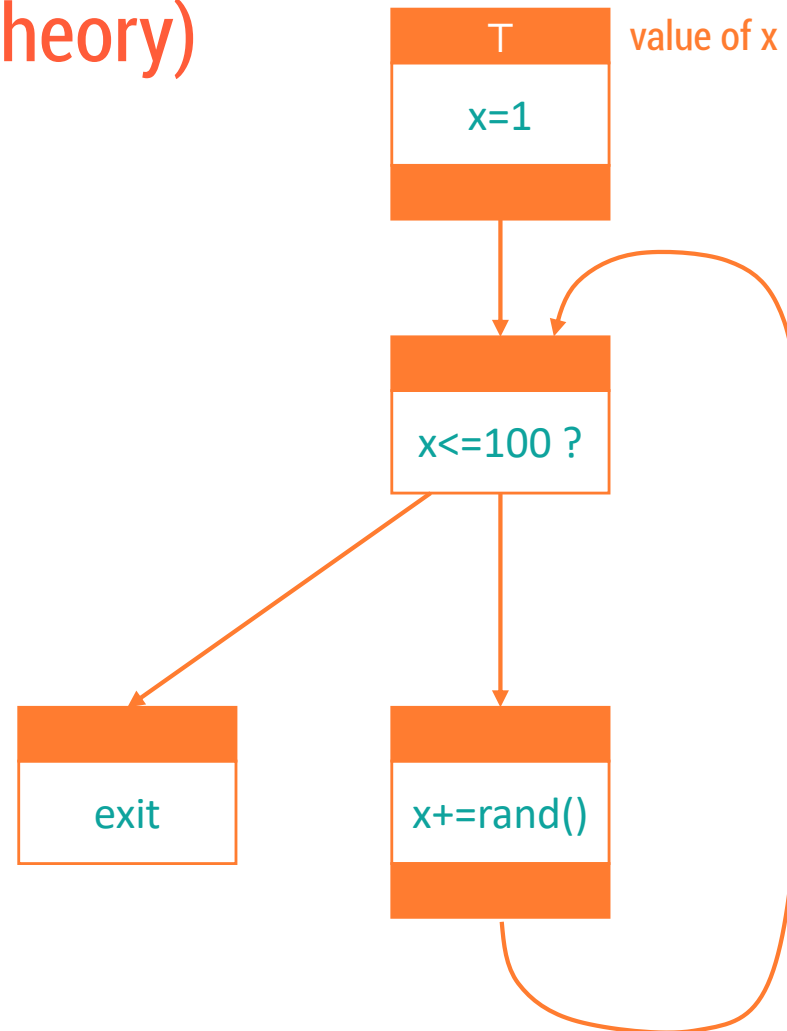


An intuitive example (ignoring theory)

```
x = 1;  
while (x <= 100)  
  x += rand_1_10();
```

rand_1_10() returns an integer in [1,10]

Prove that $0 \leq x < 200$ at the end of the program.



Second abstraction: intervals

What is Abstract Interpretation?

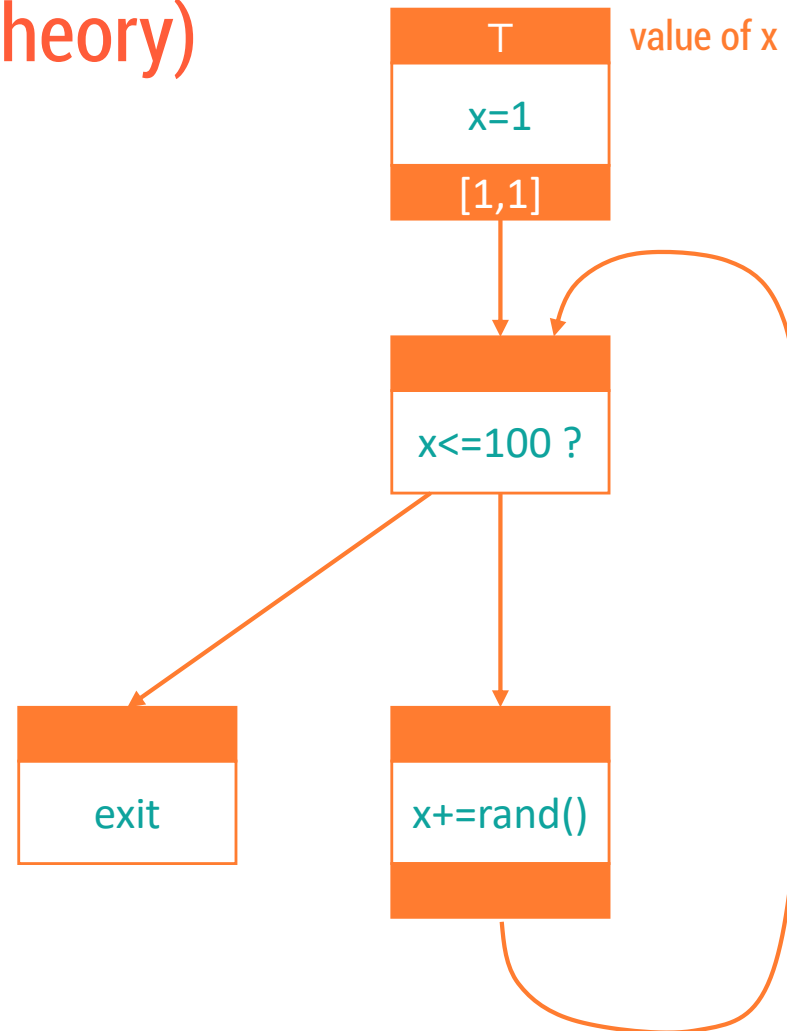


An intuitive example (ignoring theory)

```
x = 1;  
while (x <= 100)  
  x += rand_1_10();
```

rand_1_10() returns an integer in [1,10]

Prove that $0 \leq x < 200$ at the end of the program.



Second abstraction: intervals

What is Abstract Interpretation?

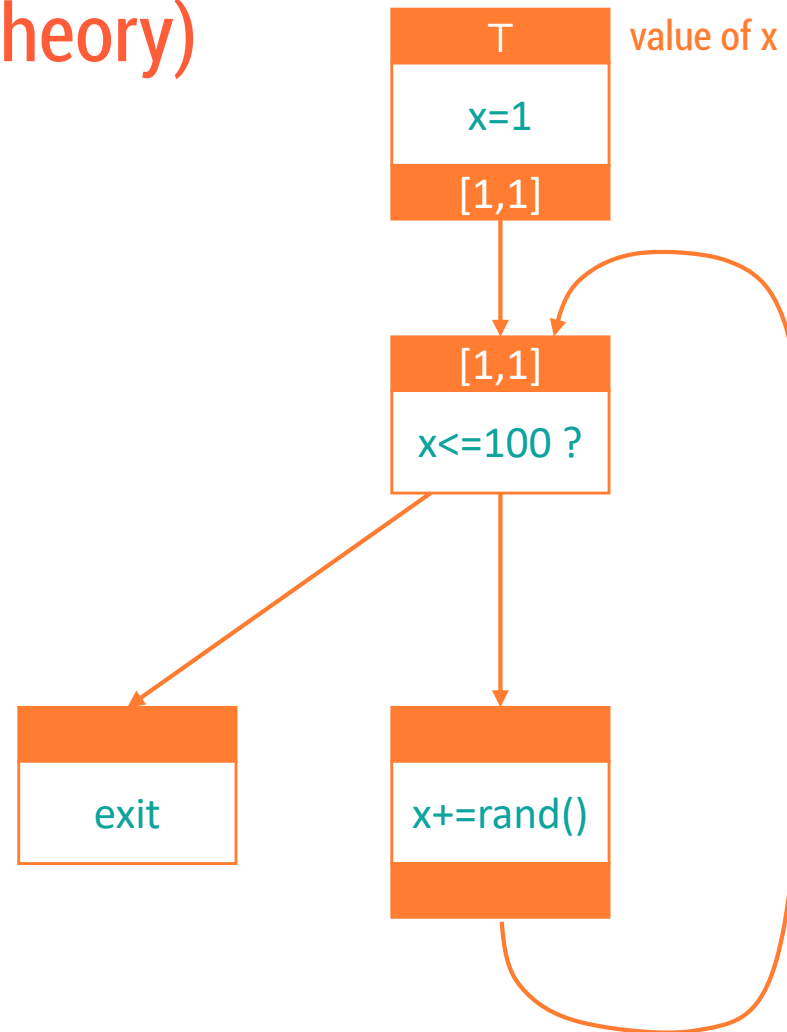


An intuitive example (ignoring theory)

```
x = 1;  
while (x <= 100)  
  x += rand_1_10();
```

rand_1_10() returns an integer in [1,10]

Prove that $0 \leq x < 200$ at the end of the program.



Second abstraction: intervals

What is Abstract Interpretation?

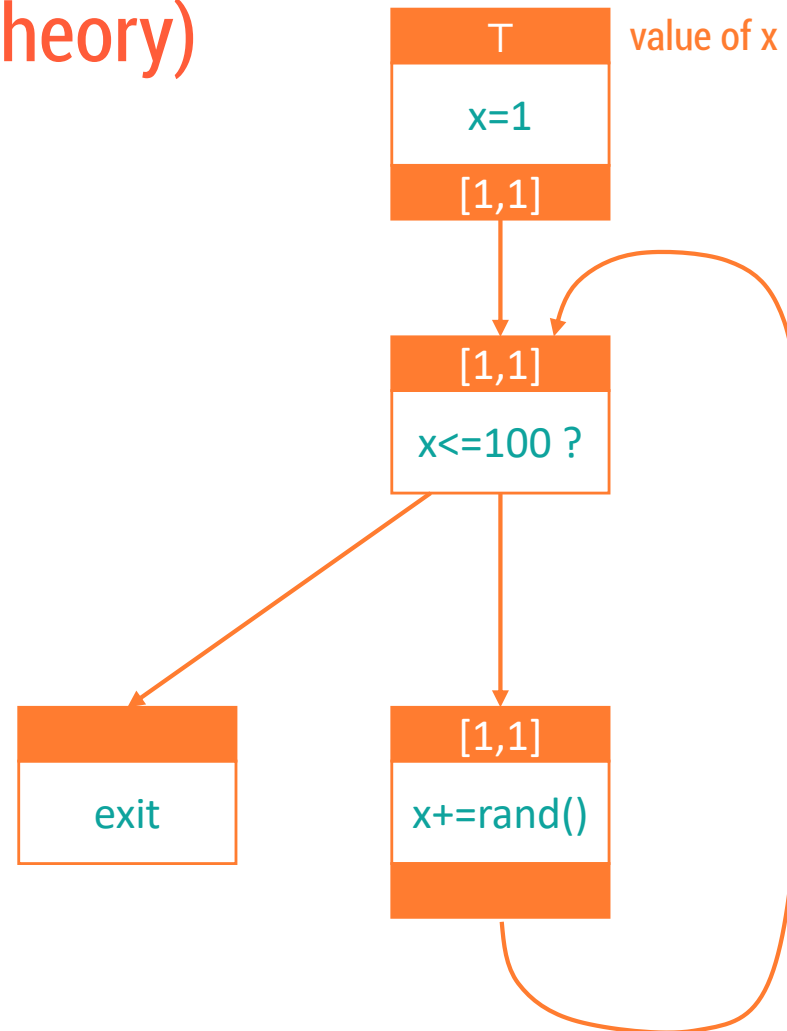


An intuitive example (ignoring theory)

```
x = 1;  
while (x <= 100)  
  x += rand_1_10();
```

rand_1_10() returns an integer in [1,10]

Prove that $0 \leq x < 200$ at the end of the program.



Second abstraction: intervals

What is Abstract Interpretation?

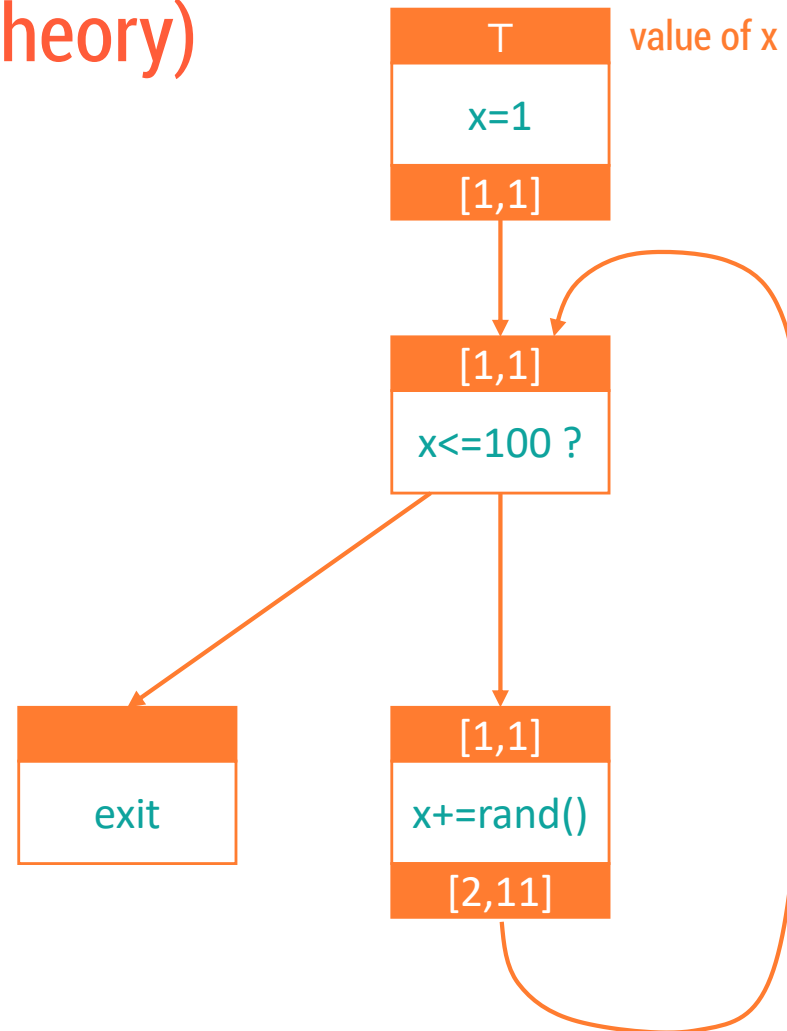


An intuitive example (ignoring theory)

```
x = 1;  
while (x <= 100)  
  x += rand_1_10();
```

rand_1_10() returns an integer in [1,10]

Prove that $0 \leq x < 200$ at the end of the program.



Second abstraction: intervals

What is Abstract Interpretation?

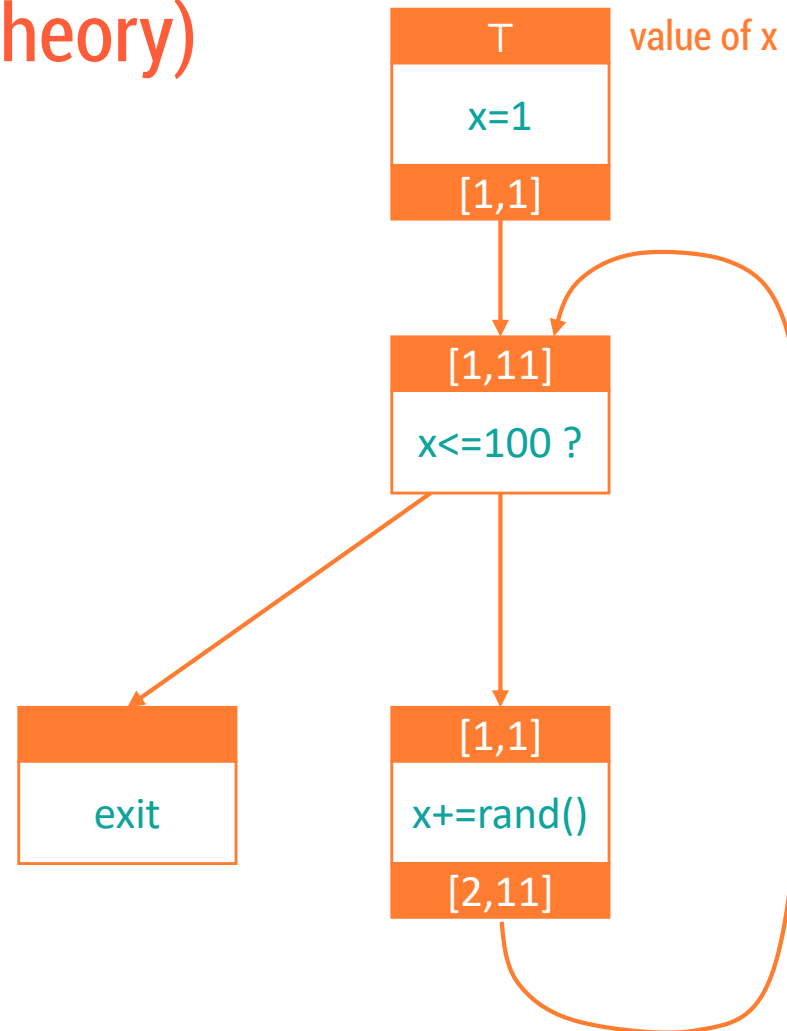


An intuitive example (ignoring theory)

```
x = 1;  
while (x <= 100)  
  x += rand_1_10();
```

rand_1_10() returns an integer in [1,10]

Prove that $0 \leq x < 200$ at the end of the program.



Second abstraction: intervals

What is Abstract Interpretation?

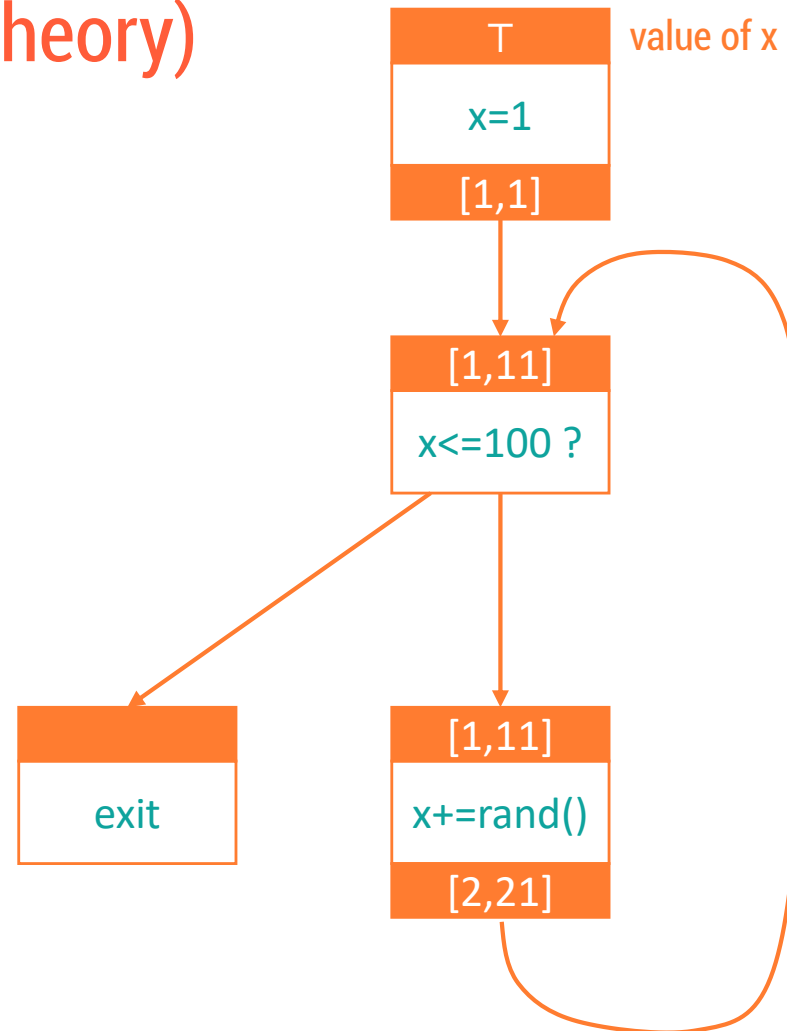


An intuitive example (ignoring theory)

```
x = 1;  
while (x <= 100)  
  x += rand_1_10();
```

rand_1_10() returns an integer in [1,10]

Prove that $0 \leq x < 200$ at the end of the program.



Second abstraction: intervals

What is Abstract Interpretation?

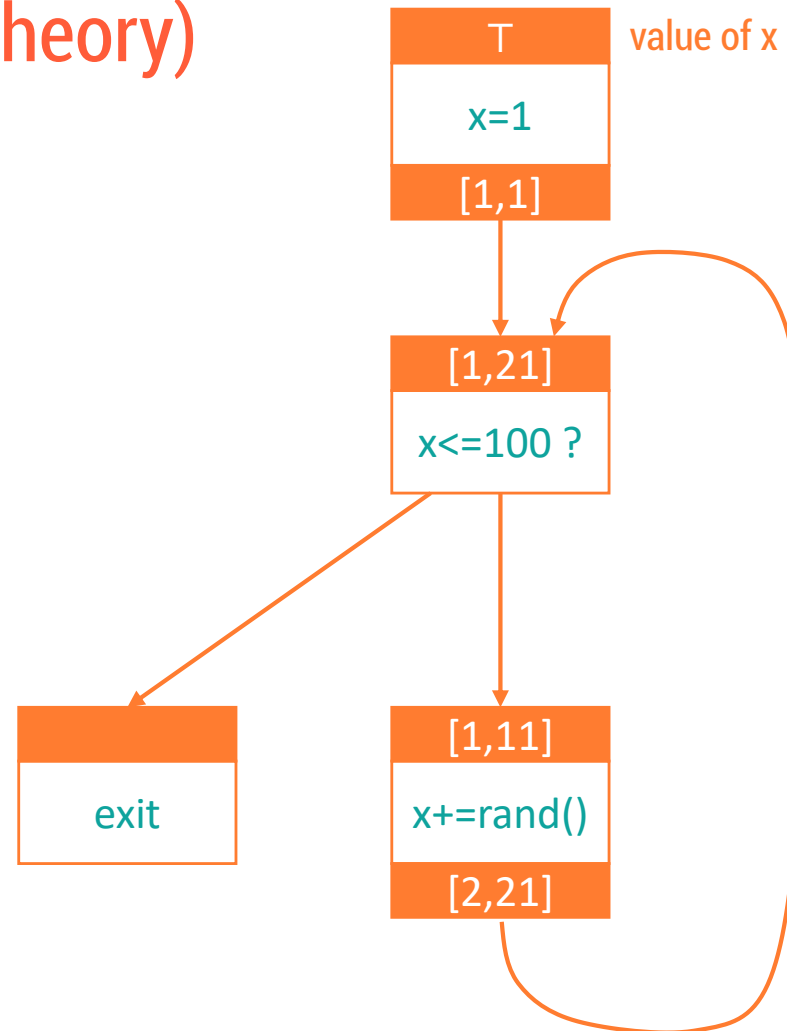


An intuitive example (ignoring theory)

```
x = 1;  
while (x <= 100)  
  x += rand_1_10();
```

rand_1_10() returns an integer in [1,10]

Prove that $0 \leq x < 200$ at the end of the program.



Second abstraction: intervals

What is Abstract Interpretation?

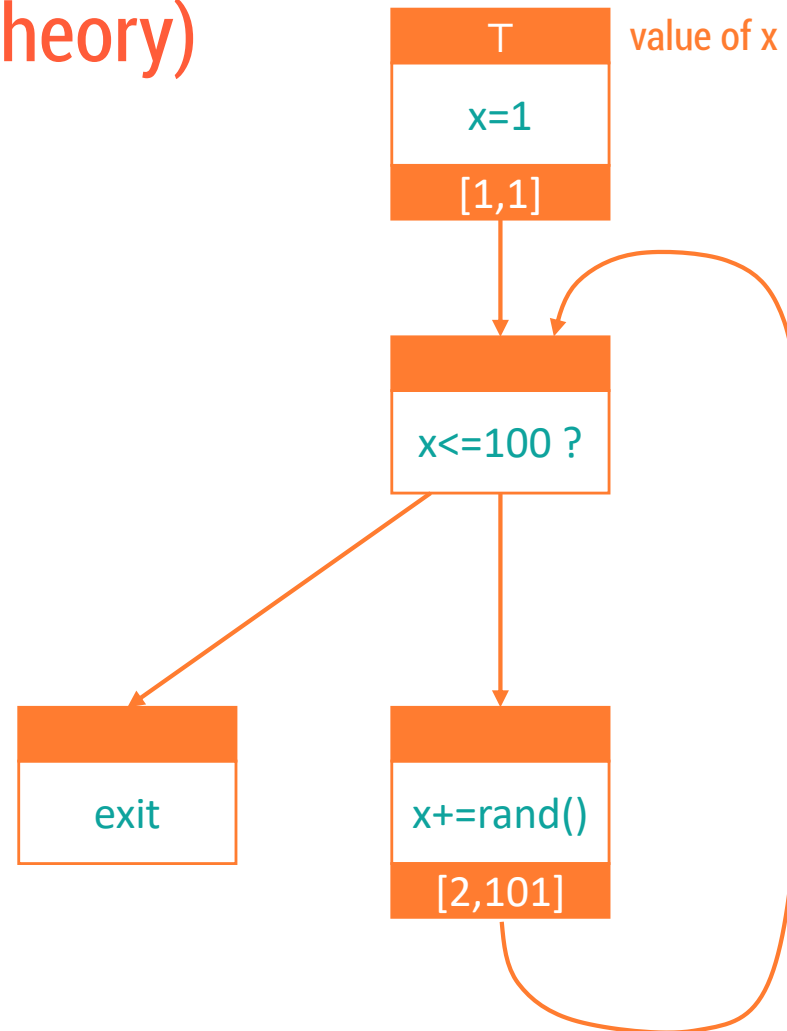


An intuitive example (ignoring theory)

```
x = 1;  
while (x <= 100)  
  x += rand_1_10();
```

`rand_1_10()` returns an integer in $[1,10]$

Prove that $0 \leq x < 200$ at the end of the program.



Second abstraction: intervals

after 10 iterations ...

What is Abstract Interpretation?

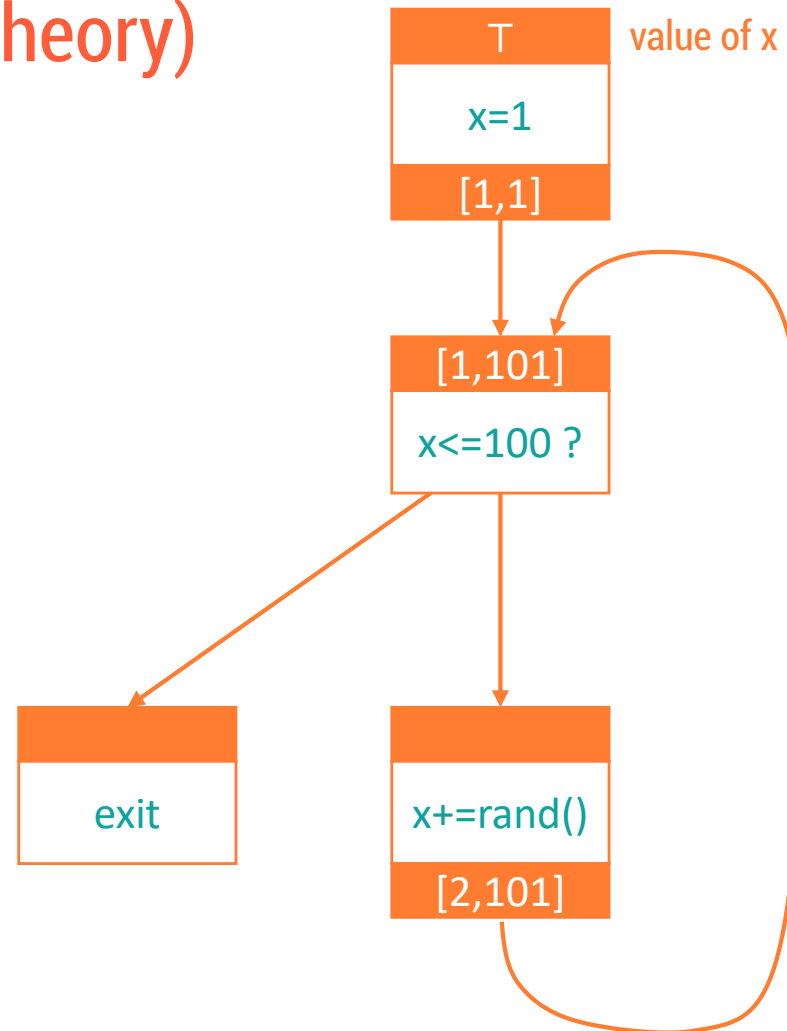


An intuitive example (ignoring theory)

```
x = 1;  
while (x <= 100)  
  x += rand_1_10();
```

rand_1_10() returns an integer in [1,10]

Prove that $0 \leq x < 200$ at the end of the program.



Second abstraction: intervals

What is Abstract Interpretation?

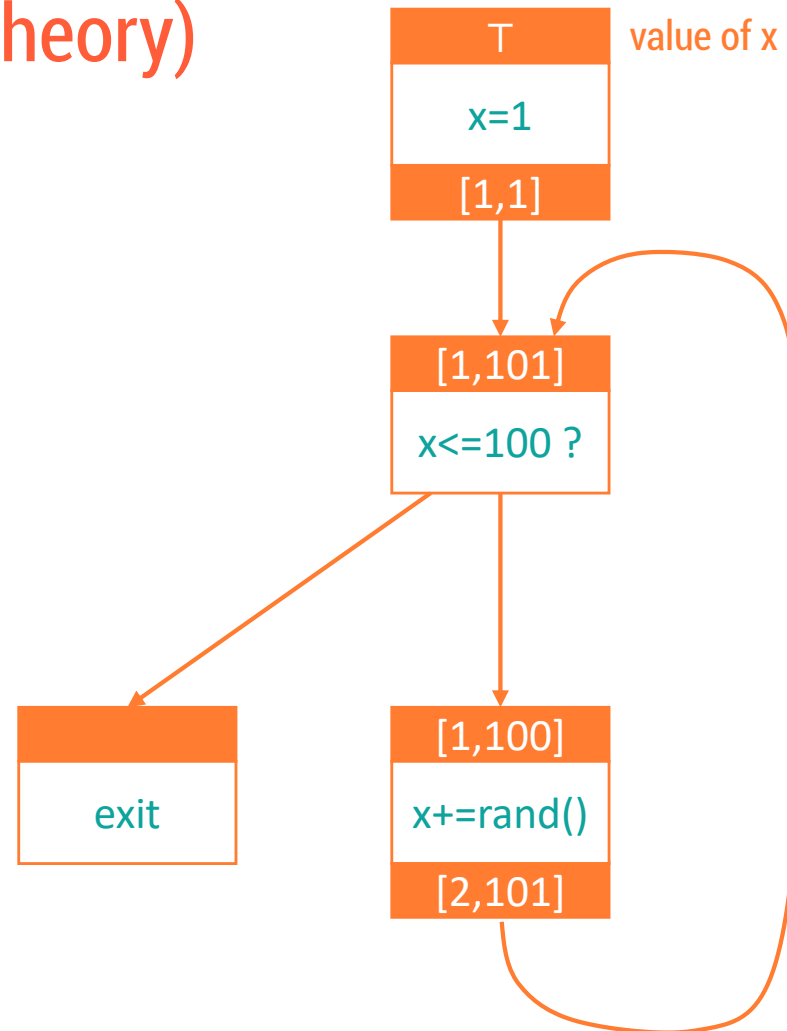


An intuitive example (ignoring theory)

```
x = 1;  
while (x <= 100)  
  x += rand_1_10();
```

rand_1_10() returns an integer in [1,10]

Prove that $0 \leq x < 200$ at the end of the program.



Second abstraction: intervals

What is Abstract Interpretation?

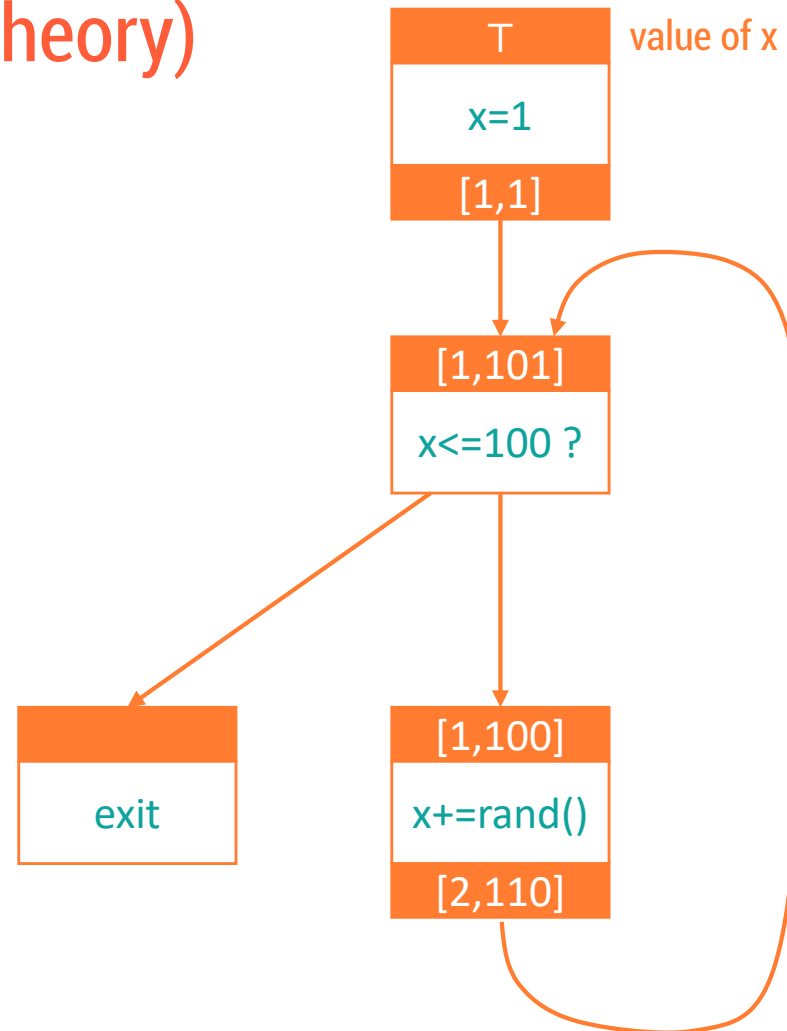


An intuitive example (ignoring theory)

```
x = 1;  
while (x <= 100)  
  x += rand_1_10();
```

rand_1_10() returns an integer in [1,10]

Prove that $0 \leq x < 200$ at the end of the program.



Second abstraction: intervals

What is Abstract Interpretation?

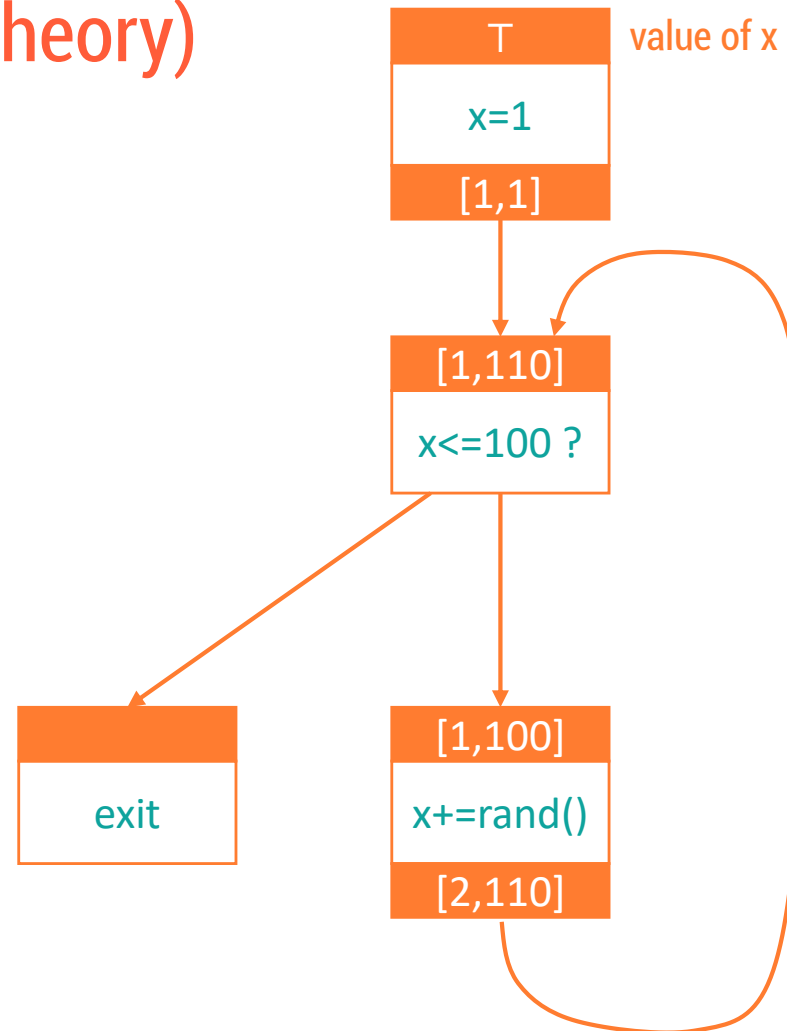


An intuitive example (ignoring theory)

```
x = 1;  
while (x <= 100)  
  x += rand_1_10();
```

rand_1_10() returns an integer in [1,10]

Prove that $0 \leq x < 200$ at the end of the program.



Second abstraction: intervals

What is Abstract Interpretation?

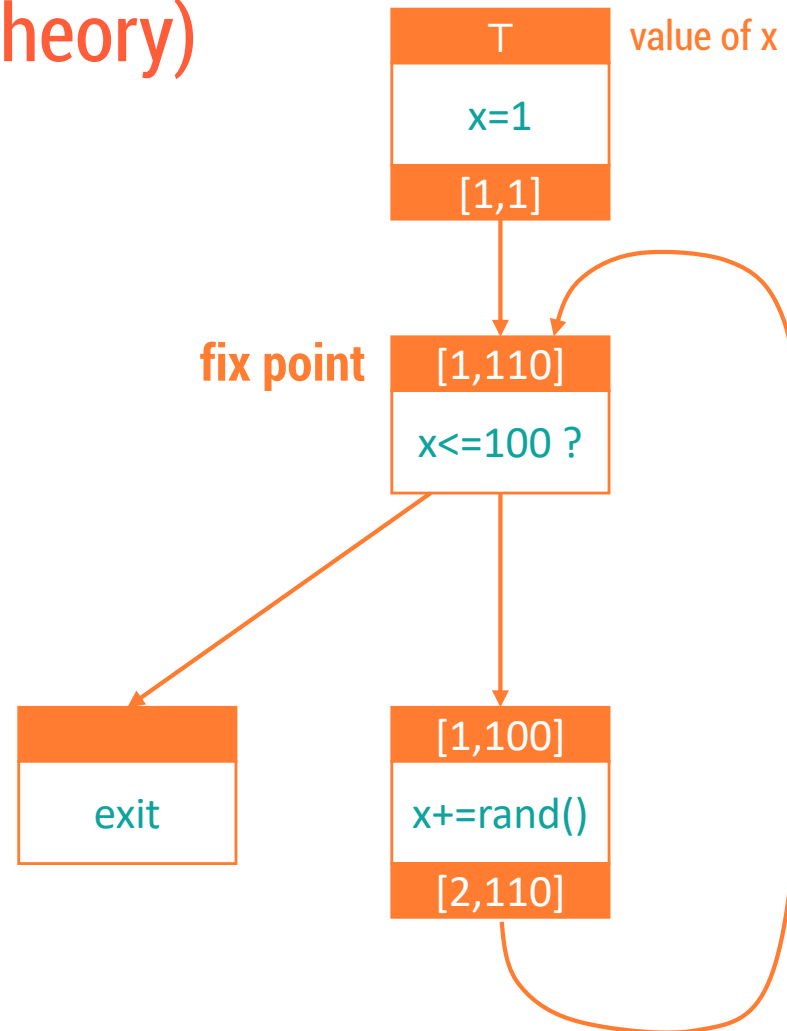


An intuitive example (ignoring theory)

```
x = 1;  
while (x <= 100)  
  x += rand_1_10();
```

rand_1_10() returns an integer in [1,10]

Prove that $0 \leq x < 200$ at the end of the program.



Second abstraction: intervals

What is Abstract Interpretation?

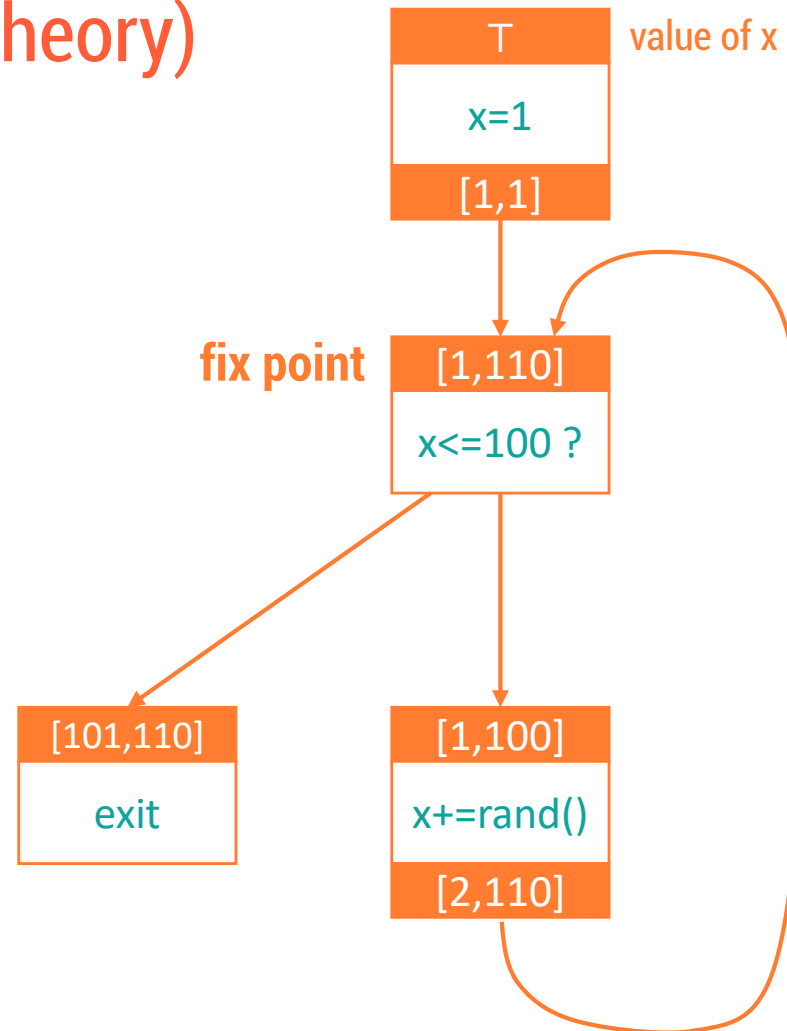


An intuitive example (ignoring theory)

```
x = 1;  
while (x <= 100)  
  x += rand_1_10();
```

rand_1_10() returns an integer in [1,10]

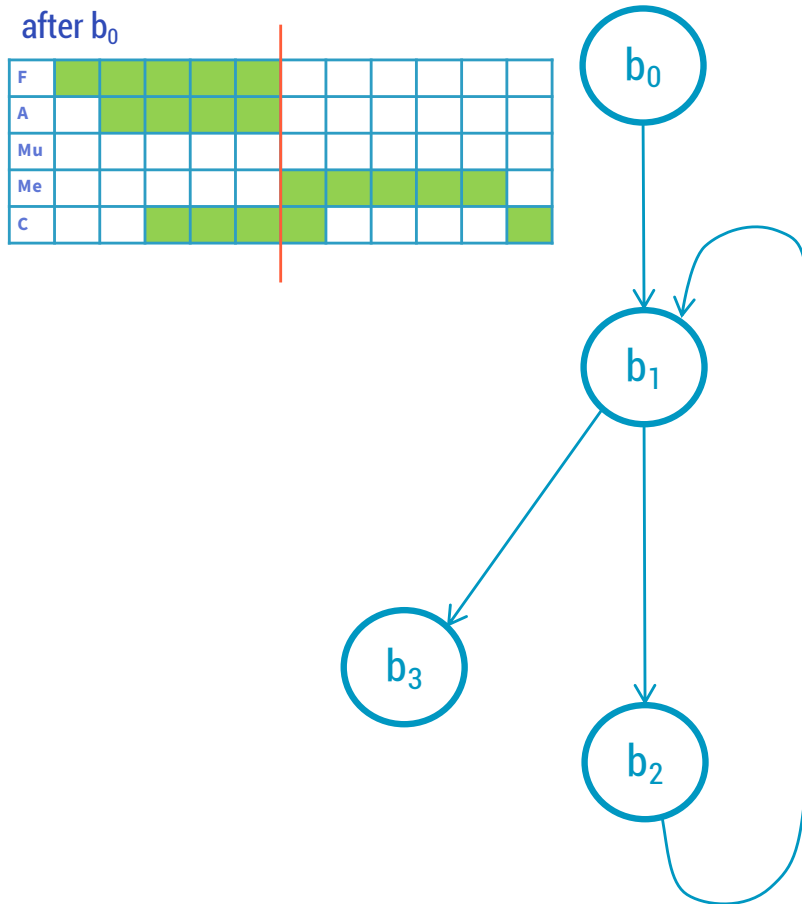
Prove that $0 \leq x < 200$ at the end of the program.



Second abstraction: intervals

Conclusion : $x < 200$

A sketch of pipeline analysis by abstract interpretation

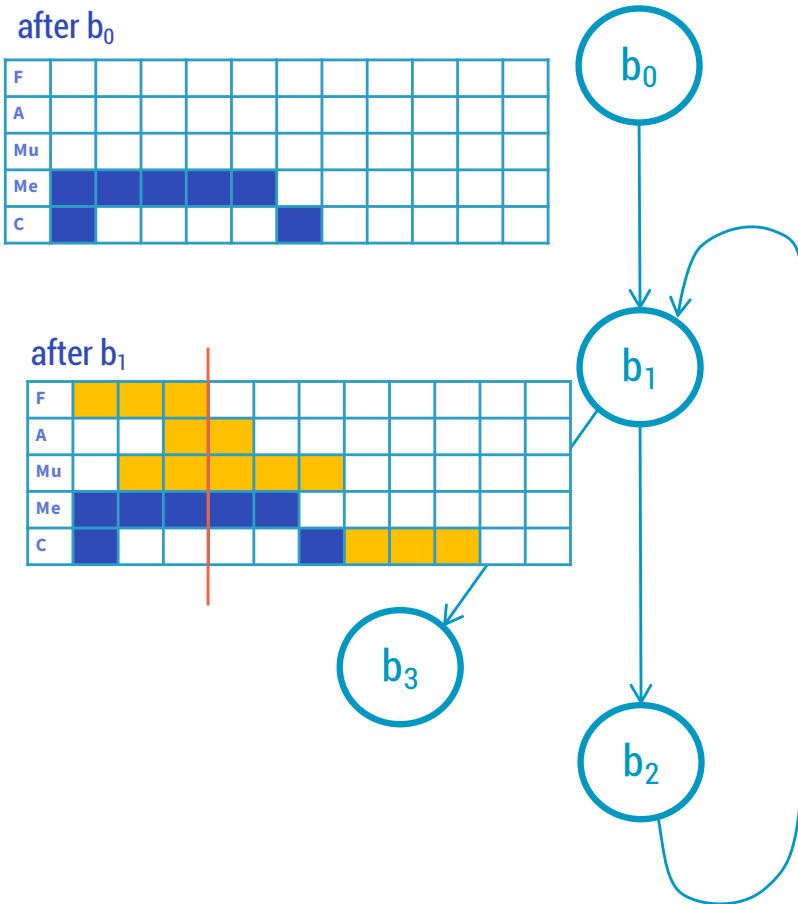


0a	__start:	mov r0,#1
0b		mov r1,#0
0c		adr r2,a
0d		adr r3,init_val
0e		ldr r3,[r3]
1a	while:	mul r0,r1,r0
1b		cmp r1,#N
1c		bhs end
2a		str r3,[r2],#4
2b		add r1,r1,#1
2c		b while
3a	end:	adr r2,product
3b		str r0,[r2]

$t_0 = 11$

Disclaimer: this is only a possible way to analyze a pipeline with "abstract" interpretation.
This is not the way it is done in tools that use this technique.

A sketch of pipeline analysis by abstract interpretation

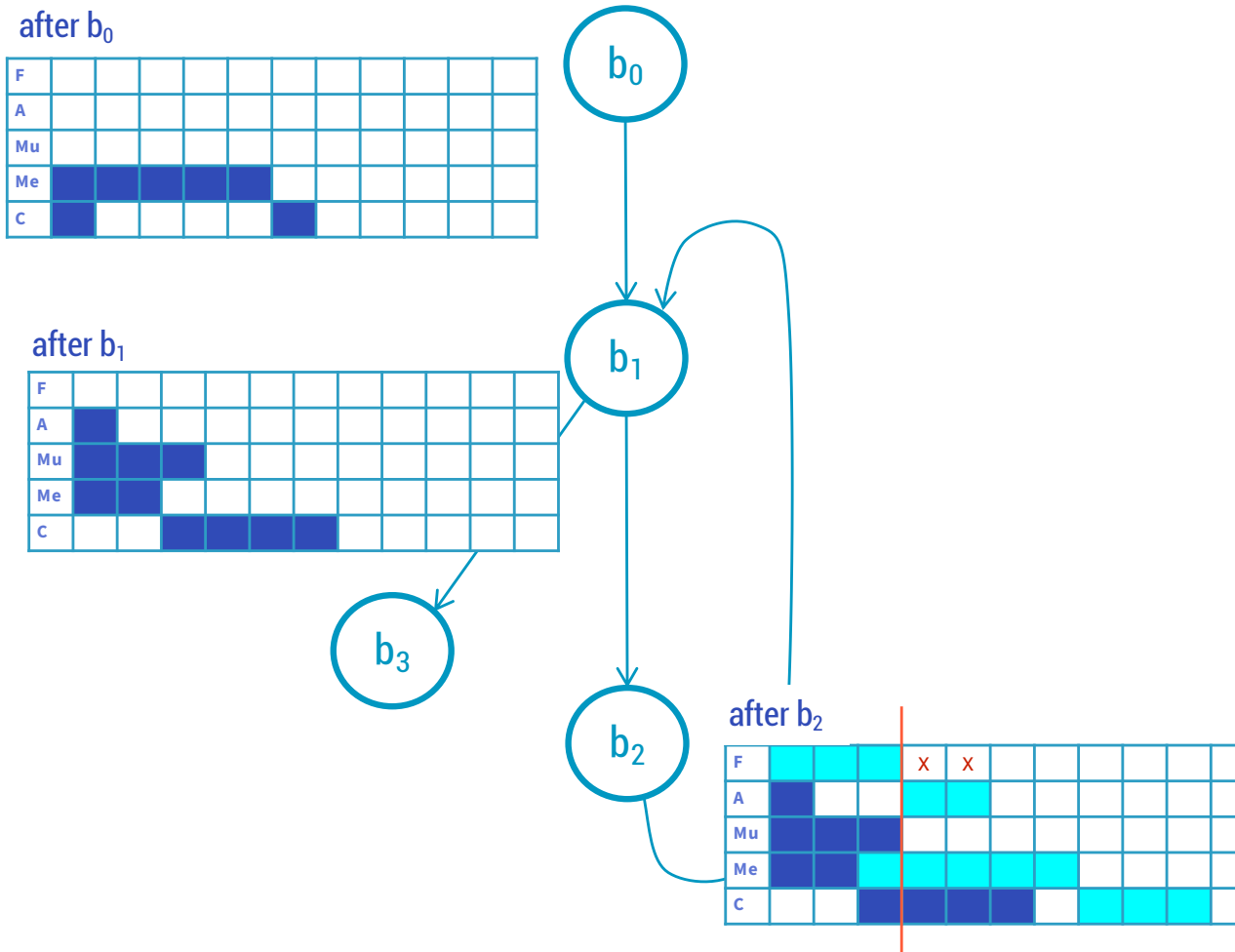


0a	__start:	mov r0,#1
0b		mov r1,#0
0c		adr r2,a
0d		adr r3,init_val
0e		ldr r3,[r3]
1a	while:	mul r0,r1,r0
1b		cmp r1,#N
1c		bhs end
2a		str r3,[r2],#4
2b		add r1,r1,#1
2c		b while
3a	end:	adr r2,product
3b		str r0,[r2]

$t_0 = 11$

$t_{0-1} = 3$

A sketch of pipeline analysis by abstract interpretation

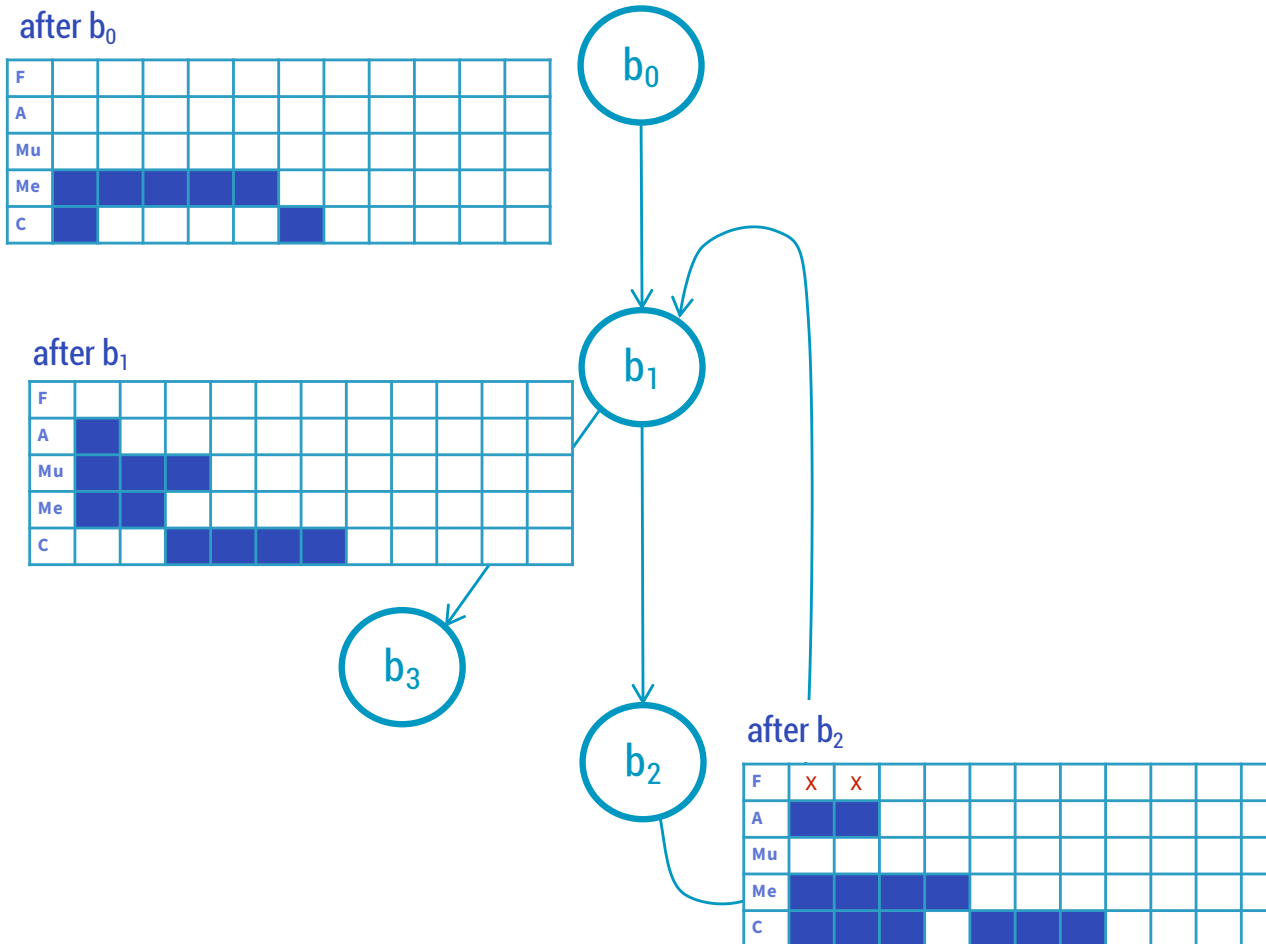


0a	__start:	mov r0,#1
0b		mov r1,#0
0c		adr r2,a
0d		adr r3,init_val
0e		ldr r3,[r3]
1a	while:	mul r0,r1,r0
1b		cmp r1,#N
1c		bhs end
2a		str r3,[r2],#4
2b		add r1,r1,#1
2c		b while
3a	end:	adr r2,product
3b		str r0,[r2]

$t_0 = 11$

$t_{0-1} = 3$

A sketch of pipeline analysis by abstract interpretation

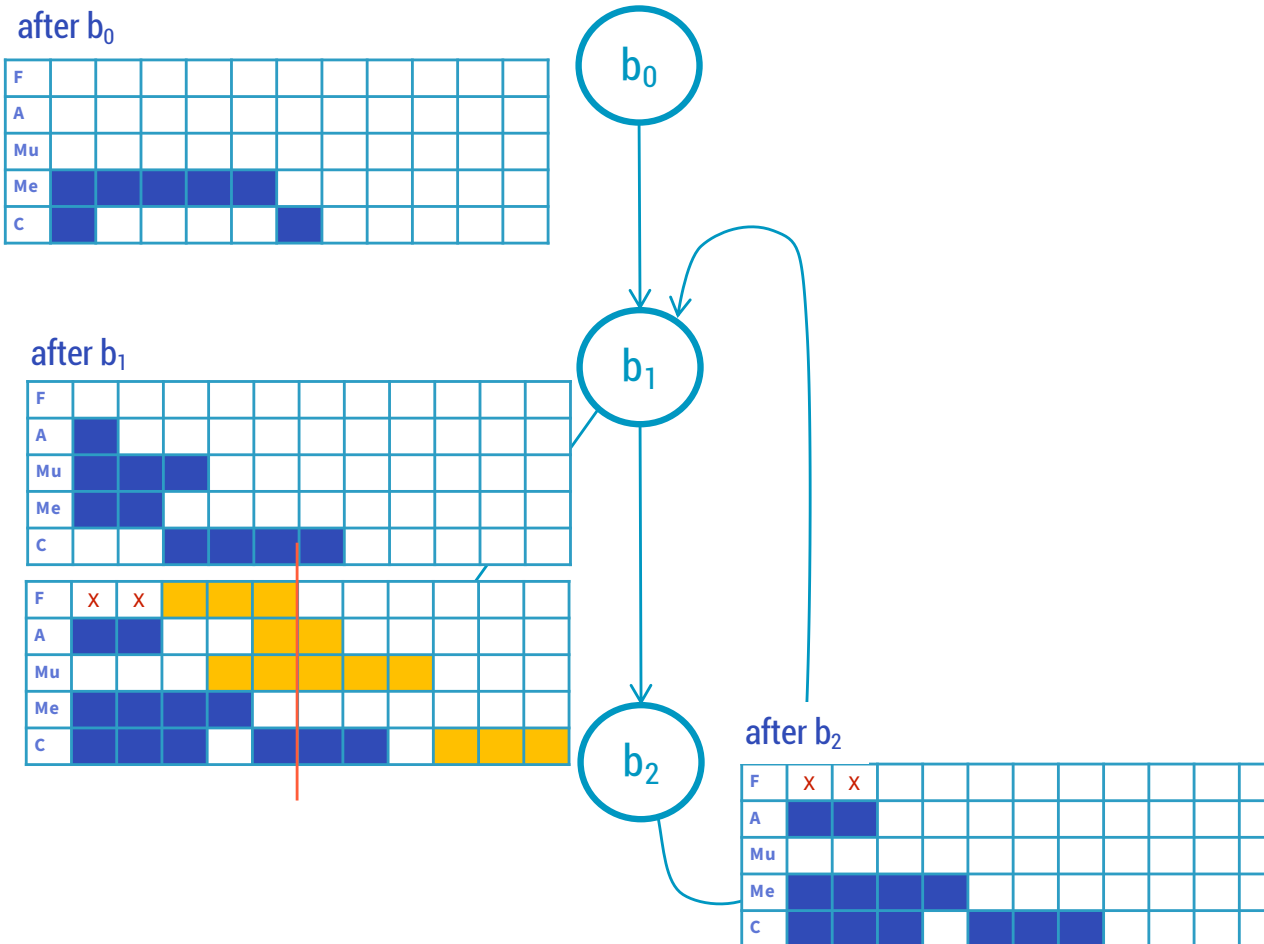


0a	__start:	mov r0,#1
0b		mov r1,#0
0c		adr r2,a
0d		adr r3,init_val
0e		ldr r3,[r3]
1a	while:	mul r0,r1,r0
1b		cmp r1,#N
1c		bhs end
2a		str r3,[r2],#4
2b		add r1,r1,#1
2c		b while
3a	end:	adr r2,product
3b		str r0,[r2]

$t_0 = 11$

$t_{0-1} = 3$

A sketch of pipeline analysis by abstract interpretation

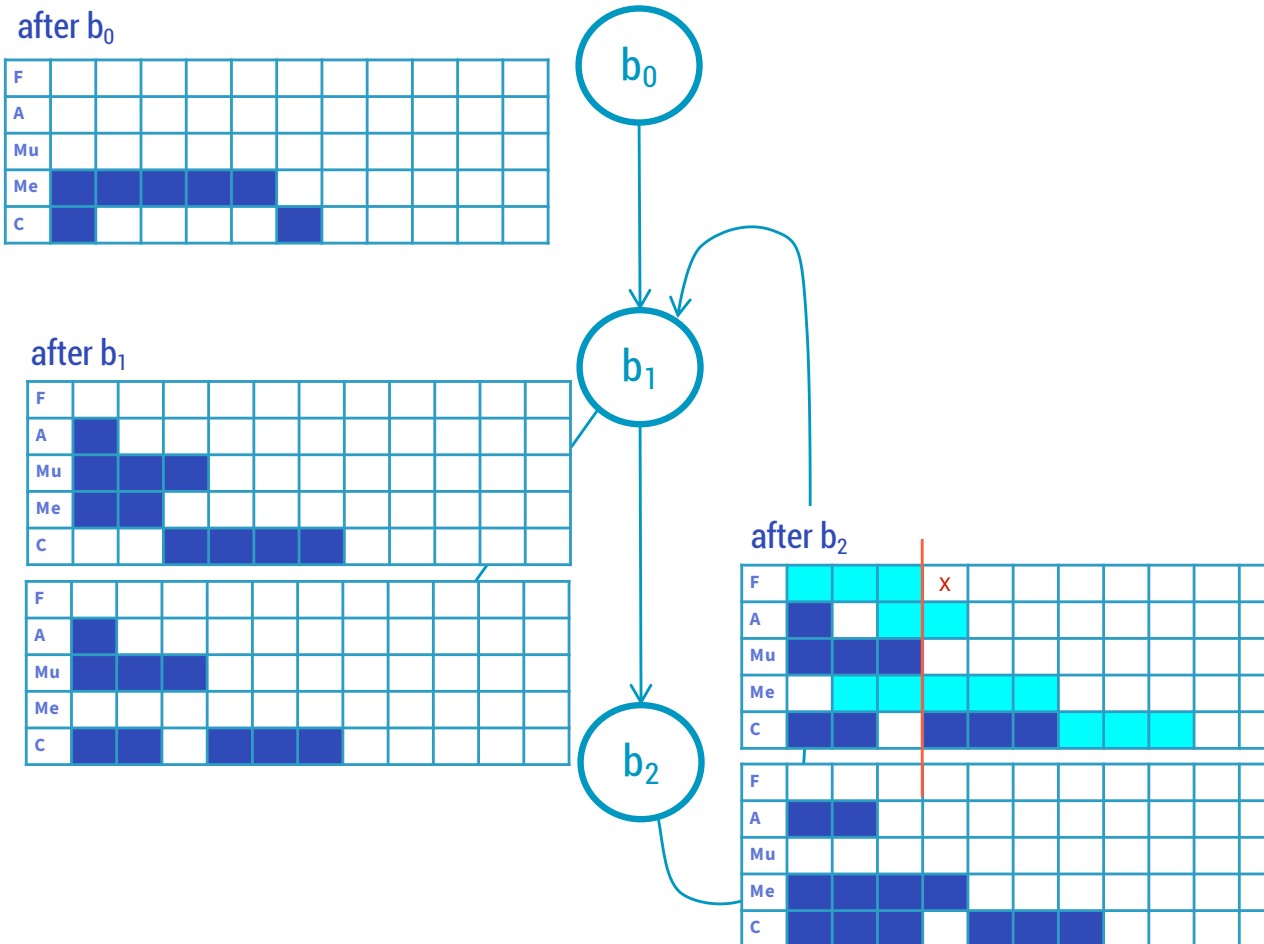


0a	__start:	mov r0,#1
0b		mov r1,#0
0c		adr r2,a
0d		adr r3,init_val
0e		ldr r3,[r3]
1a	while:	mul r0,r1,r0
1b		cmp r1,#N
1c		bhs end
2a		str r3,[r2],#4
2b		add r1,r1,#1
2c		b while
3a	end:	adr r2,product
3b		str r0,[r2]

$t_0 = 11$

$t_{0-1} = 3$

A sketch of pipeline analysis by abstract interpretation



```

0a  __start:  mov r0,#1
0b              mov r1,#0
0c              adr r2,a
0d              adr r3,init_val
0e              ldr r3,[r3]
1a  while:    mul r0,r1,r0
1b              cmp r1,#N
1c              bhs end
2a              str r3,[r2],#4
2b              add r1,r1,#1
2c              b while
3a  end:      adr r2,product
3b              str r0,[r2]
    
```

$t_0 = 11$

$t_{0-1} = 3$

A sketch of pipeline analysis by abstract interpretation

after b_0

F																	
A																	
Mu																	
Me																	
C																	

b_0

b_1

after b_1

F																	
A																	
Mu																	
Me																	
C																	

F																	
A																	
Mu																	
Me																	
C																	

F	X																
A																	
Mu																	
Me																	
C																	

b_2

after b_2

F	X																
A																	
Mu																	
Me																	
C																	

F	X	X															
A																	
Mu																	
Me																	
C																	

```

0a  __start:  mov r0,#1
0b              mov r1,#0
0c              adr r2,a
0d              adr r3,init_val
0e              ldr r3,[r3]
1a  while:    mul r0,r1,r0
1b              cmp r1,#N
1c              bhs end
2a              str r3,[r2],#4
2b              add r1,r1,#1
2c              b while
3a  end:      adr r2,product
3b              str r0,[r2]
    
```

$t_0 = 11$

$t_{0-1} = 3$

A sketch of pipeline analysis by abstract interpretation

after b_0

F																	
A																	
Mu																	
Me																	
C																	

after b_1

F																	
A																	
Mu																	
Me																	
C																	

F																	
A																	
Mu																	
Me																	
C																	

fix point!

F																	
A																	
Mu																	
Me																	
C																	

b_0

b_1

b_2

after b_2

F	X																
A																	
Mu																	
Me																	
C																	

F	X	X															
A																	
Mu																	
Me																	
C																	

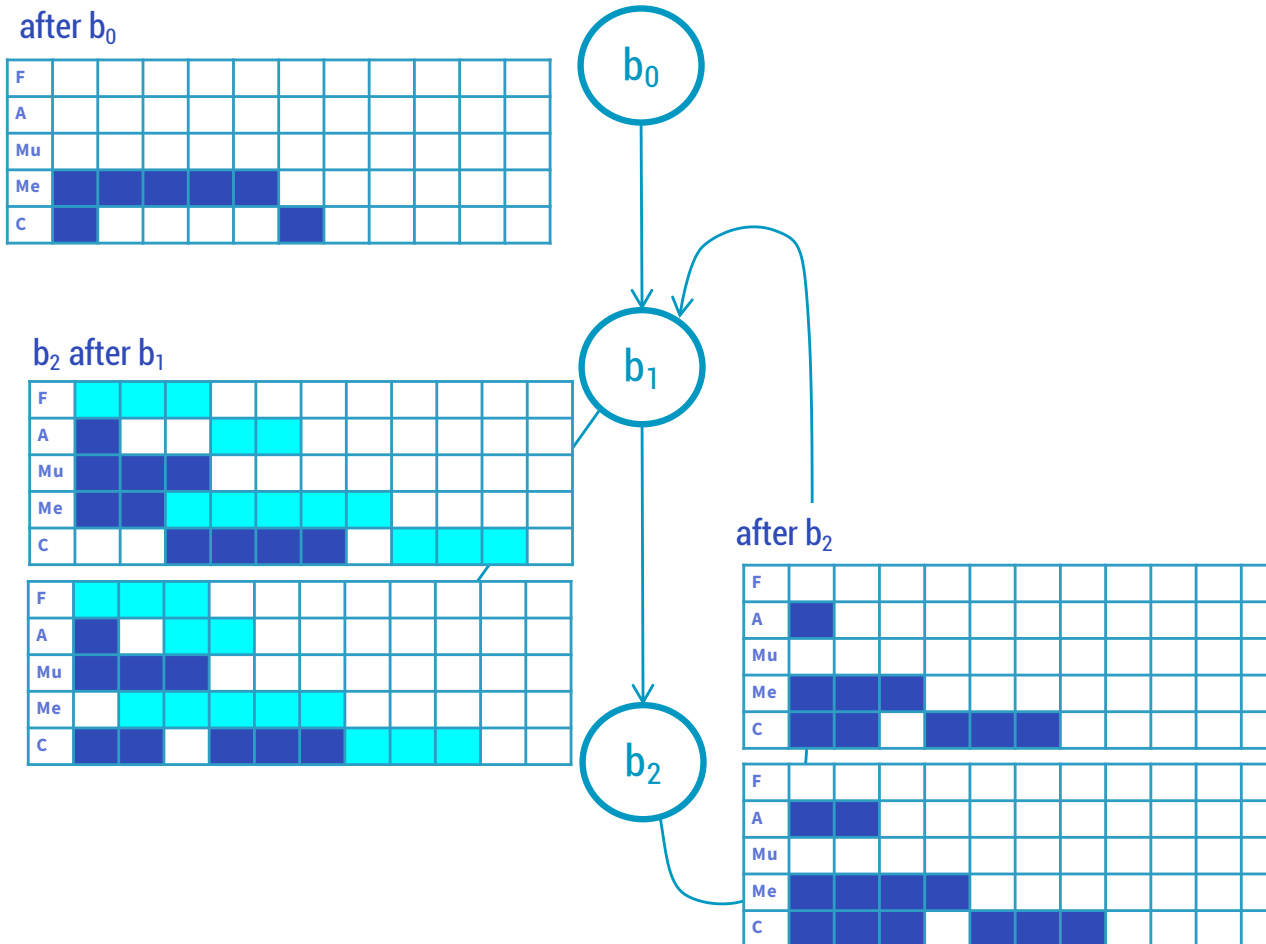
```

0a  __start:  mov r0,#1
0b                mov r1,#0
0c                adr r2,a
0d                adr r3,init_val
0e                ldr r3,[r3]
1a  while:    mul r0,r1,r0
1b                cmp r1,#N
1c                bhs end
2a                str r3,[r2],#4
2b                add r1,r1,#1
2c                b while
3a  end:      adr r2,product
3b                str r0,[r2]
    
```

$t_0 = 11$

$t_{0-1} = 3$

A sketch of pipeline analysis by abstract interpretation



```

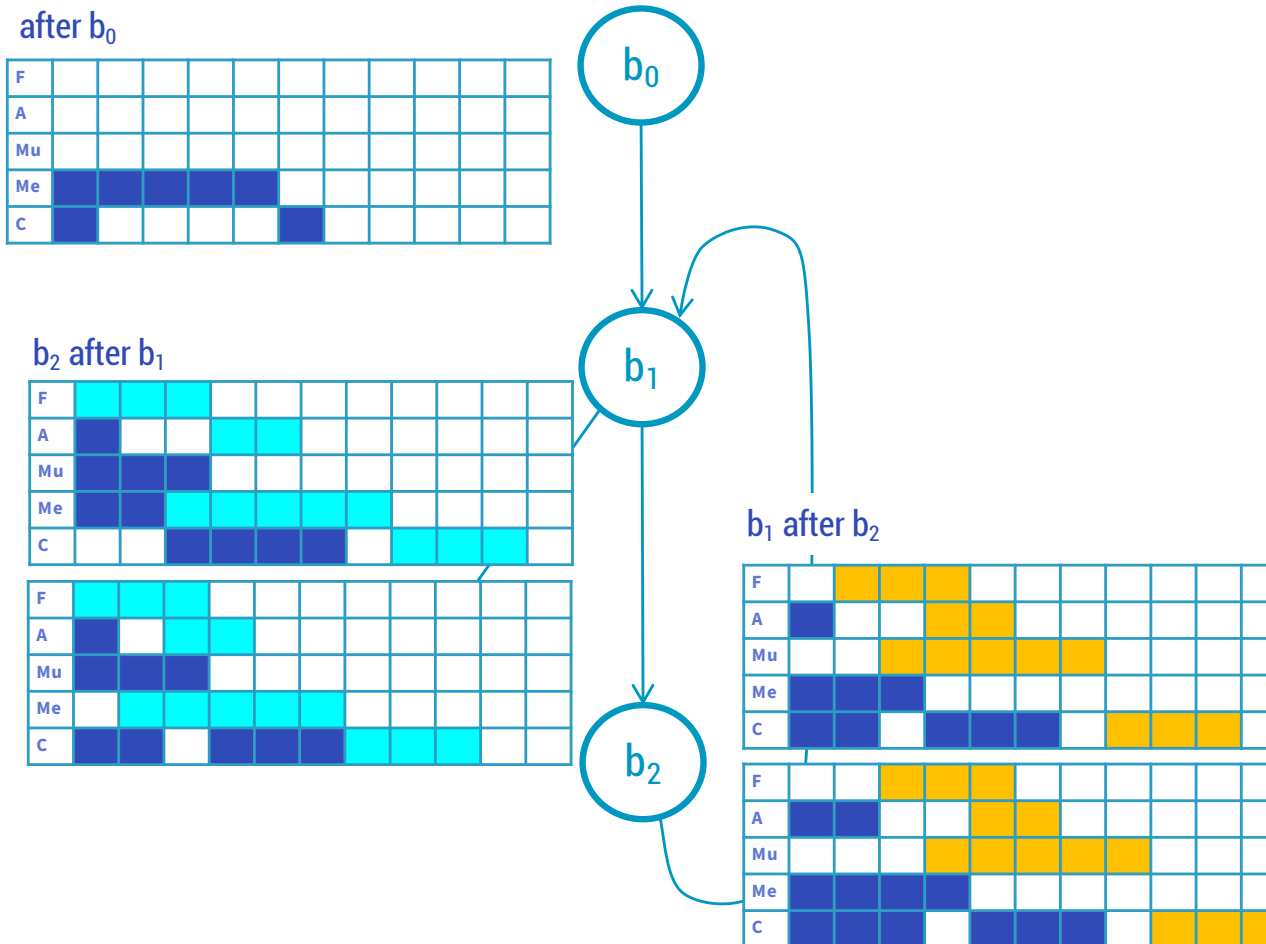
0a  __start:  mov r0,#1
0b              mov r1,#0
0c              adr r2,a
0d              adr r3,init_val
0e              ldr r3,[r3]
1a  while:    mul r0,r1,r0
1b              cmp r1,#N
1c              bhs end
2a              str r3,[r2],#4
2b              add r1,r1,#1
2c              b while
3a  end:      adr r2,product
3b              str r0,[r2]
    
```

$$t_0 = 11$$

$$t_{0-1} = 3$$

$$t_{1-2} = \max(4,3) = 4$$

A sketch of pipeline analysis by abstract interpretation



0a	__start:	mov r0,#1
0b		mov r1,#0
0c		adr r2,a
0d		adr r3,init_val
0e		ldr r3,[r3]
1a	while:	mul r0,r1,r0
1b		cmp r1,#N
1c		bhs end
2a		str r3,[r2],#4
2b		add r1,r1,#1
2c		b while
3a	end:	adr r2,product
3b		str r0,[r2]

$$t_0 = 11$$

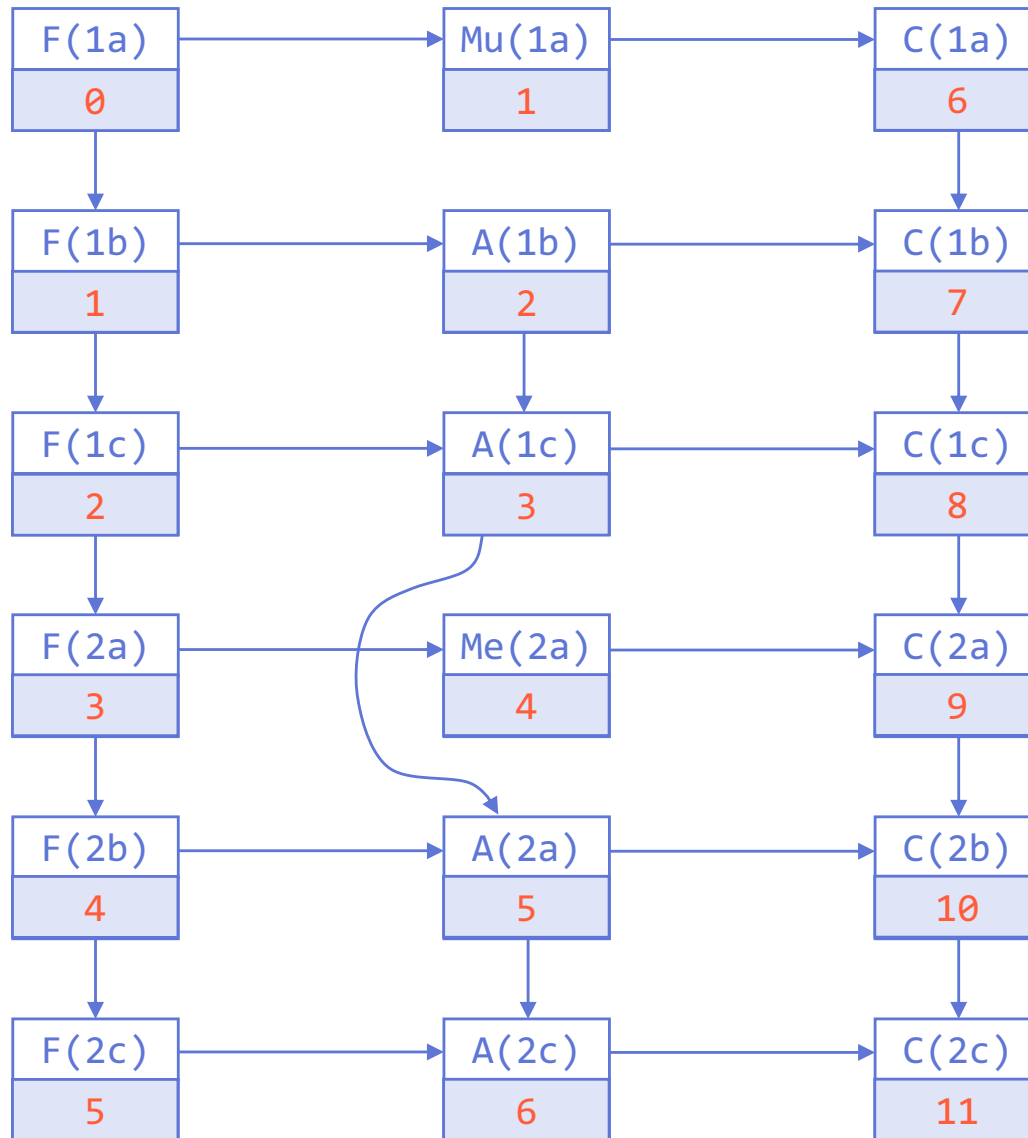
$$t_{0-1} = 3$$

$$t_{1-2} = \max(4,3) = 4$$

$$t_{2-1} = \max(4,4) = 4$$

$$t_{0-1-2} = 11 + 3 + 4 = 18$$

"Local" pipeline analysis based on execution graphs



0a	__start:	mov r0,#1
0b		mov r1,#0
0c		adr r2,a
0d		adr r3,init_val
0e		ldr r3,[r3]
1a	while:	mul r0,r1,r0
1b		cmp r1,#N
1c		bhs end
2a		str r3,[r2],#4
2b		add r1,r1,#1
2c		b while
3a	end:	adr r2,product
3b		str r0,[r2]

$t_{1-2} = 3$ cycles



"Local" pipeline analysis based on execution graphs



Parameterized initial state

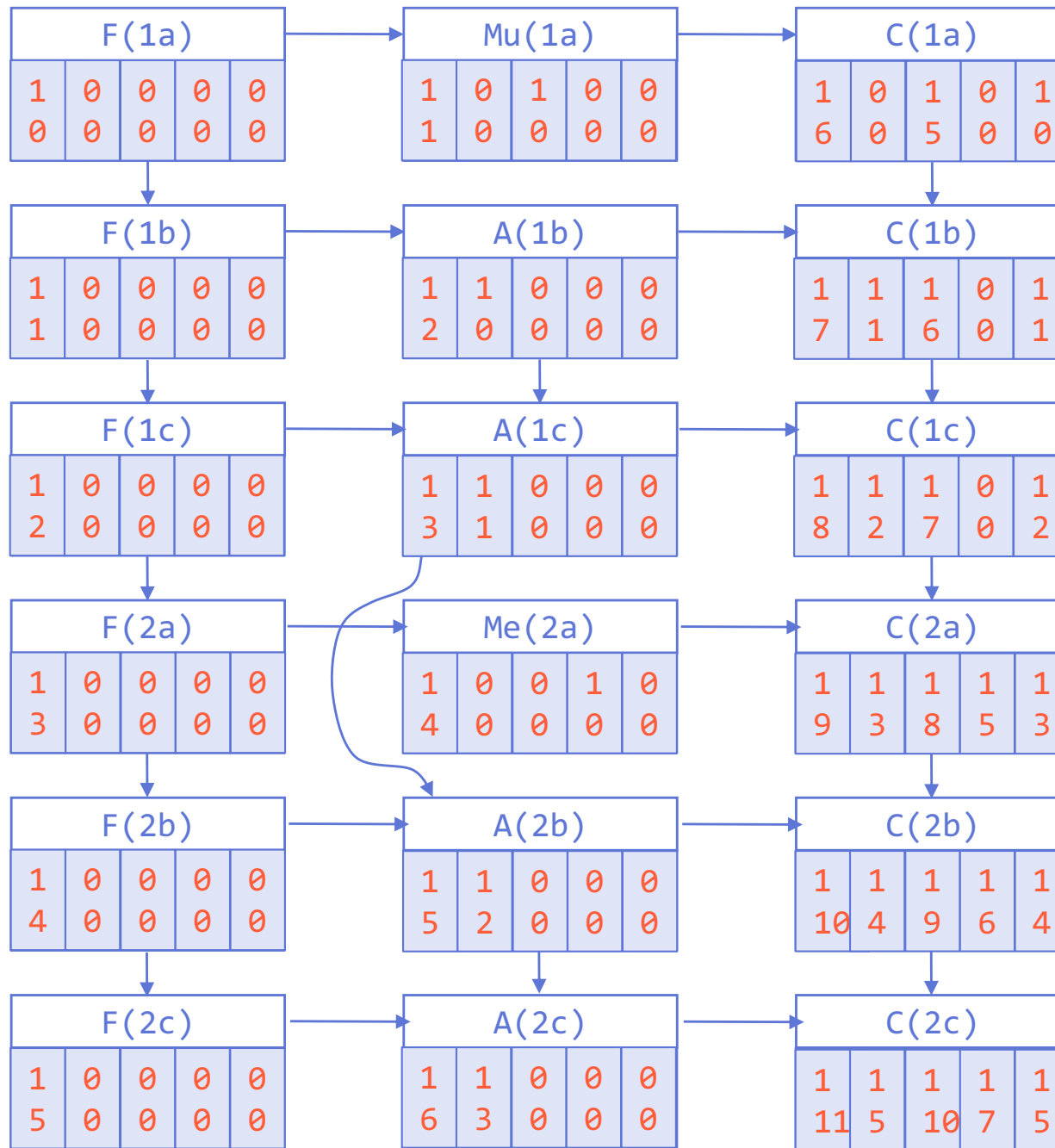
0a	__start:	mov r0,#1
0b		mov r1,#0
0c		adr r2,a
0d		adr r3,init_val
0e		ldr r3,[r3]
1a	while:	mul r0,r1,r0
1b		cmp r1,#N
1c		bhs end
2a		str r3,[r2],#4
2b		add r1,r1,#1
2c		b while
3a	end:	adr r2,product
3b		str r0,[r2]

each resource has an availability date

starting date of a node

F	A	Mu	Me	C	r0	r1	r2	r3

e_i = depends on resource?
 d_i = delay after resource
becomes available



```

0a __start:  mov r0,#1
0b           mov r1,#0
0c           adr r2,a
0d           adr r3,init_val
0e           ldr r3,[r3]
1a while:   mul r0,r1,r0
1b           cmp r1,#N
1c           bhs end
2a           str r3,[r2],#4
2b           add r1,r1,#1
2c           b while
3a end:      adr r2,product
3b           str r0,[r2]

```

F	A	Mu	Me	C

$t_{1-2} = ?$

"Local" pipeline analysis based on execution graphs



C(1c)				
1	1	1	0	1
8	2	7	0	2

C(2c)				
1	1	1	1	1
11	5	10	7	5

F	A	Mu	Me	C

$$t_{C(1c)} = \max(t_F+8, t_A+2, t_{Mu}+7, \quad, t_C+2)$$

$$t_{C(2c)} = \max(t_F+11, t_A+5, t_{Mu}+10, t_{Me}+7, t_C+5) \leq t_C + 6$$

$t_{Me} \leq t_C - 1$

$$t_{C(2c)} - t_{C(1c)} = \max(11-8, 5-2, 10-7, 6-2, 5-2)$$

$$t_{1-2} \leq 4 \text{ cycles}$$

Handling variable instruction latencies



Variable latencies?

- depending on operand values (e.g. multiplication)
- depending on the state of some components
 - example: instruction or data cache

Timing anomalies

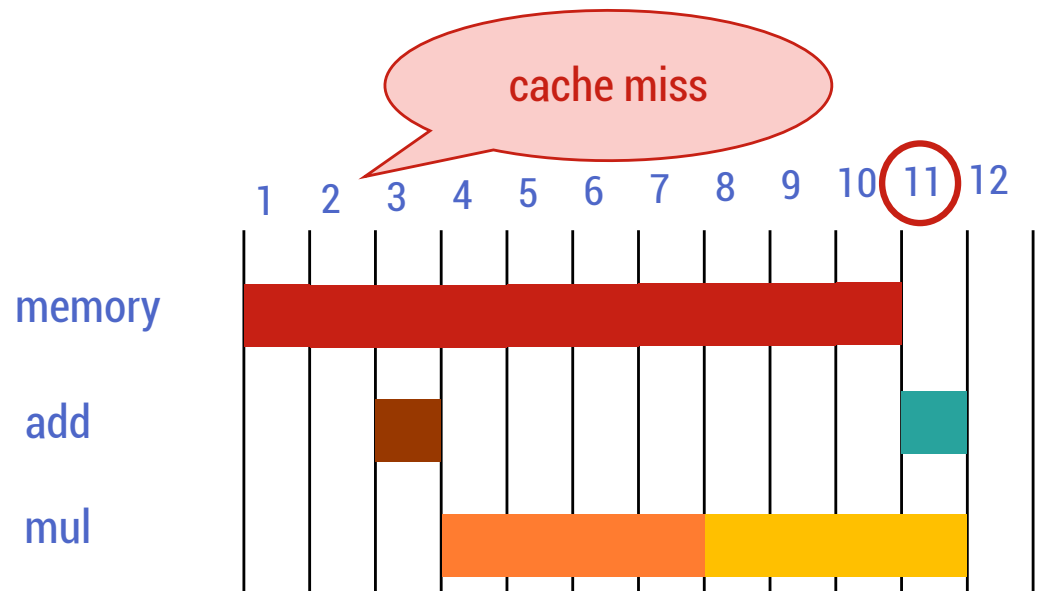
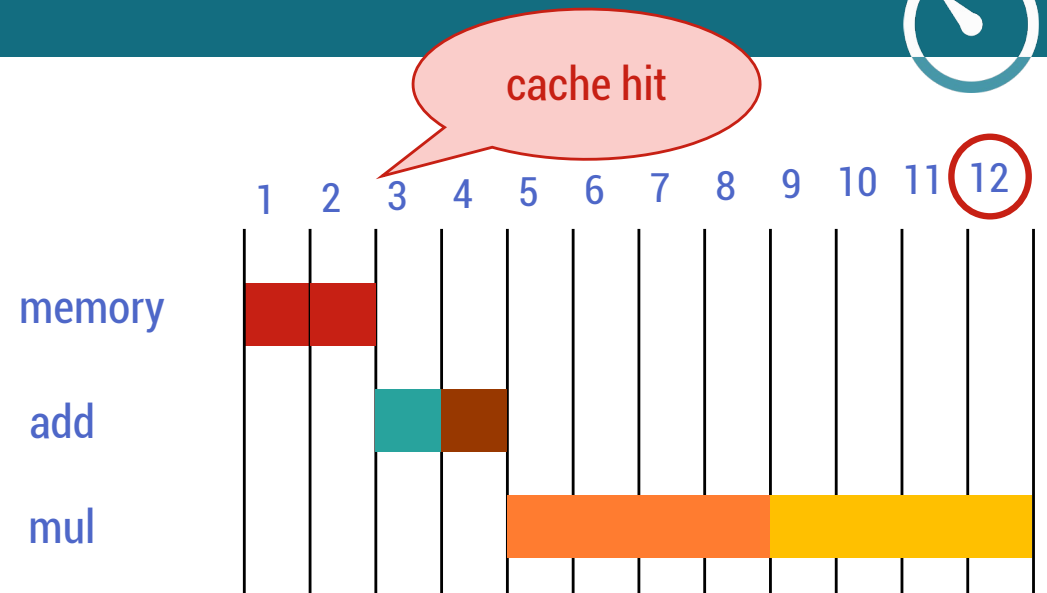
- the local worst-case might not be the global worst-case

Timing anomalies



decode
cycle

- 1 ■ ldr r4,[r3]
- 2 ■ add r5,r4,r4
- 3 ■ add r11,r10,r10
- 4 ■ mul r11,r8,r11
- 5 ■ mul r19,r1,r2



Impact of timing anomalies



How can we get safe WCET estimations?

- by considering all possible latency values, one by one
 - additional states
 - additional cost values for basic blocks
 - changes in the ILP formulation?
- by considering the latency as a parameter when determining the cost of a basic block ?

Cache analysis



What do we want to compute?



Classify accesses to cache memories

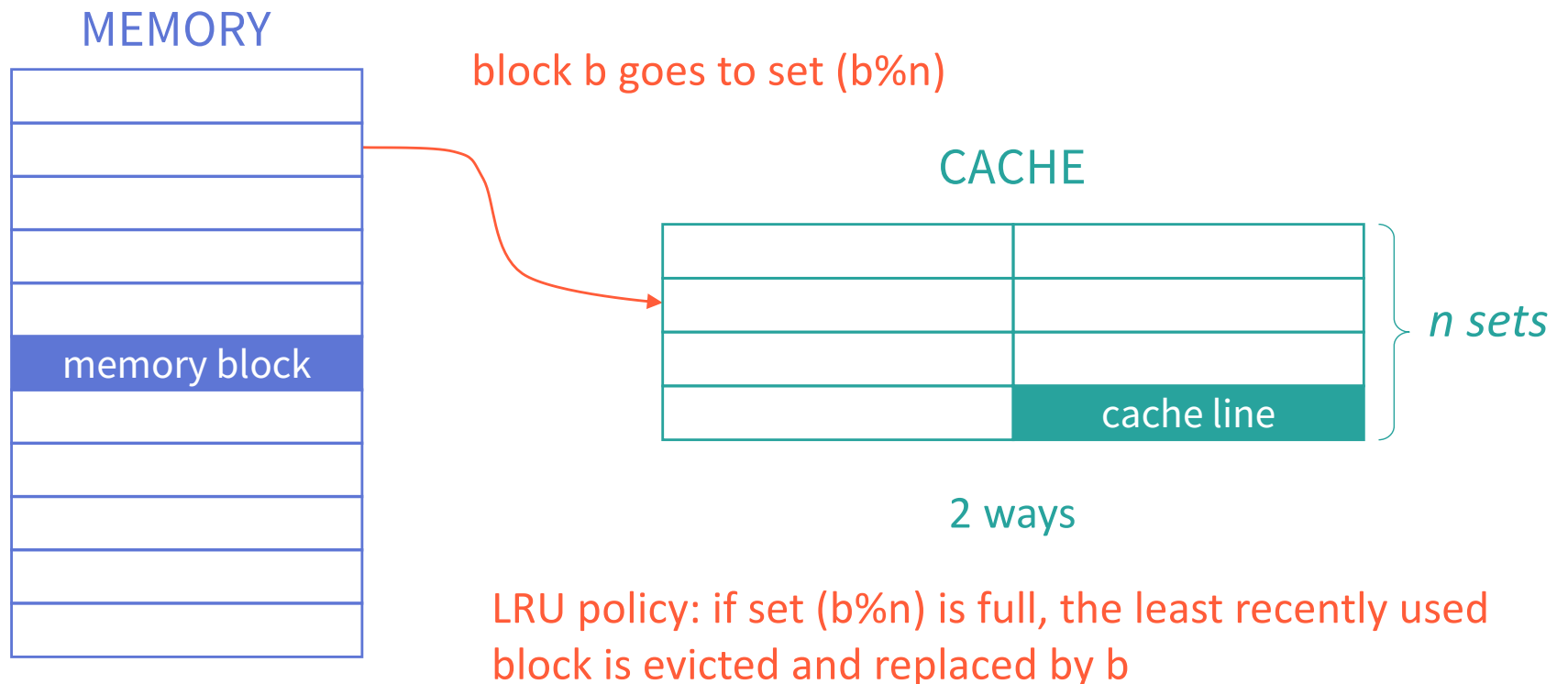
- AlwaysHit, Always Miss
 - FirstMiss
 - ? (NotClassified)
- latency is known
- latency is unknown
-

Cache analysis by Abstract Interpretation



Assumptions

- set-associative **instruction** cache
- with an **LRU** replacement policy



Cache analysis by Abstract Interpretation



(Concrete) State of a cache set

sequence of accesses: a-b-c-d

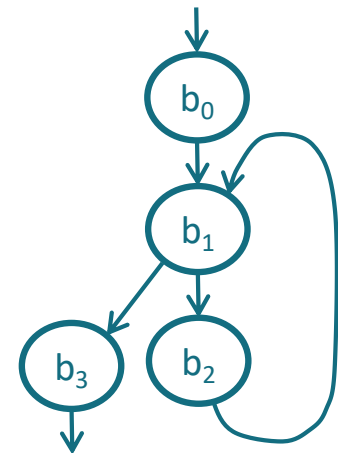
<i>contents of the set</i>			
a	c	d	b

<i>age (LRU)</i>			
3	1	0	2

d	0
c	1
b	2
a	3

Abstract interpretation-based analysis

- compute possible cache states at each point in the program
- two analyses:
 - MAY: which memory blocks *may* be in the cache?
 - MUST: ... *must* ?
- for each analysis:
 - abstract cache states (ACS)
 - update and join functions

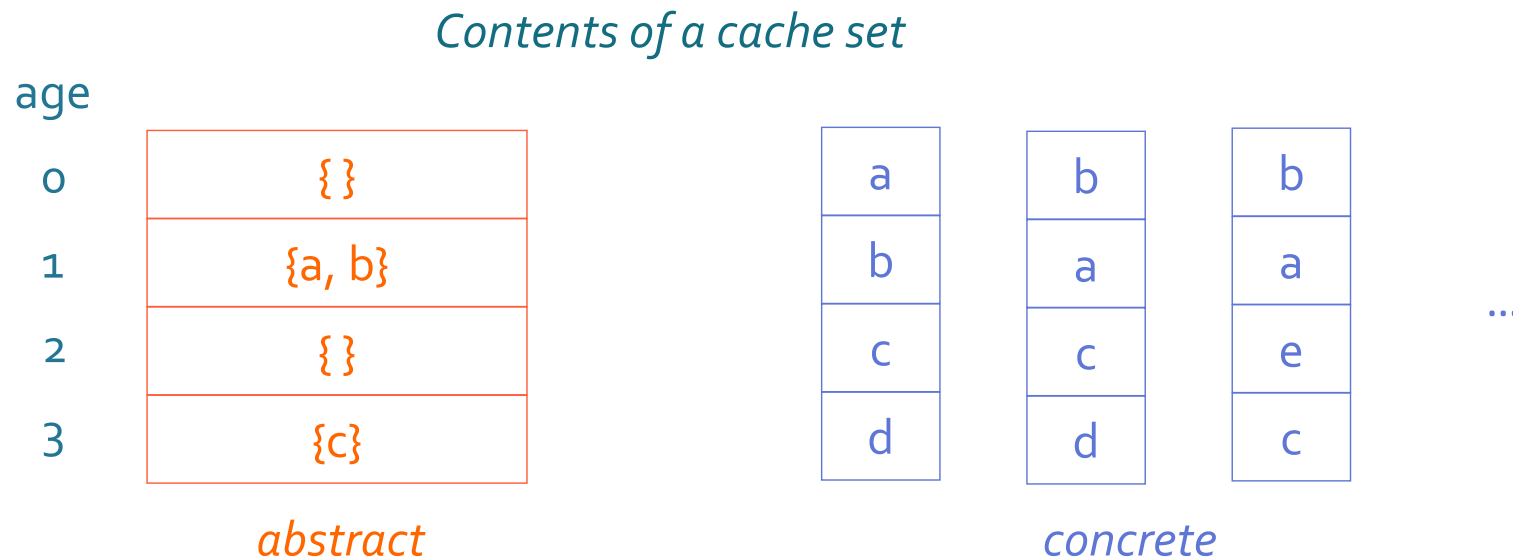


Cache analysis by Abstract Interpretation



MUST analysis

- abstract cache state = upper bounds on block ages



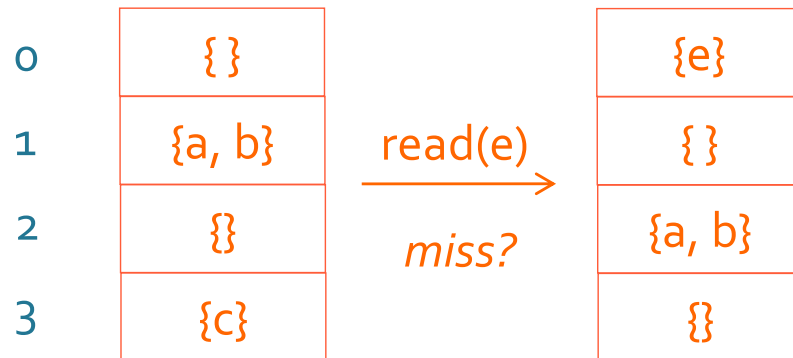
Cache analysis by Abstract Interpretation



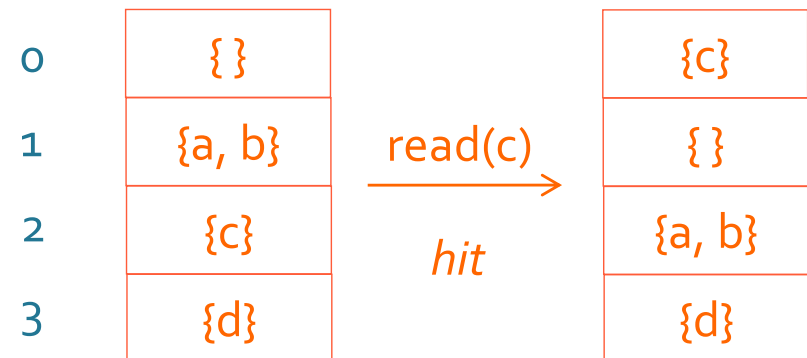
MUST analysis

- update() function

age



age

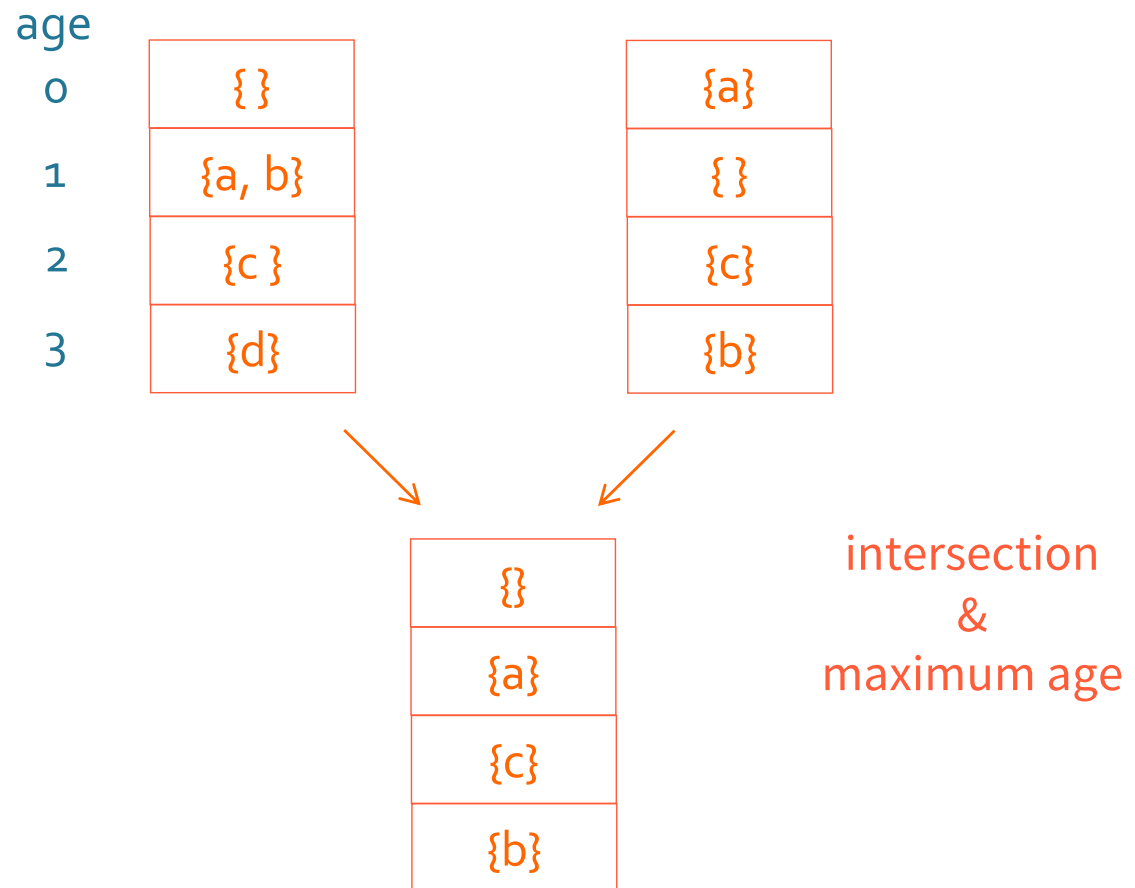


Cache analysis by Abstract Interpretation



MUST analysis

- join() function

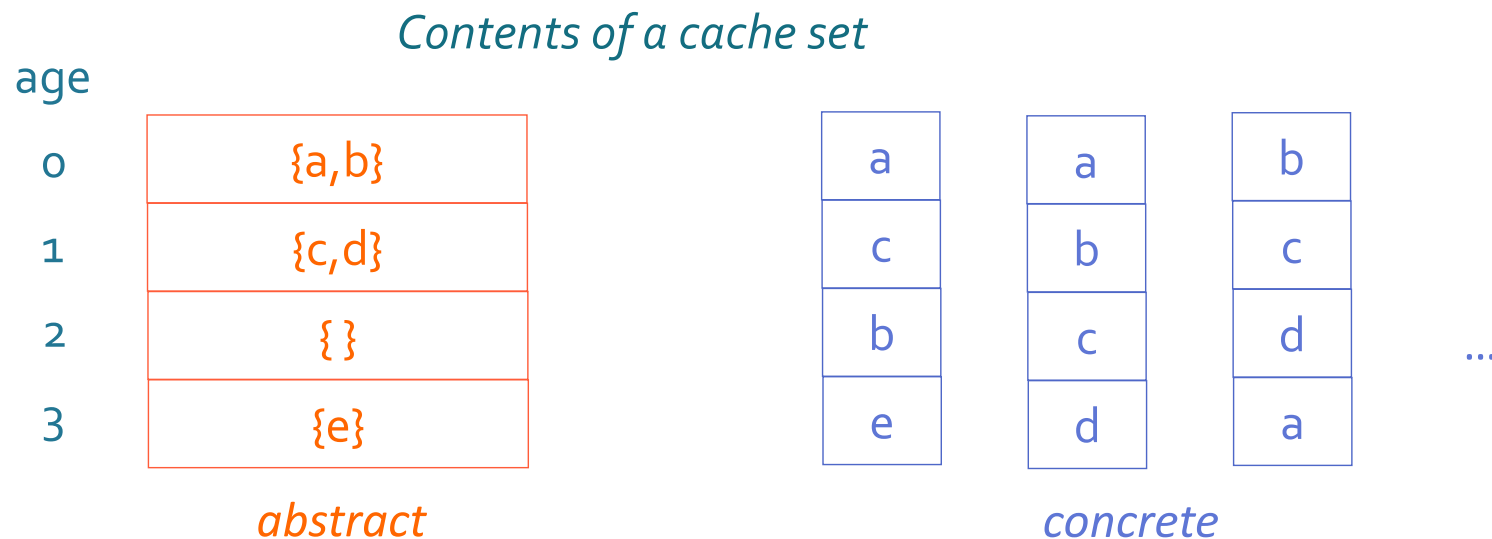


Cache analysis by Abstract Interpretation



MAY analysis

- abstract cache state = lower bounds on block ages



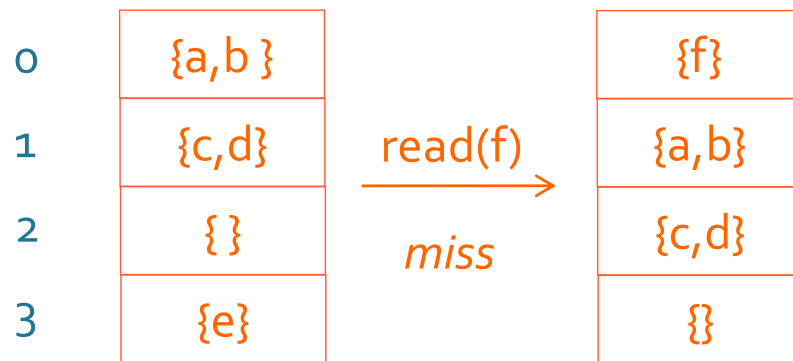
Cache analysis by Abstract Interpretation



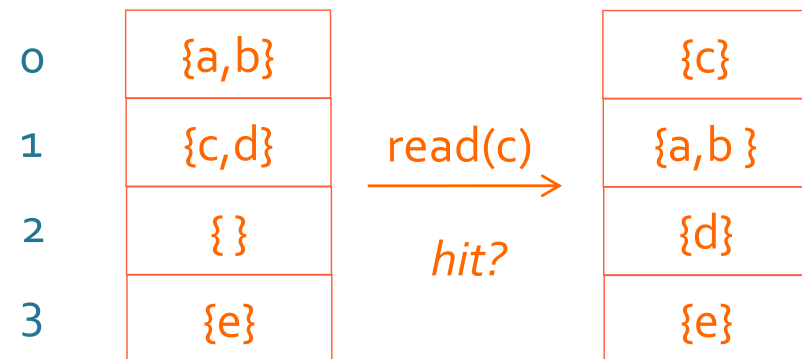
MAY analysis

- `update()` function

age



age

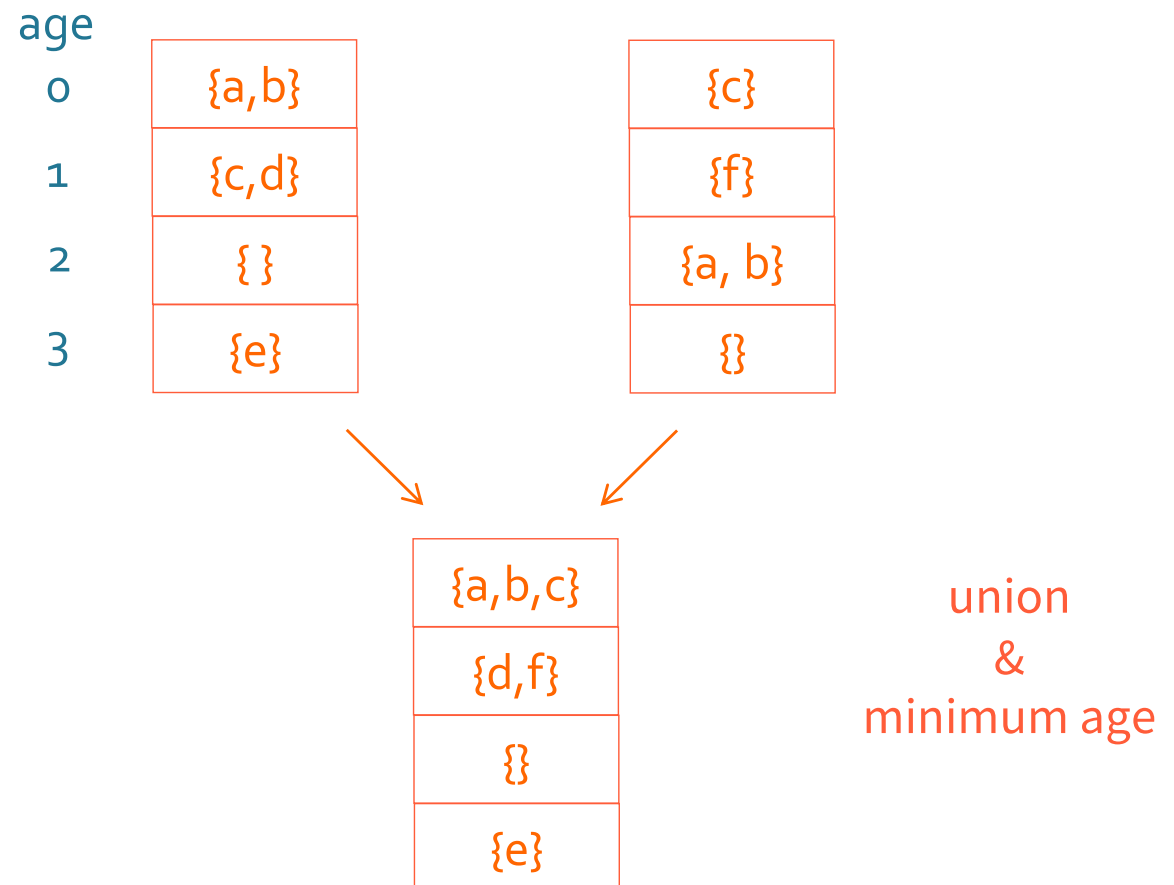


Cache analysis by Abstract Interpretation



MAY analysis

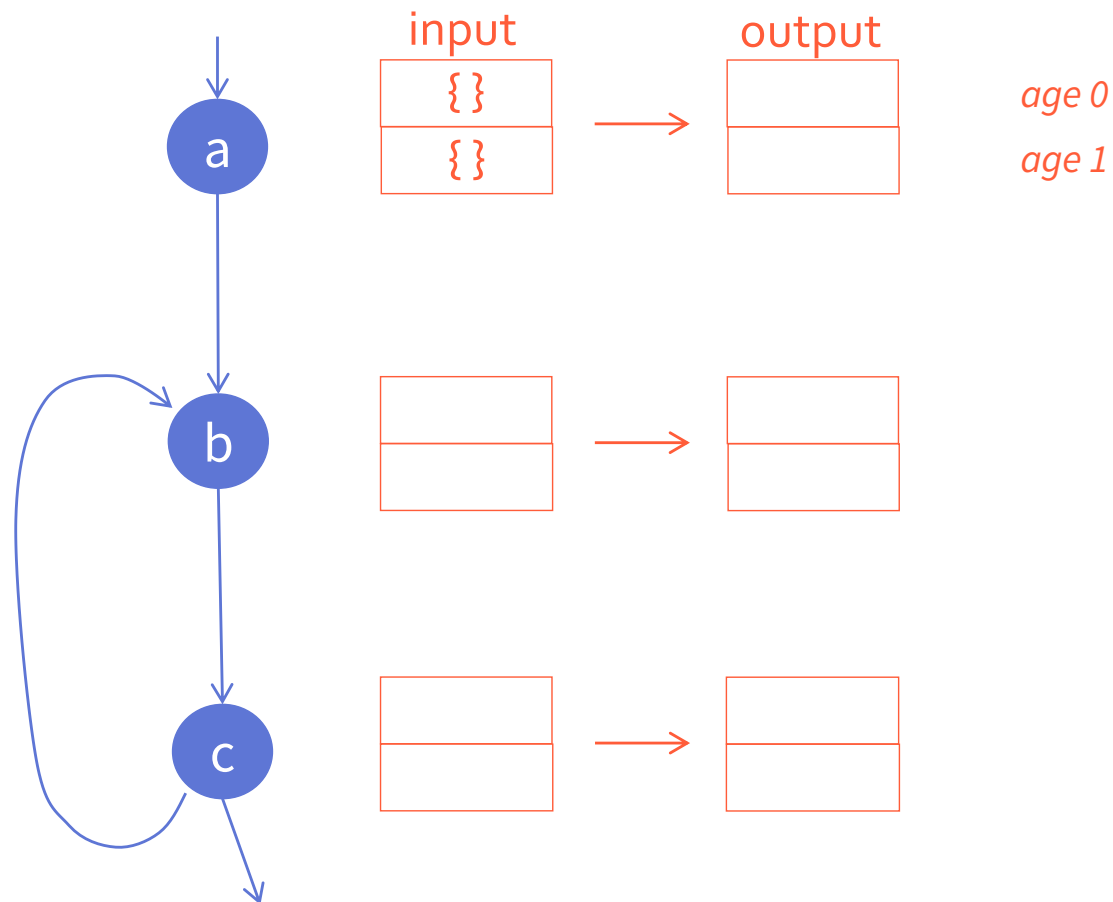
- `join()` function



Cache analysis by Abstract Interpretation



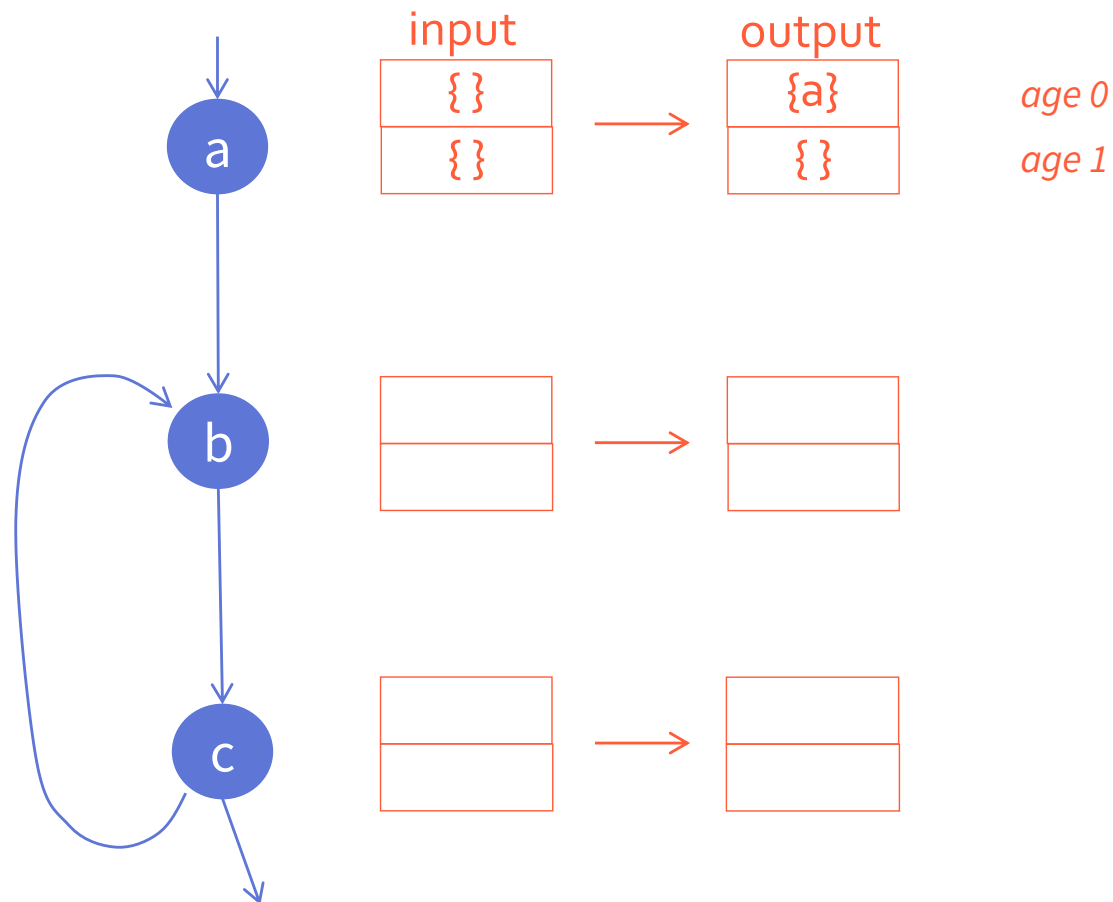
MAY analysis



Cache analysis by Abstract Interpretation



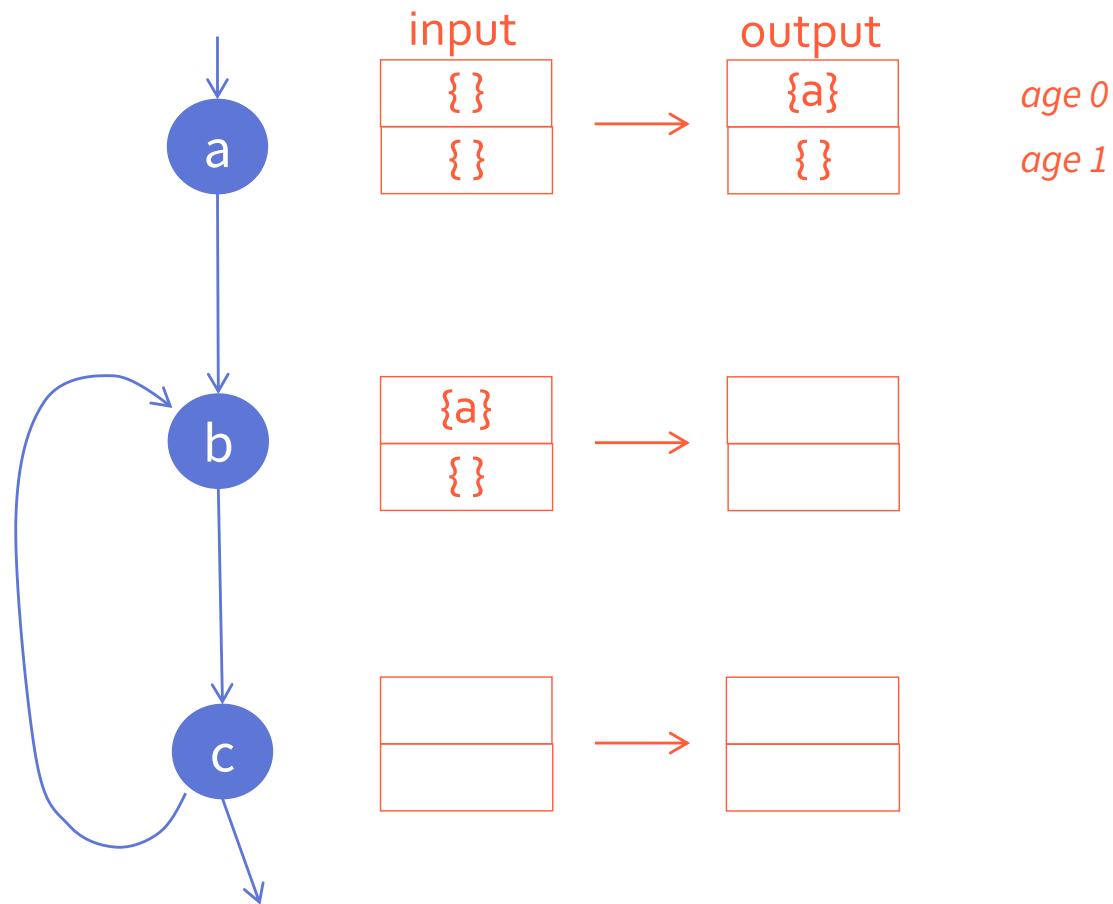
MAY analysis



Cache analysis by Abstract Interpretation



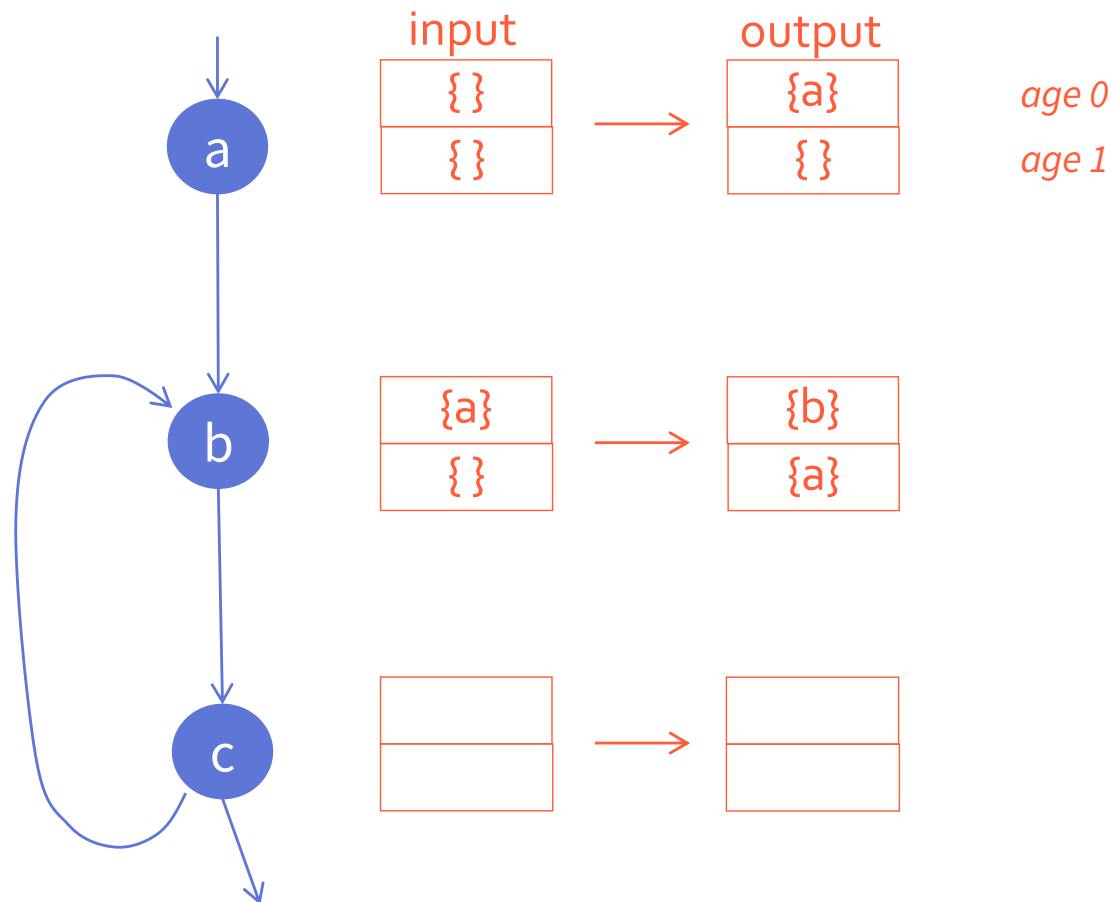
MAY analysis



Cache analysis by Abstract Interpretation



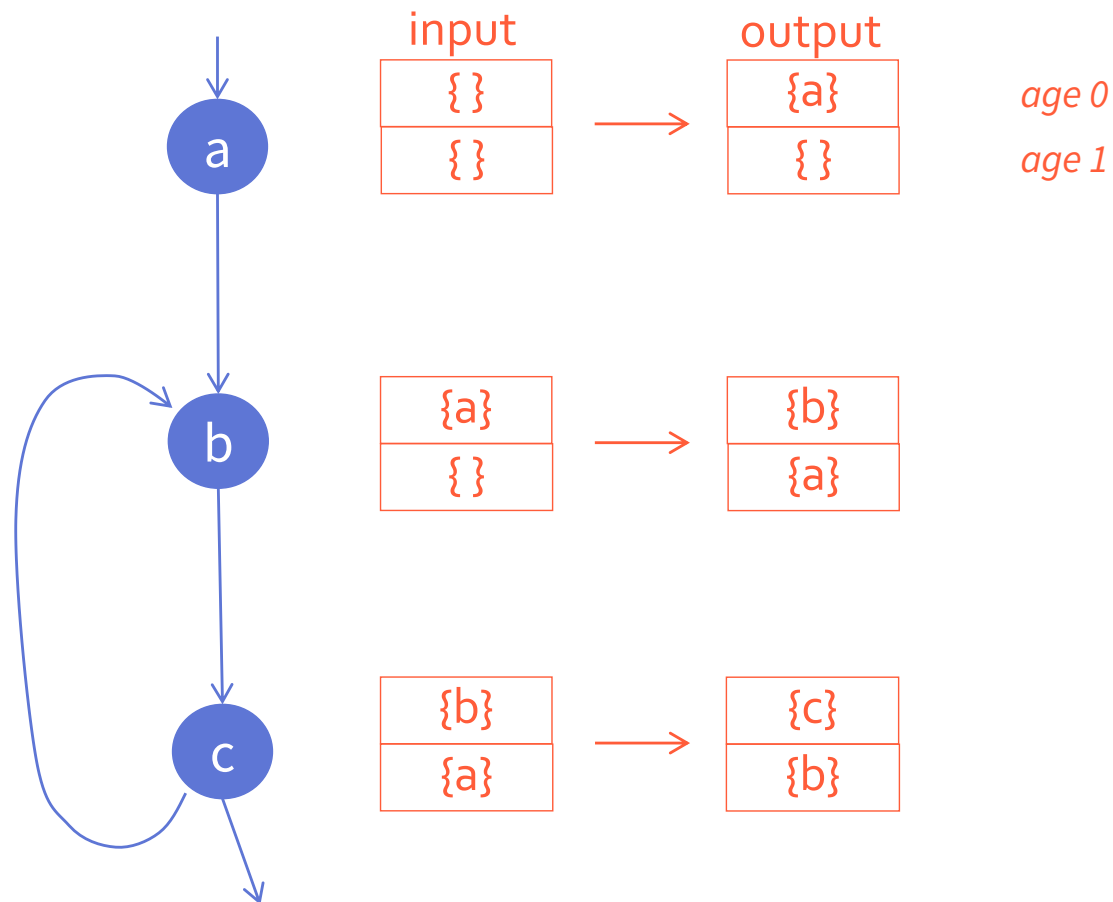
MAY analysis



Cache analysis by Abstract Interpretation



MAY analysis

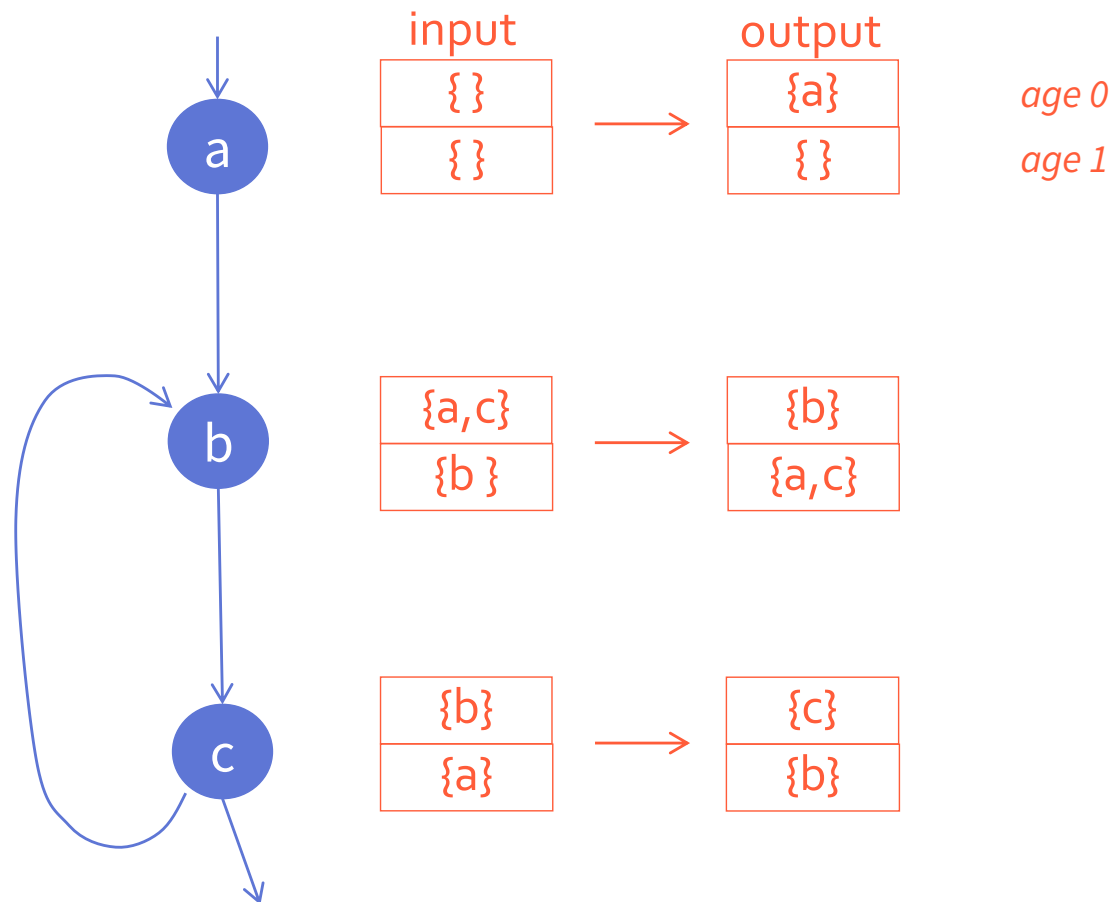


join = union & minimum age

Cache analysis by Abstract Interpretation



MAY analysis

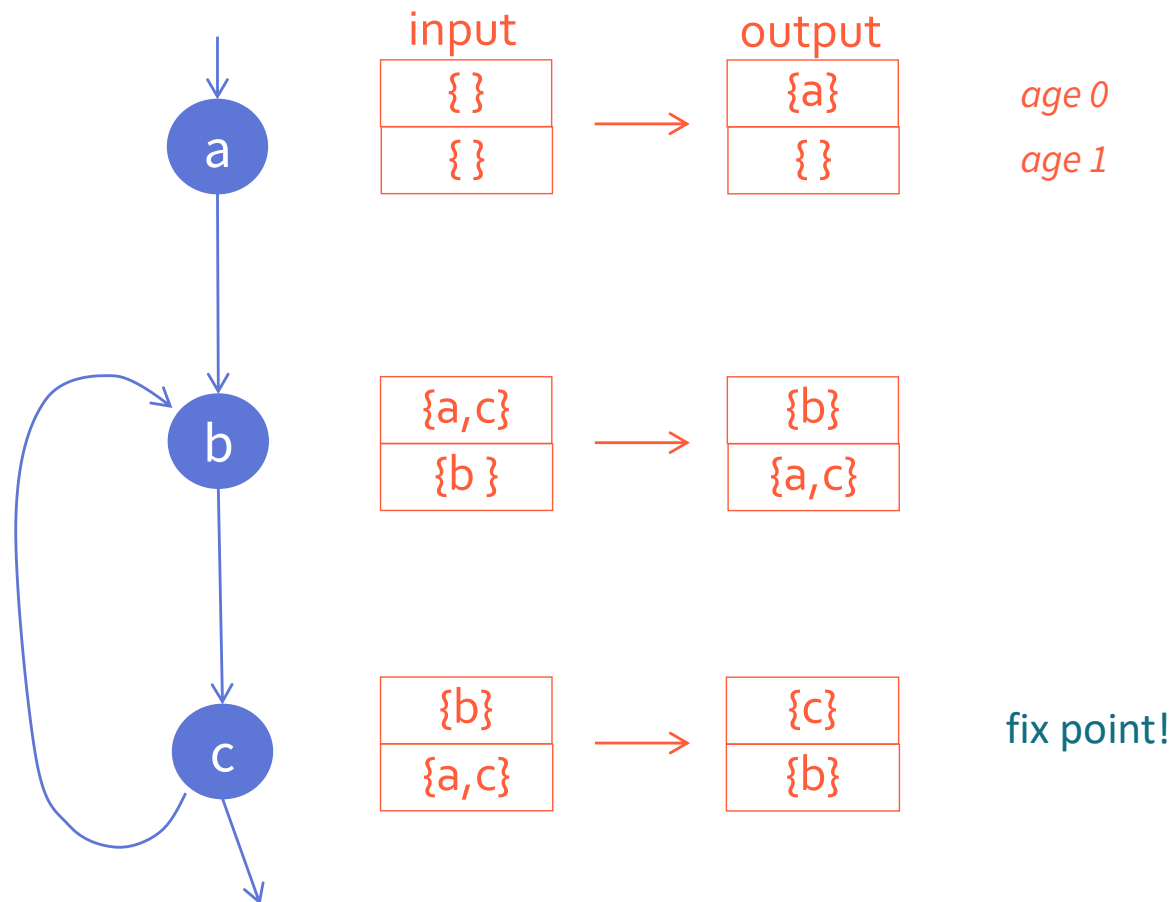


join = union & minimum age

Cache analysis by Abstract Interpretation



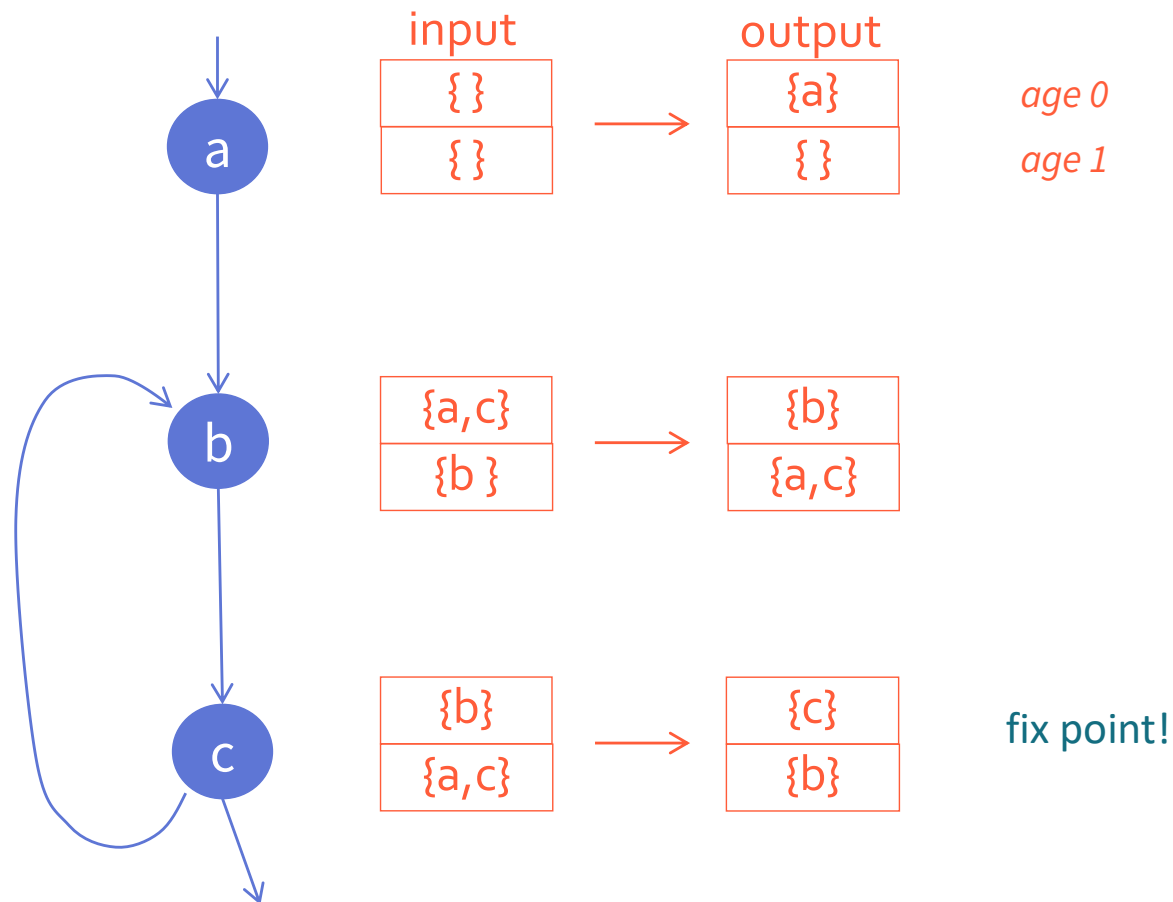
MAY analysis



Cache analysis by Abstract Interpretation



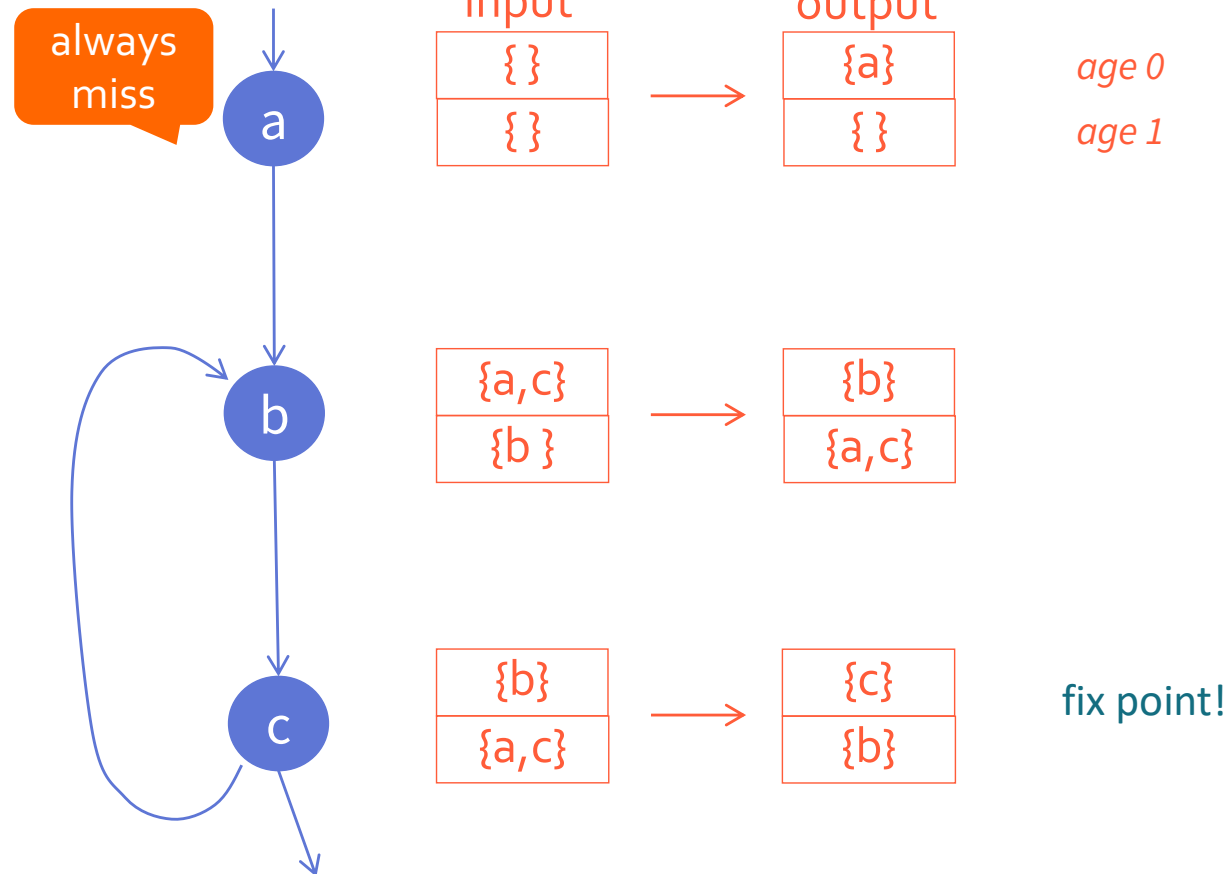
MAY analysis



Cache analysis by Abstract Interpretation



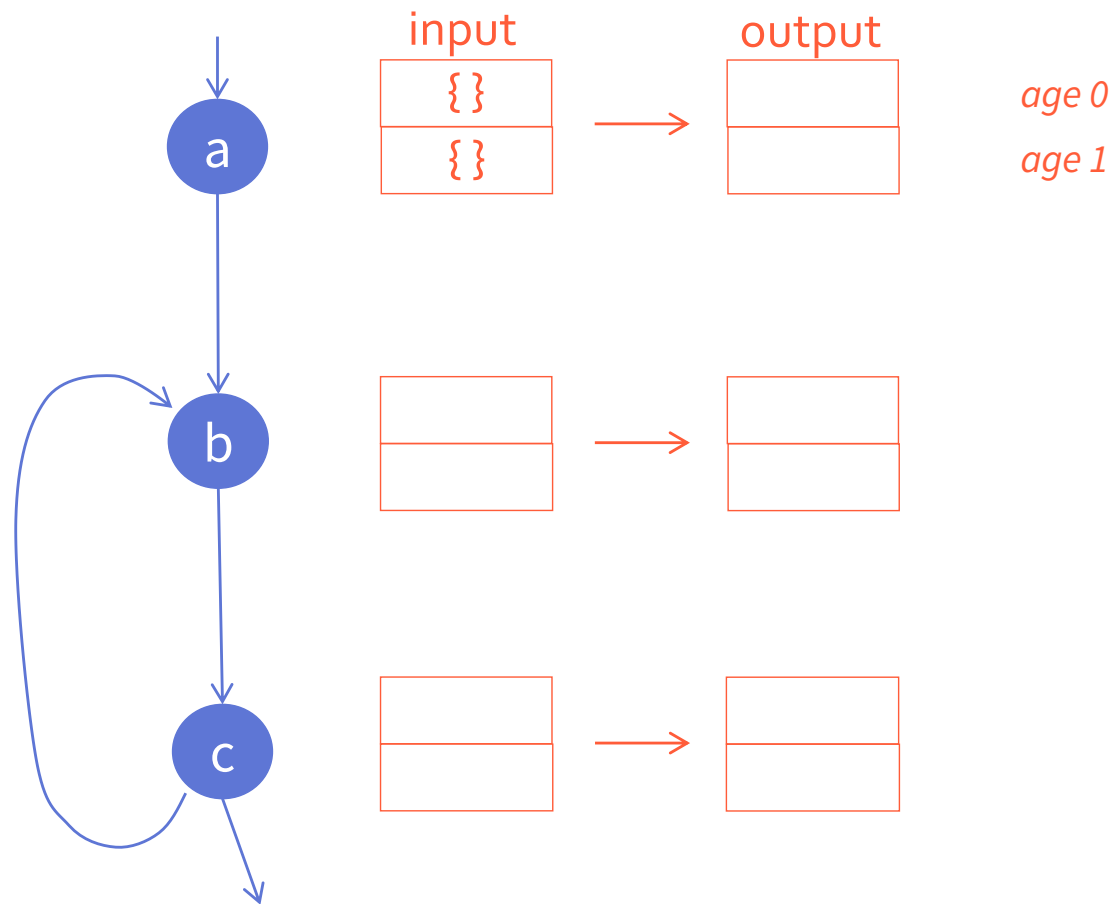
MAY analysis



Cache analysis by Abstract Interpretation



MUST analysis

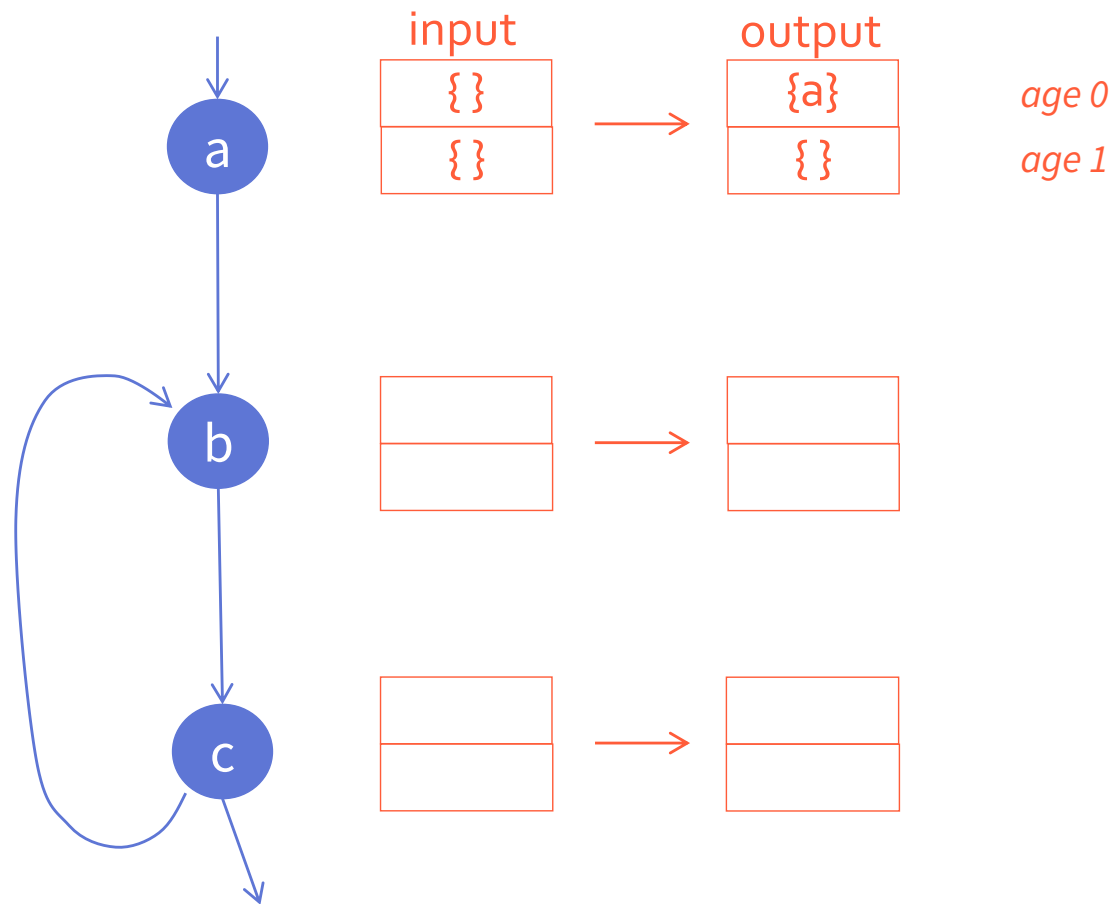


join = intersection & maximum age

Cache analysis by Abstract Interpretation



MUST analysis

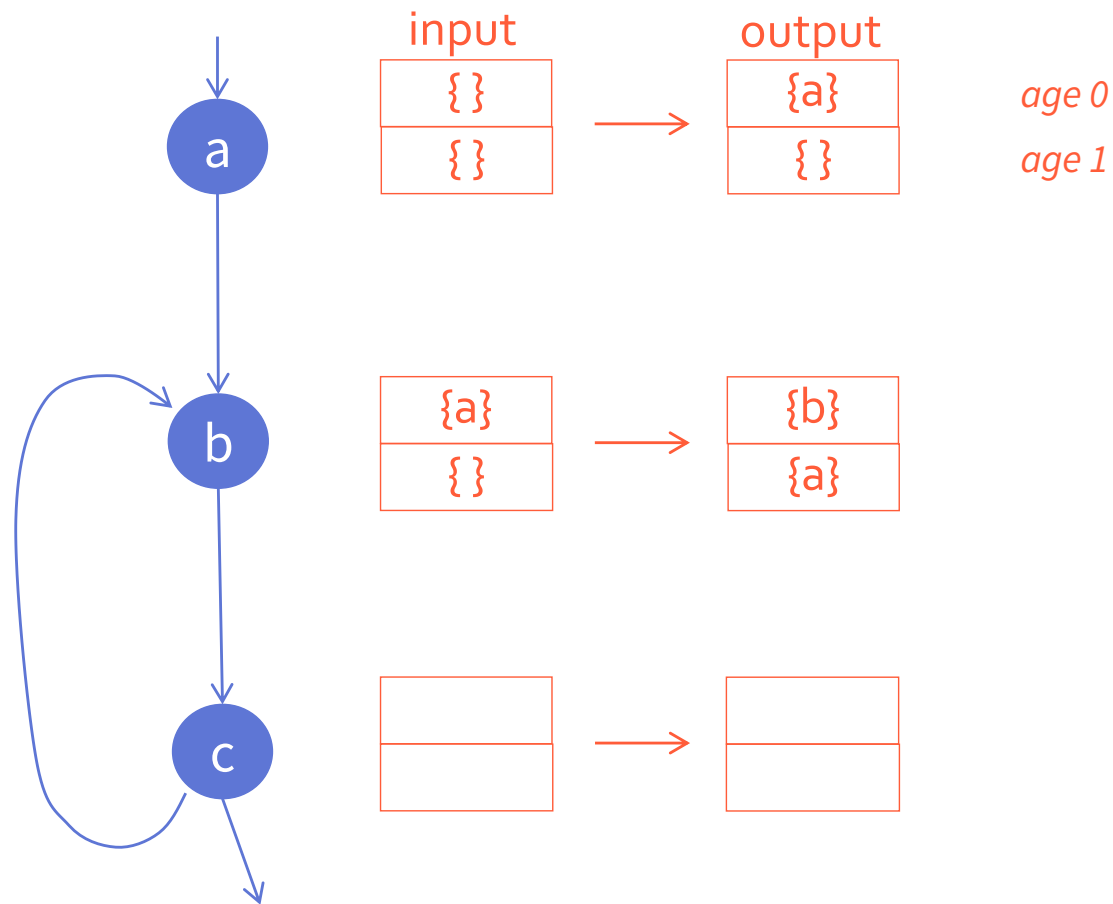


join = intersection & maximum age

Cache analysis by Abstract Interpretation



MUST analysis

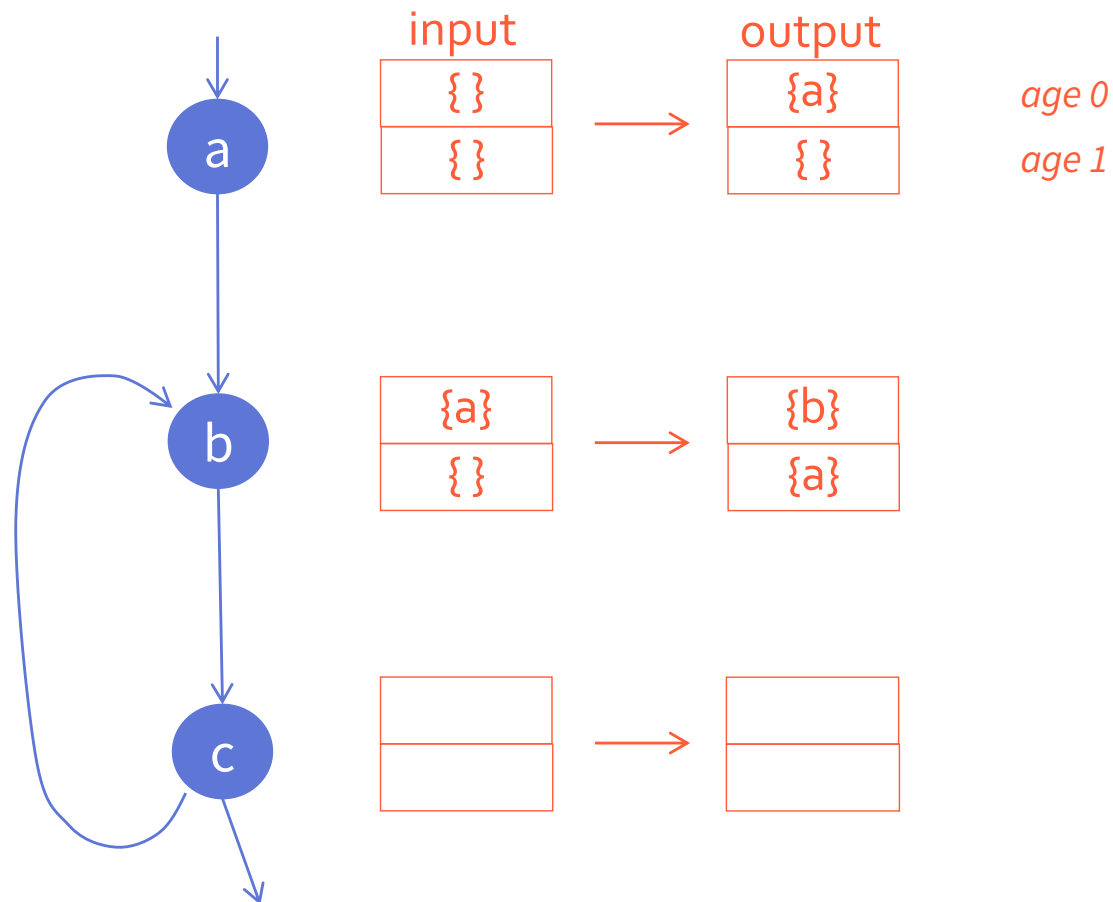


join = intersection & maximum age

Cache analysis by Abstract Interpretation



MUST analysis

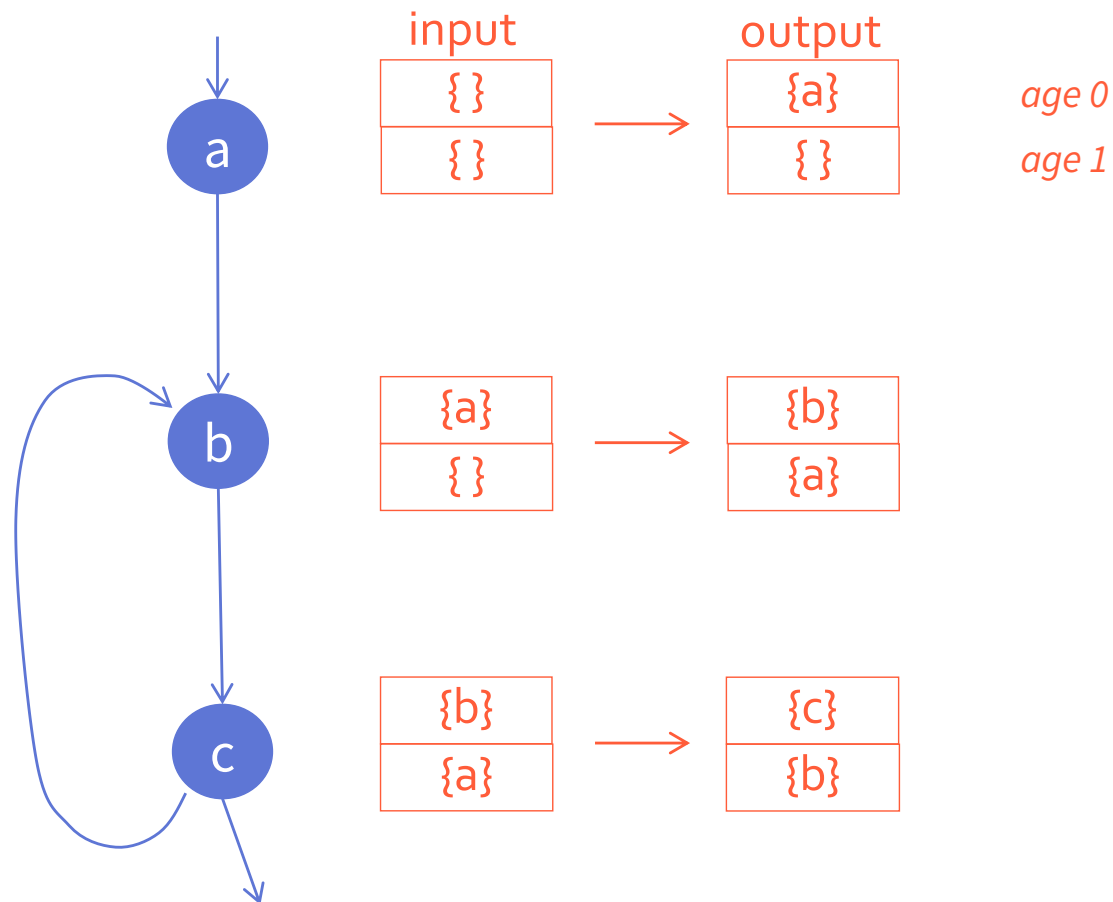


join = intersection & maximum age

Cache analysis by Abstract Interpretation



MUST analysis

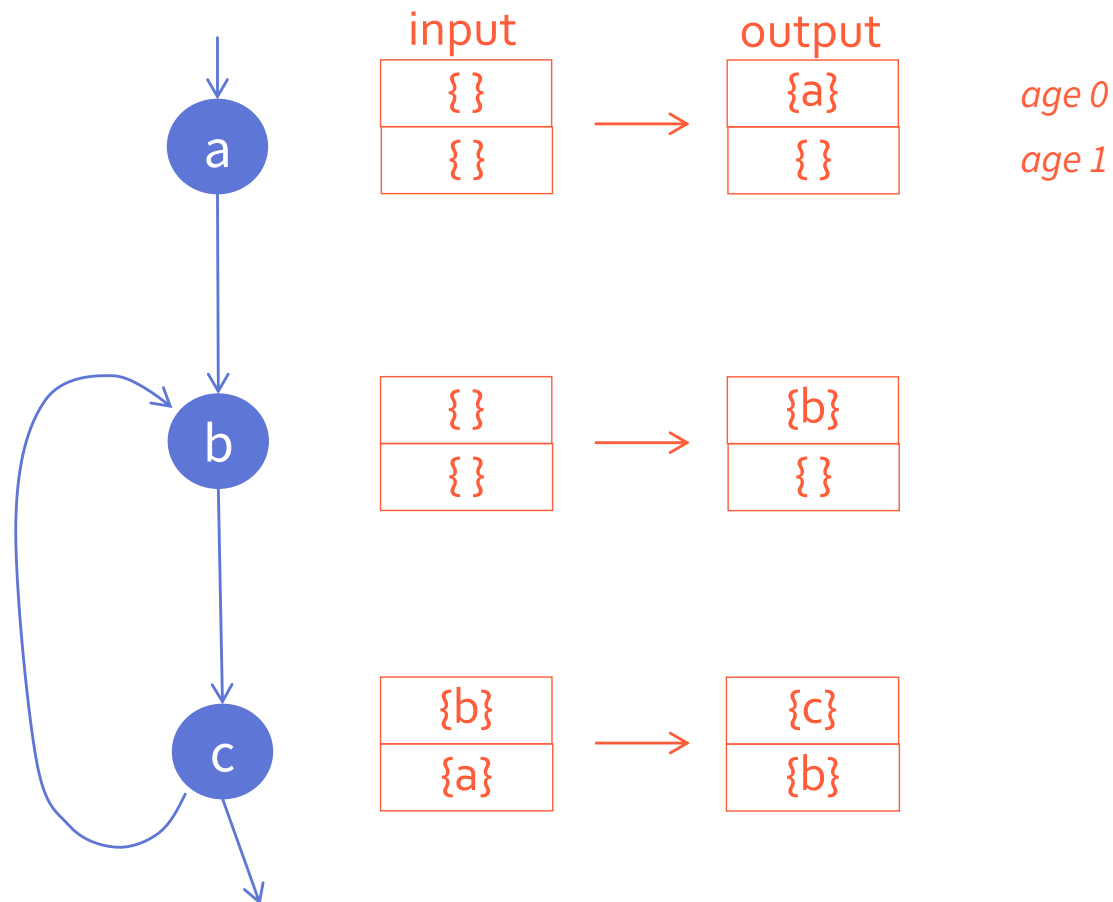


join = intersection & maximum age

Cache analysis by Abstract Interpretation



MUST analysis

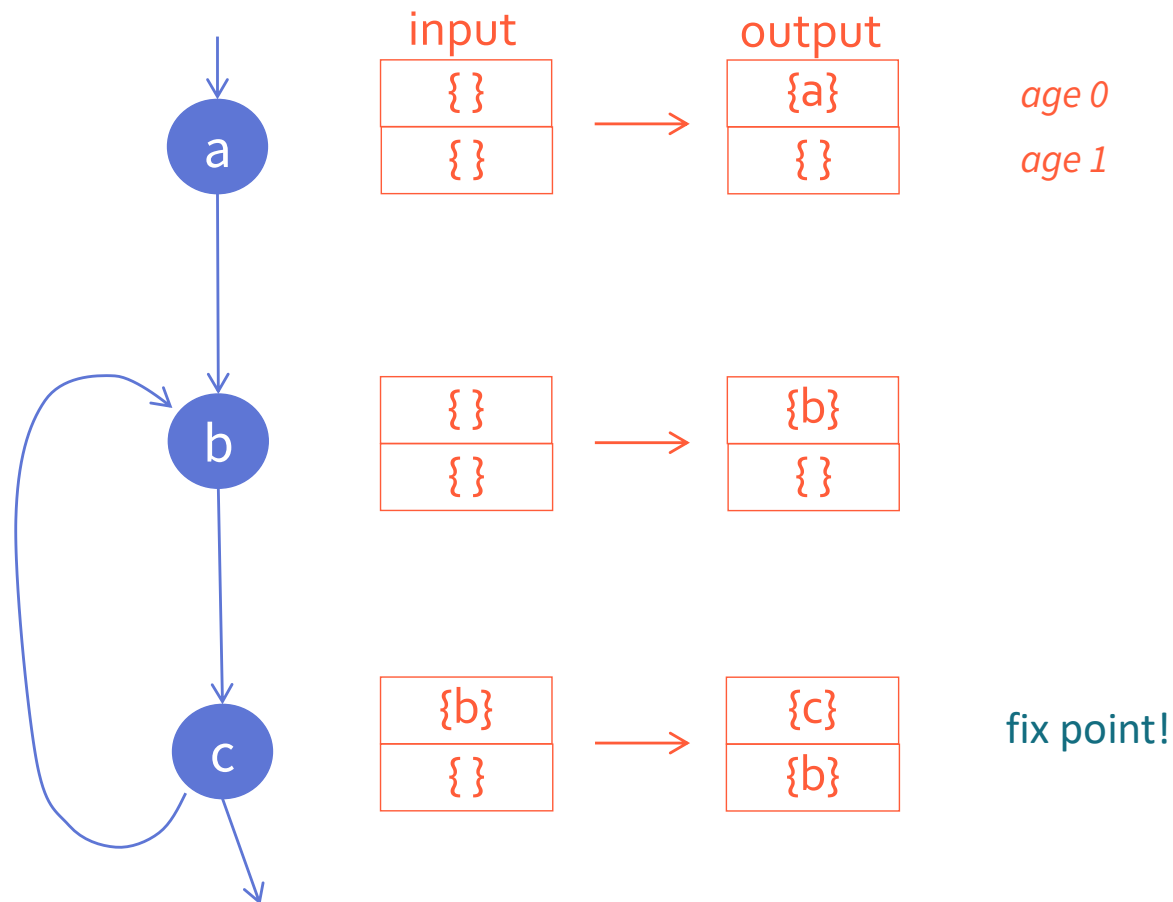


join = intersection & maximum age

Cache analysis by Abstract Interpretation



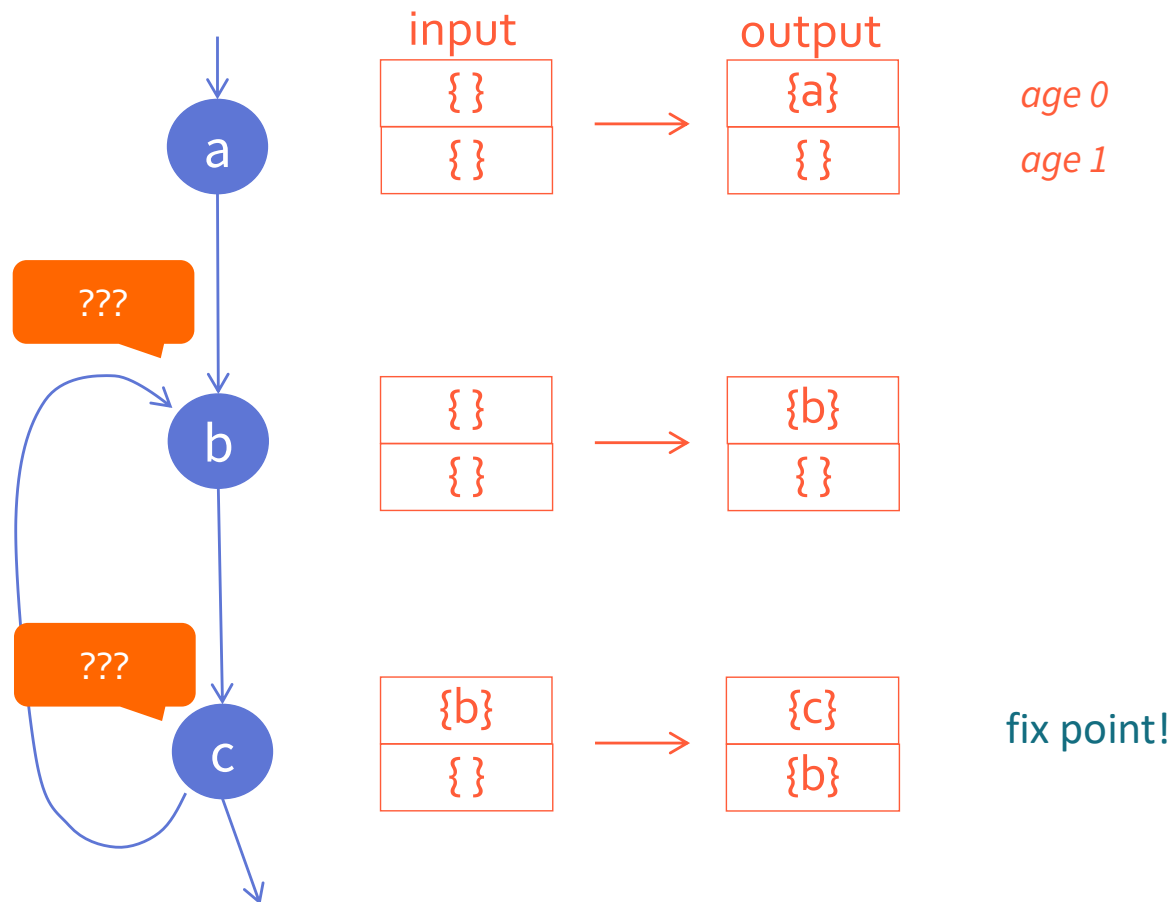
MUST analysis



Cache analysis by Abstract Interpretation



MUST analysis



Cache analysis by Abstract Interpretation



PERSISTENCE analysis

- abstract cache state = maximum ages + specific age for evicted blocks

age	
0	{ }
1	{a, b}
2	{ }
3	{c}
<i>evicted</i>	{e,f}

Cache analysis by Abstract Interpretation



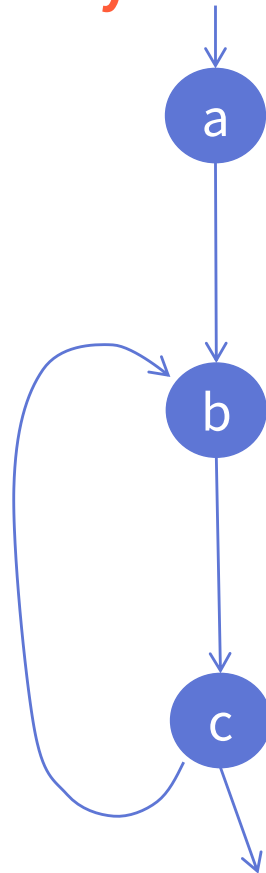
PERSISTENCE analysis

- `update()`
 - as for the MUST analysis
- `join()`
 - union & maximum age
- determines which blocks cannot be evicted once loaded in the cache

Cache analysis by Abstract Interpretation



PERSISTENCE analysis



input

{ }
{ }
{ }



output

age 0
age 1
evicted

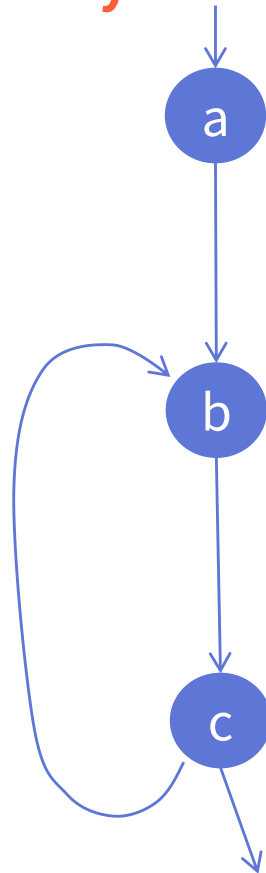




Cache analysis by Abstract Interpretation



PERSISTENCE analysis



input

{ }
{ }
{ }



output

{a}
{ }
{ }

age 0
age 1
evicted

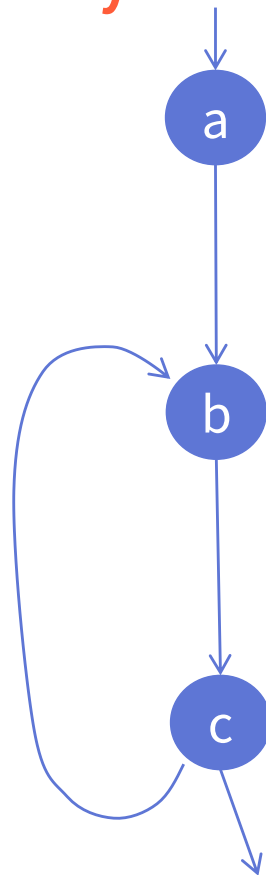




Cache analysis by Abstract Interpretation



PERSISTENCE analysis



input

{ }
{ }
{ }



output

{a}
{ }
{ }

age 0
age 1
evicted

{a}
{ }
{ }



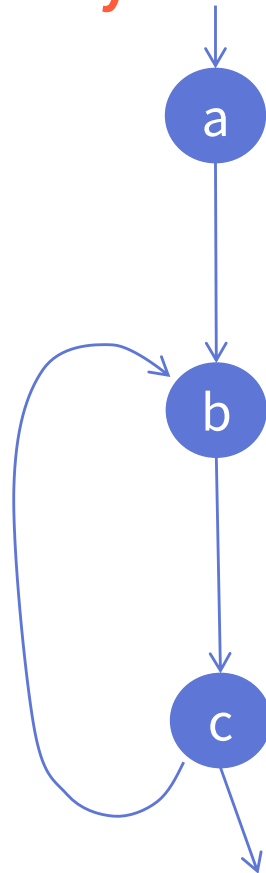
{b}
{a}
{ }



Cache analysis by Abstract Interpretation



PERSISTENCE analysis



input

{ }
{ }
{ }



output

{a}
{ }
{ }

age 0
age 1
evicted

{a}
{ }
{ }



{b}
{a}
{ }

{b}
{a}
{ }

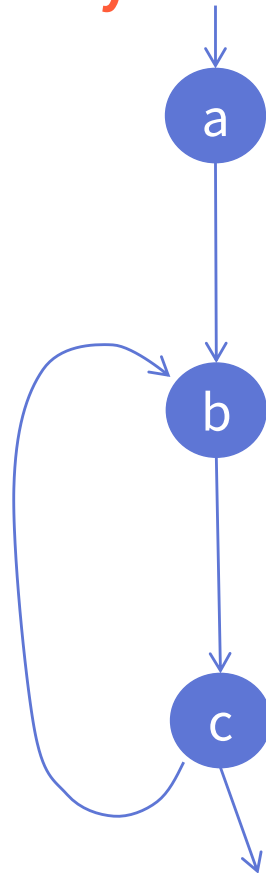


{c}
{b}
{a}

Cache analysis by Abstract Interpretation



PERSISTENCE analysis



input

{ }
{ }
{ }



output

{a}
{ }
{ }

age 0
age 1
evicted

{a}
{ }
{ }



{b}
{a}
{ }

{b}
{a}
{ }

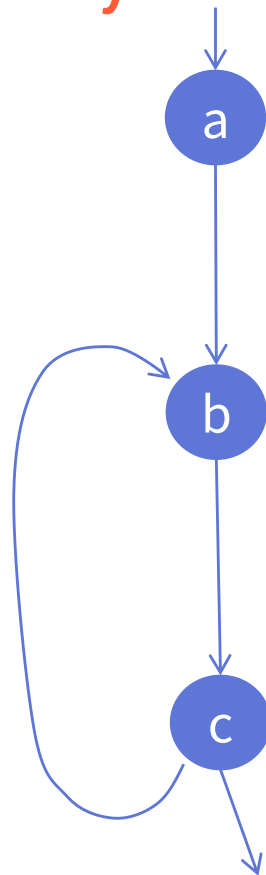


{c}
{b}
{a}

Cache analysis by Abstract Interpretation



PERSISTENCE analysis



input

{ }
{ }
{ }



output

{a}
{ }
{ }

age 0
age 1
evicted

{c}
{b}
{a}



{b}
{c}
{a }

{b}
{a}
{ }

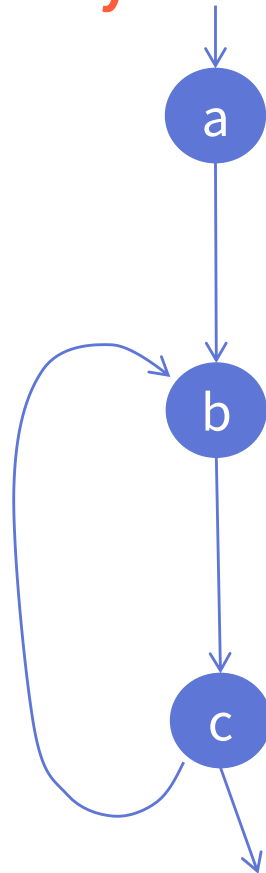


{c}
{b}
{a}

Cache analysis by Abstract Interpretation



PERSISTENCE analysis



input

{ }
{ }
{ }



output

{a}
{ }
{ }

age 0
age 1
evicted

{c}
{b}
{a}



{b}
{c}
{a}

{b}
{c}
{a}



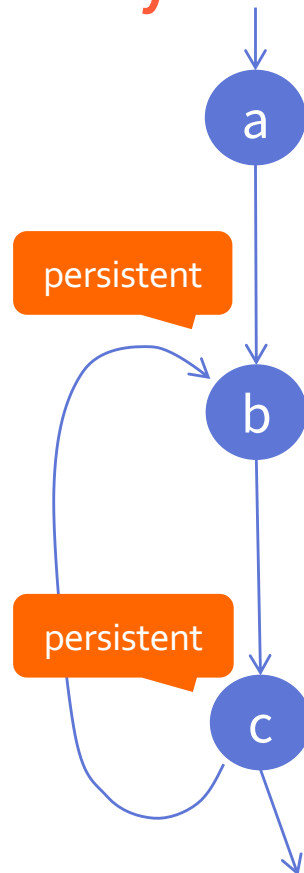
{c}
{b}
{a}

fix point!

Cache analysis by Abstract Interpretation



PERSISTENCE analysis



input

{ }
{ }
{ }



output

{a}
{ }
{ }

age 0
age 1
evicted

{c}
{b}
{a}



{b}
{c}
{a}

{b}
{c}
{a}



{c}
{b}
{a}

fix point!

Analysis of data caches



Main challenge:

- unknown and multiple memory addresses

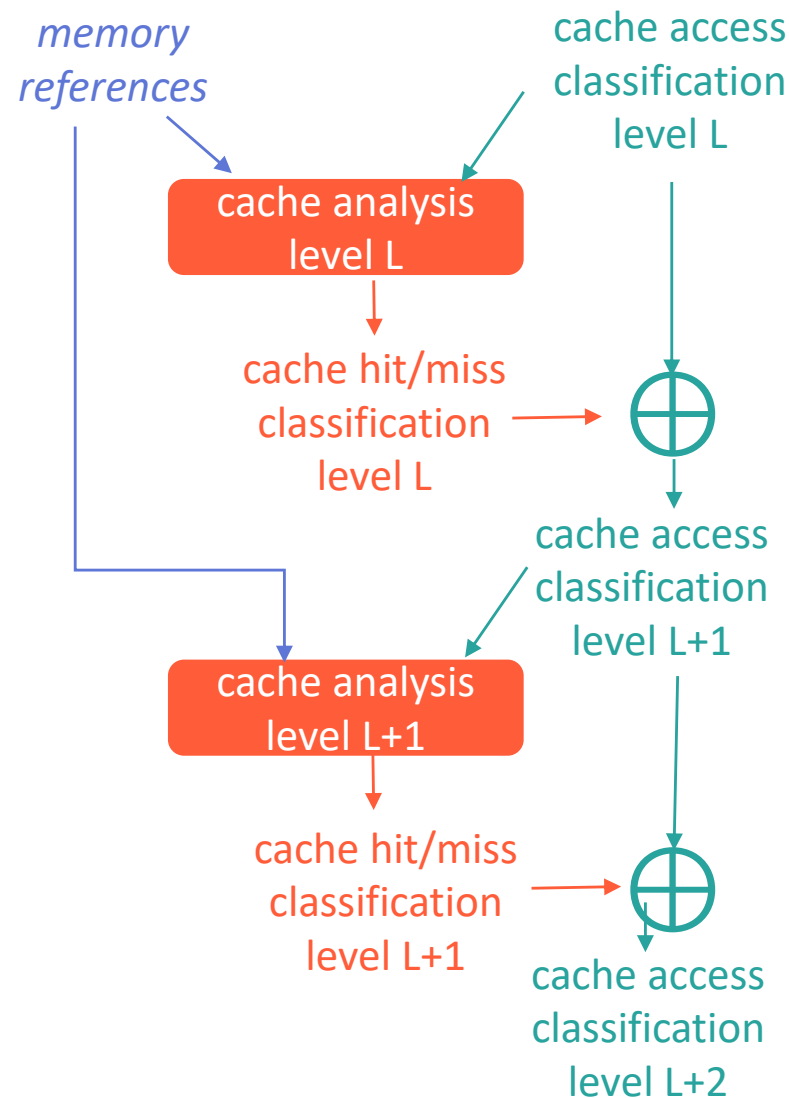
```
char a[MAX];

void isort() {
    int i,p,j,x;

    for (i=1 ; i<MAX ; i++) {
        p = 0;
        while(a[p] < a[i])
            p++;
        x = a[i];
        for (j=i-1 ; j>=p ; j--)
            a[j+1] = a[j];
        a[p] = x;
    }
}
```

```
isort:    stmfd sp!,{r0-6}
          mov  r0,#1
          adr  r10,a
for1:     cmp  r0,#MAX
          bcs  end
          mov  r1,#0
while:    ldrb r2,[r10,r1]
          ldrb r3,[r10,r0]
          cmp  r2,r3
          bcs  next1
          add  r1,r1,#1
          b    while
next1:    sub  r4,r0,#1
for2:     cmp  r4,r1
          blt  next2
          ldrb r5,[r10,r4]
          add  r6,r4,#1
          strb r5,[r10,r6]
          sub  r4,r4,#1
          b    for2
next2:    strb r3,[r10,r1]
          add  r0,r0,#1
          b    for1
end:      ldmfd sp!,{r0-6}
          mov  pc,lr
```

Analysis of a hierarchy of caches



Branch prediction



Impact of branch prediction on the execution times of basic blocks

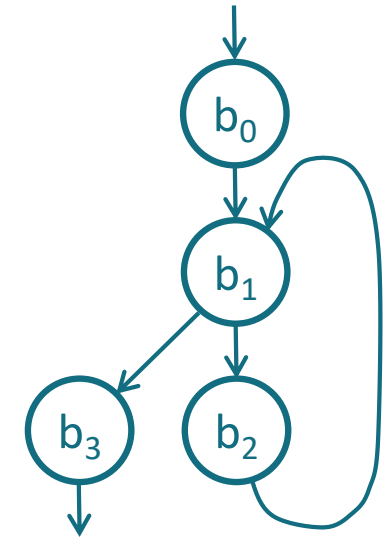
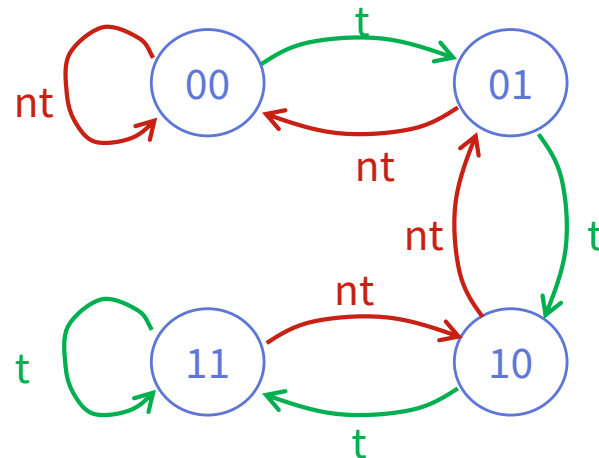


How is the direction predicted?

How is the target address predicted?

Branch History Table (BHT)

10
11
01
00
⋮
10



Branch Target Buffer (BTB)

tag	target @
⋮	⋮

Analysis of the predicted direction



Semantics-based analysis

Pattern	Initial counter state			
	00	01	10	00
T^m				
N^m				
$(TN)^m$				
$(NT)^m$				
$(N^n T)^m, n=2$				
$(N^n T)^m, n>2$				
$(T^n N)^m, n=2$				
$(T^n N)^m, n>2$				

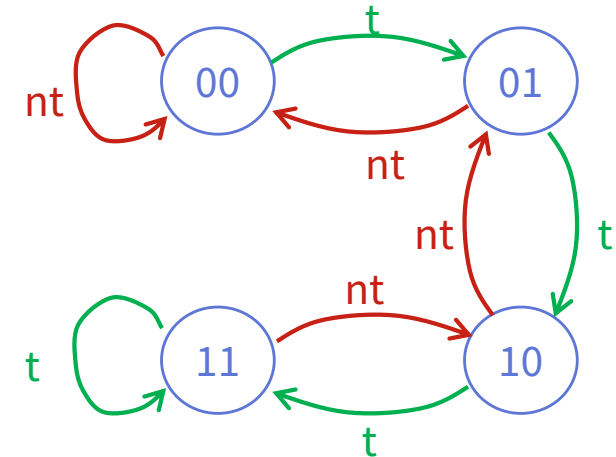
Analysis of the predicted direction



Semantics-based analysis

- $(TN)^m$ from state=01

```
for (int i=0; i<m*2; i++)  
    if (i%2 == 0)  
        ...
```



state	01							
predicted dir°								
actual dir°	T	N	T	N	T	N	T	N
mispred?								

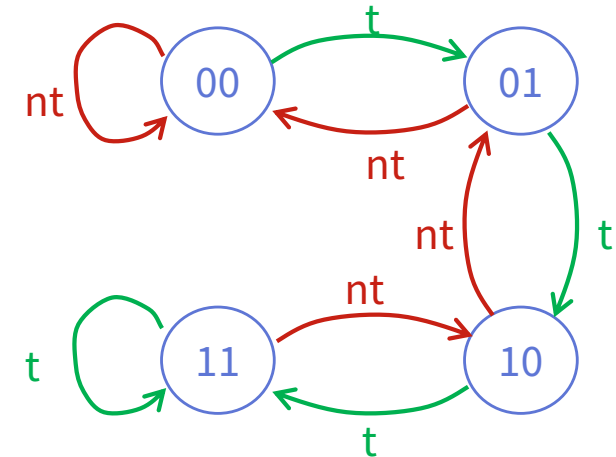
Analysis of the predicted direction



Semantics-based analysis

- $(TN)^m$ from state=01

```
for (int i=0; i<m*2; i++)  
    if (i%2 == 0)  
        ...
```



state	01	10	01	10	01	10	01	10
predicted dir°	N	T	N	T	N	T	N	T
actual dir°	T	N	T	N	T	N	T	N
mispred?	x	x	x	x	x	x	x	x

2m mispredictions

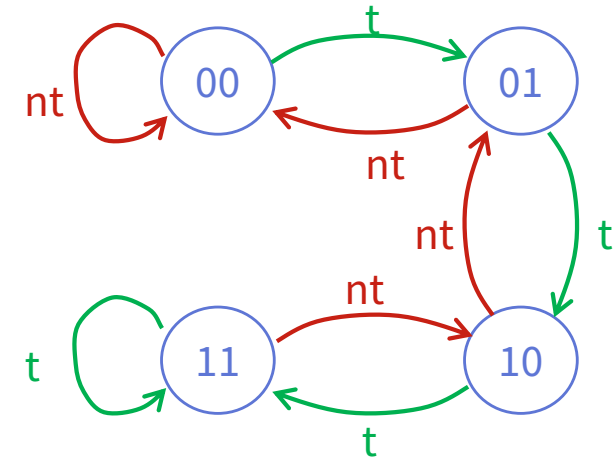
Analysis of the predicted direction



Semantics-based analysis

- $(N^nT)^m$ ($n>2$) from state=10

```
for (int i=0; i<m; i++)  
    for (int j=0; j<n ; j++)  
        ...
```



state	10									
predicted dir°										
actual dir°										
mispred?										

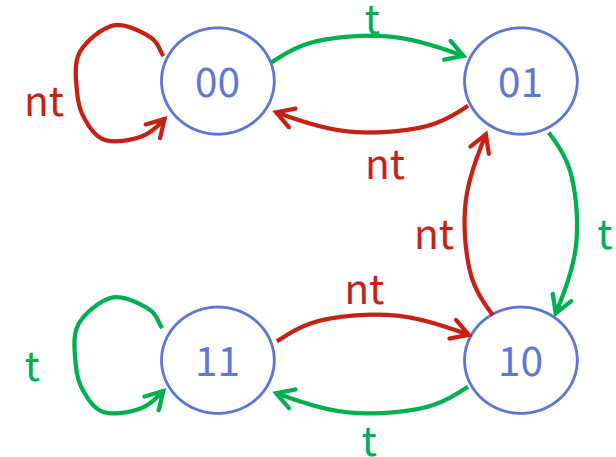
Analysis of the predicted direction



Semantics-based analysis

- $(N^nT)^m$ ($n>2$) from state=10

```
for (int i=0; i<m; i++)  
    for (int j=0; j<n ; j++)  
        ...
```



state	10	01	00	00	00	01	00	00	00	00
predicted dir°	T	N	N	N	N	N	N	N	N	N
actual dir°	N	N	N	N	T	N	N	N	N	T
mispred?	x				x					x

m+1 mispredictions

Analysis of the predicted direction



Semantics-based analysis

Pattern	Initial counter state			
	00	01	10	00
T^m	2	1	0	0
N^m	0	0	1	2
$(TN)^m$	m	2m	m	m
$(NT)^m$	m	m	2m	m
$(N^n T)^m, n=2$	m	m	1+m	3+m
$(N^n T)^m, n>2$	m	m	1+m	2+m
$(T^n N)^m, n=2$	3+m	1+m	m	m
$(T^n N)^m, n>2$	2+m	1+m	m	m